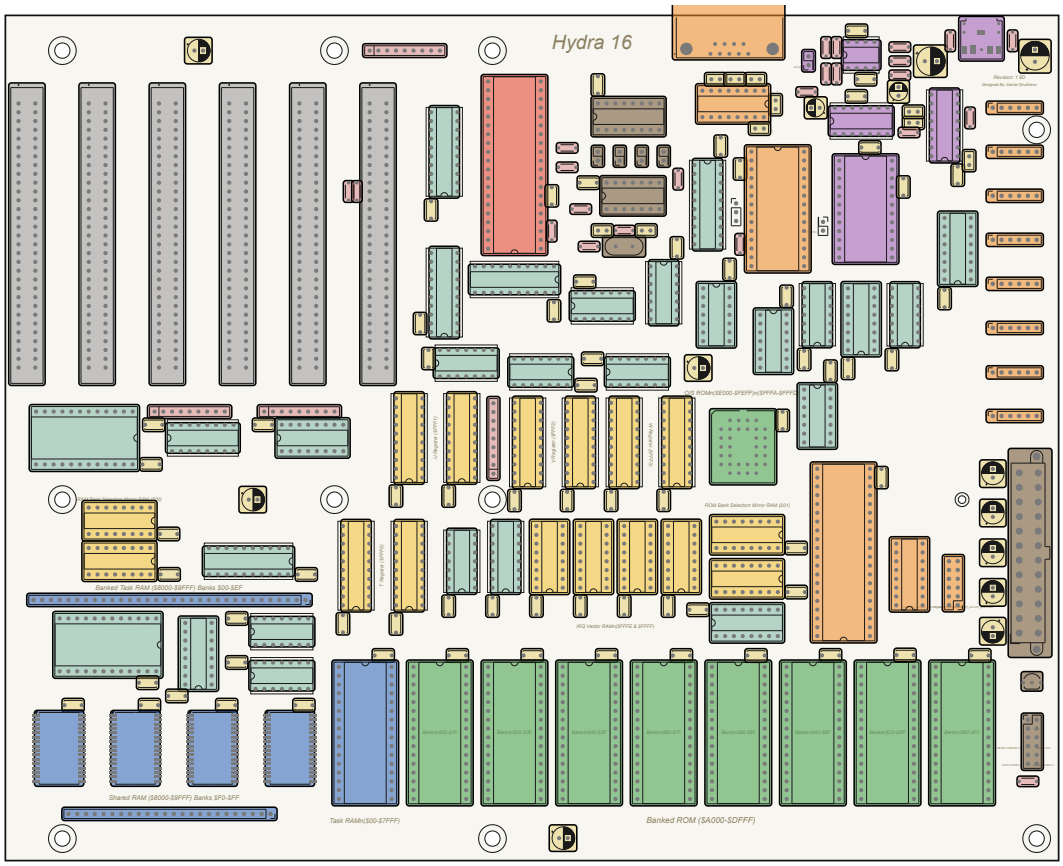
THE HYDRA PROJECT

HYDRA-16

Technical Reference



Every keyword, word, form, command and system call:
its syntax, its exact behaviour, its limits and its errors

HydraOS 1.0 · preview

About This Reference

This is the *Hydra-16 Technical Reference*, **in progress**. Its first two chapters, HyBASIC's keywords and the commands, are complete; the other three have one or two entries each so far, to show what they will be.

The *User's and Builder's Manual* teaches the Hydra, and its appendices list what each language has in a line apiece. This book is where to look when a line isn't enough: when the question is what a keyword does with an empty string, which errors a call can return, or what is left in a variable after a loop ends. It doesn't teach. It assumes the manual has been read, and it says the same thing about every entry, in the same order.

What it will hold

CHAPTER	ENTRIES	WRITTEN SO FAR
1. HyBASIC keywords	Every statement, function and operator	`ABS` to `XOR`: every keyword
2. Commands	The programs in `/bin`, and the shell's own commands	Every one
3. System calls	Every call a program can make	`OPEN`, `READ`
4. HyForth words	The core and every library	`>number`
5. hylang forms	The special forms and the functions	`let`

The other reference appendices of the manual (the 65C02's instructions, the character sets, the memory map, the errors) will follow in the same way.

How to read an entry

Every entry has the same parts, in this order. A part with nothing to say is left out.

`Syntax`
: Each form the thing can take. Words in `CAPITALS` (or, for a command or a HyForth word, as printed) are typed as they are; *italic* names stand for what you supply; `[ ]` encloses what can be left out; `{ | }` is a choice of one; `...` means the last item can repeat.

`Arguments`
: Each one: its type, its range, and what it means. For a system call, the registers in.

`Result`
: What a function returns, a word leaves on the stack, or a call leaves in the registers.

`Description`
: What happens, in the order it happens: what is evaluated and when, what is changed, where it stops.

`Errors`
: Each error it can raise, with its number and its message as the Hydra prints it, and what causes it.

Notes
: Limits, edge cases, and behaviour that differs from what another system's user would expect.

See also
: Related entries.

Example
: A transcript, taken from the Hydra. Every one in this book is replayed in the emulator when the book is built, so what is printed here is what the ROM does.

How it was written

Each entry was written from the interpreter's, the program's or the kernel's source, and each statement of behaviour in it was then tried on the system. Where the source and the trial disagreed with what one would expect of the language elsewhere (QuickBASIC, the Forth standard), the entry says what the Hydra does.

Section and chapter numbers such as "section 13.2" and "Appendix G" are the *User's and Builder's Manual*'s.

Contents

About This Reference **2**

1 HyBASIC Keywords **9**

ABS 9
ABSOLUTE 10
AND 10
APPEND 11
ARG$ 11
AS 12
ASC 12
ASM ... END ASM 13
ATN 15
BANK 16
BASE 17
BASE$ 18
BEEP 18
BINARY 19
BIT 19
BYE 19
CALL 20
CASE 21
CDBL, CSNG 21
CHDIR 22
CHR$ 22
CINT, CLNG 23
CIRCLE 24
CLEAR 25
CLOSE 26
CLS 26
COLOR 27
COMMAND$ 27
COMPLEX 28
CONST 29
CONT 30
COS 31
CSRLIN 31
CVN 32
DATA 33
DATE$ 34
DECLARE 34
DEFINT, DEFLNG, DEFSNG, DEFDBL, DEFSTR 35
DELETE 36
DENOMINATOR 36
DIGITS 37
DIM 38

DIR$ 40
DO ... LOOP 40
DRAW 42
EDIT 43
ELSE, ELSEIF 44
END 44
ENV$, ENVIRON$ 44
ENVIRON 45
EOF 46
EQV 47
ERASE 47
ERL, ERR, ERR$ 48
ERROR 49
EXIT 49
EXP 50
FIB 51
FILES 52
FIX 52
FIXED 53
FOR ... NEXT 54
FRE 56
FREEFILE 56
FUNCTION ... END FUNCTION 57
GCD 58
GET 59
GOSUB 60
GOTO 61
GPRINT 62
HEX$ 63
IF 64
IMAG 65
IMP 66
INCLUDE 66
INKEY$ 67
INPUT 68
INPUT$ 70
INSTR 71
INT 72
IS 73
KILL 73
LBOUND, UBOUND 74
LCASE$, UCASE$ 75
LEFT$, RIGHT$ 75
LEN 76
LET 77
LINE 78
LINE INPUT 79
LIST 80
LOAD 81

LOC, LOF 82
LOCATE 83
LOG 83
LOOP 84
LTRIM$, RTRIM$ 84
MID$ 85
MKDIR, RMDIR 86
MKN$ 87
MOD 88
NAME 89
NEW 90
NEXT 90
NOT 90
NUMERATOR 91
OCT$ 91
ON 91
OPEN 93
OPTION 95
OR 96
OUTPUT 96
PAINT 96
PALETTE 97
PEEK, POKE 98
PI 99
PLAY 100
POINT 101
POS 101
PRESERVE 102
PRESET, PSET 102
PRINT 103
PUT 106
RANDOMIZE 107
RATIONAL 108
READ 109
REAL 110
REDIM 110
REM 111
RESTORE 111
RESUME 112
RETURN 113
RND 113
RREG 114
RUN 115
SAVE 116
SCREEN 116
SEEK 118
SELECT CASE 119
SGN 120
SHARED 121

SHELL 122
SHELL$ 123
SHL, SHR 124
SIN 124
SLEEP 125
SOUND 126
SPACE$ 127
SPC 128
SPRITE 128
SQR 129
STATIC 130
STATUS 131
STEP 131
STOP 131
STR$ 131
STRING$ 132
SUB ... END SUB 133
SWAP 134
SYS 135
SYSTEM 136
TAB 137
TAN 137
THEN 138
TIME$, TIMER 138
TO 138
TYPE ... END TYPE 139
UNTIL 140
USING 140
VAL 140
VIEW 141
WHILE ... WEND 142
WIDTH 143
WINDOW 144
WRITE 144
XOR 146

2 Commands 147

as 147
basic 148
calc 149
cat 150
cmp 151
cp 151
date 153
db 154
df 154
dis 155
du 155
echo 156
ed 157

edit 158
forth 158
free 159
fsck 159
grep 160
head 162
hello 162
hwtest 163
hylang 163
kill 164
label 164
ls 165
mkdir 166
mkfs 167
mods 167
more 168
mv 168
new-window 169
ns 170
play 171
ps 172
pwd 172
rc 173
rm 174
rmdir 175
slay 175
sleep 176
sort 176
tail 177
tee 178
top 179
touch 179
uniq 179
wc 180
xd 181
xmodem 181
The system's own programs 182
The shell's own commands 183

3 System Calls 186
OPEN 186
READ 188

4 HyForth Words 191
>number 191

5 hylang Forms 193
let 193

CHAPTER 1

HyBASIC Keywords

HyBASIC compiles a program whole before it runs it: every line is read, every name resolved and every block matched, and only then does the first statement execute. An error the compiler finds (a `FOR` with no `NEXT`, a label that isn't there) is therefore reported before anything has run, with the line it was found on. A line typed at the prompt without a number is compiled and run the same way, as a program of one line.

An error is reported as its message, in small letters. `ERR` then holds its number, `ERR$` the message and `ERL` the line; the numbers given in each entry are `ERR`'s. A failure reported by the system (a disk that is full, a name that isn't there) has HyBASIC's own number where there is one that fits (53, file not found), and otherwise 256 plus the system's code (Appendix G).

A number is an integer of any length, a rational, a decimal or a complex number, and arithmetic on them is exact (Chapter 18). A string is a sequence of bytes, each 0 to 255, up to 64K of them. A string written in a program is enclosed in double quotes and can hold any character but the double quote itself: there is no doubled form for one inside a string. `CHR$(34)` is the way to make one.

ABS

A numeric function: the absolute value of a real number.

SYNTAX

```
ABS(x)
```

ARGUMENTS

x

A numeric expression whose value is real: an integer, a rational or a decimal.

RESULT

x if *x* is zero or more, and *–x* if it is less. The result is the same kind of number as *x*, and exact: an integer gives an integer, of whatever length; a rational gives a rational in lowest terms; a decimal gives a decimal with the same digits. There is no largest value and no overflow.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex. A complex number has no sign, and `ABS` does not give its modulus
13	`type mismatch`	*x* is a string
2	`syntax error`	No argument, or more than one

NOTES

For the modulus of a complex number *z*, use `SQR(REAL(z) ^ 2 + IMAG(z) ^ 2)`.

SEE ALSO

`SGN`, `INT`, `FIX`, `REAL`, `IMAG`.

EXAMPLE

```
> PRINT ABS(-7); ABS(7); ABS(-7/2); ABS(-2.5)
 7  7  7/2  2.5
> PRINT ABS(-2 ^ 100)
 1267650600228229401496703205376
> PRINT ABS(3+4i)
illegal function call
> PRINT ERR; SQR(REAL(3+4i) ^ 2 + IMAG(3+4i) ^ 2)
 5  5
```

ABSOLUTE

Part of `CALL ABSOLUTE`, which calls machine code at an address. See `CALL`.

AND

An operator: the bitwise AND of two integers, which is also the logical AND of two truth values.

SYNTAX

```
a AND b
```

ARGUMENTS

a*, *b

Numeric expressions whose values are real.

RESULT

An integer. Each bit of it is 1 where the same bit is 1 in both *a* and *b*, and 0 elsewhere.

DESCRIPTION

Both operands are evaluated, always: there is no short cut when the first is 0. Each is then rounded to the nearest integer, a half going to the even one (1.5 and 2.5 both become 2), and the two integers are combined bit by bit.

The integers can be of any length. A negative one is taken in two's complement with as many sign bits as are needed, so –1 is all ones, and `-1 AND x` is *x* for every integer *x*.

A comparison gives –1 for true and 0 for false. Since –1 is all ones and 0 is none, `AND` of two comparisons is their logical AND, and `IF a > 0 AND b > 0 THEN` means what it says. `AND` of a truth value and some other number is a bit operation still: `(1 = 1) AND 2` is 2, which counts as true, but `1 AND 2` is 0, which is false, though both of its operands count as true.

`AND` binds more tighter than `OR`, `XOR`, `EQV` and `IMP`, and less tightly than `NOT` and the comparisons: `NOT a AND b = c OR d` is `((NOT a) AND (b = c)) OR d`.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	An operand is a string
5	`illegal function call`	An operand is complex

SEE ALSO

`OR`, `XOR`, `NOT`, `EQV`, `IMP`, `BIT`, `SHL`.

EXAMPLE

```
> PRINT 12 AND 10; -1 AND 255; (1 < 2) AND (3 < 2); 1 AND 2
 8  255  0  0
> PRINT 2.5 AND 3; 1.5 AND 3; 2 ^ 40 + 5 AND 2 ^ 40 + 3
 2  2  1099511627777
> PRINT 0 AND (1 / 0)
division by zero
```

APPEND

A mode of `OPEN`: `OPEN path$ FOR APPEND AS #f` opens a file for writing at its end. See `OPEN`.

ARG$

A string function: one of the arguments the program was started with.

SYNTAX

```
ARG$(n)
```

ARGUMENTS

n

A numeric expression: which argument. 0 is the program's own name, 1 the first argument after it.

RESULT

The argument, as a string. The arguments are those of the command that started HyBASIC with a program to run: after `basic prog.bas one two`, `ARG$(0)` is `prog.bas`, `ARG$(1)` is `one` and `ARG$(2)` is `two`. An argument is passed whole, as the shell made it: one that was quoted there can hold spaces.

The empty string if there is no argument *n*: past the last one, for a negative *n*, and for every *n* when HyBASIC was started without a program (at its prompt).

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*n* is a string

NOTES

There is no count of the arguments. Read them until one is empty, or split `COMMAND$`.

SEE ALSO

`COMMAND$`, `ENVIRON$`.

EXAMPLE

```
% echo 'PRINT "["; ARG$(0); "]["; ARG$(1); "]["; ARG$(2); "]["; ARG$(3); "]"' >args.bas
% basic args.bas one 'two words'
[args.bas][one][two words][]
```

AS

Part of four statements, where it introduces a name, a number or a type: `DIM v AS type`, `OPEN path$ FOR mode AS #f`, `NAME old$ AS new$`, and a field's line in `TYPE`. See those.

ASC

A numeric function: the code of a string's first byte.

SYNTAX

```
ASC(s$)
```

ARGUMENTS

s$

A string expression of one byte or more.

RESULT

An integer from 0 to 255: the value of the first byte of *s$*. The rest of the string is not looked at. A string holds bytes, not characters of any one set, so the result is the byte itself; which character the console draws for it depends on the character set in use (Appendix E). The codes 32 to 126 are ASCII's.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*s$* is the empty string
13	`type mismatch`	The argument is a number

NOTES

`ASC` is the inverse of `CHR$` for one byte: `ASC(CHR$(n))` is *n* for every *n* from 0 to 255, including 0 (a zero byte doesn't end a string). To test a string that may be empty, test `LEN(s$)` first.

SEE ALSO

`CHR$`, `LEN`, `MID$`, `VAL`.

EXAMPLE

```
> PRINT ASC("A"); ASC("Abc"); ASC(" "); ASC(CHR$(0) + "x"); ASC(CHR$(255))
 65  65  32  0  255
> PRINT ASC("")
illegal function call
```

ASM ... END ASM

A block: 65C02 assembly language inside a program, assembled when the program is compiled.

SYNTAX

```
ASM
    assembly language lines
END ASM

CALL ASM label [, a [, x [, y]]]
```

ARGUMENTS

label

A label defined in one of the program's `ASM` blocks, spelled as it is there. The assembler's names keep their case: `Add1` and `add1` are two labels.

a, x, y

Numeric expressions: the values the A, X and Y registers hold as the routine is entered, a byte each. Each is 0 if it is left out.

DESCRIPTION

`ASM` alone on a line starts a block and `END ASM` alone on a line ends it. The lines between are not HyBASIC: they are in the language of the `as` assembler (Chapter 27), with its labels, directives and expressions. A program can have several blocks; they are assembled together, as one source, in the order they appear, so a label in one can be used in another.

The blocks are assembled when the program is compiled, before it runs. Their code goes into a RAM bank of its own, 8K long, which is at `$8000` to `$9FFF` while the code runs. Statements don't run "through" a block: when control reaches `ASM`, it goes on after `END ASM`.

`CALL ASM label` calls the code at *label* as a subroutine (`jsr`), which returns with `rts`. `RREG` then gives the registers it returned with.

The assembler is given some of the program's names as symbols. A global numeric variable's name is the address of its value: five bytes, the first its kind (0 for an integer) and the next four an integer's value, low byte first. An integer `CONST`'s name is its value. Each is defined in capitals and in small letters. The names `A`, `X` and `Y` are left out, since they are the registers'.

ERRORS

ERR	MESSAGE	CAUSE
90	`ASM:` and the assembler's message	An error in a block, reported at its line when the program is compiled
90	`ASM:` *label*`: no such label in an ASM block`	`CALL ASM` names a label that no block defines

NOTES

The code's bank is at `$8000` only while the code runs, so the code can't use `BANK`'s bank at the same time, and a `PEEK` of `$8000` from HyBASIC doesn't see the code.

A program can name at most 64 different labels in its `CALL ASM` statements.

The code runs as part of HyBASIC's task, with the whole machine open to it. A routine that doesn't return, or that writes where it shouldn't, takes HyBASIC down with it.

SEE ALSO

`CALL`, `SYS`, `RREG`, `PEEK`, `POKE`, `BANK`.

EXAMPLE

```
> 10 CALL ASM seven : RREG A : PRINT A
> 20 CALL ASM add1, 41 : RREG A : PRINT A
> 30 END
> 40 ASM
> 50 seven: lda #7
> 60        rts
> 70 add1:  inc a
> 80        rts
> 90 END ASM
> RUN
 7
 42
```

ATN

A numeric function: the arctangent of a real number.

SYNTAX

```
ATN(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The angle, in radians, whose tangent is *x*: more than $-\pi/2$ and less than $\pi/2$. `ATN(0)` is the integer 0. Any other result is a decimal, correct to the number of significant digits that `DIGITS` has set (12 unless it has been changed), whatever kind of number *x* is.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

`4 * ATN(1)` is the usual way to get π in a BASIC, but here it multiplies the rounding of `ATN(1)` by four. `PI` gives π to the digits set, directly.

SEE ALSO

`TAN`, `SIN`, `COS`, `PI`, `DIGITS`.

EXAMPLE

```
> PRINT ATN(1); ATN(0); ATN(-1); ATN(1/2)
 0.785398163397  0 -0.785398163397  0.463647609001
> DIGITS 30 : PRINT ATN(1) : DIGITS 12
 0.785398163397448309615660845820
```

BANK

A statement and a function: chooses, or reports, the RAM bank that `PEEK` and `POKE` see at `$8000`.

SYNTAX

```
BANK n
BANK()
```

ARGUMENTS

n

A numeric expression from 0 to 255: a RAM bank's number.

RESULT

`BANK()` is the number last set by `BANK`, or 0 if none has been.

DESCRIPTION

The addresses `$8000` to `$9FFF` are a window, 8K wide, onto one RAM bank at a time (Appendix F). `BANK n` sets the bank that a `PEEK` or `POKE` of an address in that range will use. It changes nothing else: the bank is put in the window for the moment of each `PEEK` and `POKE` and then taken out again, because HyBASIC keeps its own data in banks and uses the window itself.

HyBASIC takes RAM banks for its own use as it needs them, for the program's text, its strings and its arrays, and `BANK` doesn't check for them: a `POKE` into a bank HyBASIC is using damages it. A program that wants a bank to itself should take one high in the task's range, which HyBASIC reaches last if at all.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 0 or more than 255
13	`type mismatch`	*n* is a string

NOTES

n is not checked against the banks the task has. A task with three RAM modules has banks 0 to 47.

`BANK` has no effect on the bank an `ASM` block's code runs in.

SEE ALSO

`PEEK`, `POKE`, `SYS`, `ASM`.

EXAMPLE

```
> PRINT BANK() : BANK 40 : POKE &H8000, 65 : PRINT PEEK(&H8000); BANK()
 0
 65  40
> BANK 41 : POKE &H8000, 66 : BANK 40 : PRINT PEEK(&H8000)
 65
```

BASE

A statement: sets the base that numbers are written and read in.

SYNTAX

```
BASE b$
```

ARGUMENTS

b$

A string expression: a base string (sections 18.4 to 18.6). `"d"` is decimal, `"x"` hexadecimal, `"b"` binary, `"o"` octal; `"7r"` is base 7.

DESCRIPTION

From the next statement on, every number that HyBASIC converts to text is written in the base, and every number it converts from text is read in it. That covers `PRINT`, `WRITE`, `STR$` and `PRINT #`; `INPUT`, `READ` and `VAL`; and the numbers written in the program's own lines that are compiled after the statement has run, which at the prompt means the lines typed after it.

A number with a prefix says its own base whatever the setting: `&HFF` and `&B1010` are 255 and 10 in every base.

The setting stays until another `BASE`, and is `"d"` when HyBASIC starts.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*b$* isn't a base string
13	`type mismatch`	*b$* is a number

NOTES

`OPTION BASE`, which sets the lowest index of arrays, is a different statement: see `OPTION`.

To write one number in another base without changing the setting, use `HEX$`, `OCT$`, or `STR$` with a base string.

SEE ALSO

`BASE$`, `DIGITS`, `HEX$`, `OCT$`, `STR$`, `VAL`.

EXAMPLE

On the second line the 10 and the 6 are read in hex, as sixteen and six, and their sum, twenty-two, is written in hex:

```
> X = 255 : BASE "x" : PRINT X
 FF
> PRINT 10 + 6; BASE$
 16 x
> BASE "d" : PRINT X
 255
```

BASE$

A string function: the base string that `BASE` last set.

SYNTAX

```
BASE$
```

RESULT

The string given to the last `BASE` statement, as it was given; `d` if there has been none.

SEE ALSO

`BASE`.

EXAMPLE

```
> PRINT BASE$ : BASE "16r" : PRINT BASE$ : BASE "d"
d
16r
```

BEEP

A statement: sounds the terminal's bell.

SYNTAX

```
BEEP
```

DESCRIPTION

Writes the byte 7 (BEL) to standard output. A terminal sounds its bell, or flashes its window, as it is set to. If output has been sent to a file, the byte goes into the file.

SEE ALSO

`SOUND`, `PLAY`.

BINARY

A mode of `OPEN`: `OPEN path$ FOR BINARY AS #f` opens a file for reading and writing bytes at any position. See `OPEN`, `GET` and `PUT`.

BIT

A numeric function: one bit of an integer.

SYNTAX

```
BIT(x, n)
```

ARGUMENTS

x

A numeric expression whose value is real. It is rounded to the nearest integer, a half going to the even one.

n

A numeric expression: which bit. Bit 0 is the lowest, worth 1; bit *n* is worth 2 to the power *n*.

RESULT

1 if bit *n* of *x* is set, and 0 if it is clear.

A negative *x* is taken in two's complement with endless sign bits, so every bit of −1 is 1, however high. 0 if *n* is negative.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	An argument is a string
2	`syntax error`	One argument only

SEE ALSO

`AND`, `OR`, `XOR`, `SHL`, `SHR`.

EXAMPLE

```
> PRINT BIT(5, 0); BIT(5, 1); BIT(5, 2); BIT(-1, 100); BIT(2 ^ 70, 70)
 1  0  1  1  1
```

BYE

A statement: leaves HyBASIC. `BYE [n]` is another spelling of `SYSTEM [n]`. See `SYSTEM`.

CALL

A statement: calls a `SUB`, machine code at an address, or a label in an `ASM` block.

SYNTAX

```
[CALL] name [(] [argument [, argument] ...] [)]
CALL ABSOLUTE (address)
CALL ASM label [, a [, x [, y]]]
```

ARGUMENTS

name

The name of a `SUB` defined anywhere in the program.

argument

An expression, one for each of the `SUB`'s parameters, of the parameter's type.

address

A numeric expression, 0 to 65,535: where the machine code is.

DESCRIPTION

A `SUB`. The word `CALL` is optional, and so are the parentheses round the arguments: `Show "x", 2`, `CALL Show("x", 2)` and `CALL Show "x", 2` are the same call. The `SUB` runs, and the program goes on with the statement after the call.

An argument that is a variable, or an array's element, is passed **by reference**: the parameter is another name for it, and an assignment to the parameter changes it. Any other expression is passed by value. To pass a variable by value, put it in parentheses of its own, which makes it an expression: `Inc (X)`.

`CALL ABSOLUTE` calls the machine code at *address* as a subroutine, with the A, X and Y registers 0. It is `SYS address` by QuickBASIC's name. `RREG` gives the registers afterwards. See `SYS`.

`CALL ASM` calls a label of an `ASM` block. See `ASM`.

ERRORS

ERR	MESSAGE	CAUSE
35	`subprogram not defined`	No `SUB` has that name. Found when the program is compiled
37	`argument-count mismatch`	More arguments, or fewer, than the `SUB` has parameters
13	`type mismatch`	A string where the parameter is a number, or a number where it is a string

NOTES

A `SUB` can be called before the line that defines it: the whole program is compiled before any of it runs.

A `FUNCTION` is not called with `CALL`. It is used in an expression.

SEE ALSO

`SUB`, `FUNCTION`, `DECLARE`, `SYS`, `ASM`, `RREG`.

EXAMPLE

```
> 10 SUB Inc (N) : N = N + 1 : END SUB
> 20 X = 1 : Inc X : PRINT X
> 30 CALL Inc((X)) : PRINT X
> 40 DIM A(3) : A(1) = 5 : CALL Inc(A(1)) : PRINT A(1)
> RUN
 2
 2
 6
```

CASE

Part of `SELECT CASE`: `CASE` starts each of its choices. See `SELECT`.

CDBL, CSNG

Numeric functions: their argument, unchanged.

SYNTAX

```
CDBL(x)
CSNG(x)
```

ARGUMENTS

x

A numeric expression.

RESULT

x. In QuickBASIC these convert to double and to single precision. HyBASIC has one kind of numeric variable, which holds any number exactly, so there is nothing to convert: a rational stays a rational, and no digits are lost or gained. They are here so that programs written for QuickBASIC run.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*x* is a string

SEE ALSO

`CINT`, `FIXED`, `RATIONAL`, `DIGITS`.

EXAMPLE

```
> PRINT CDBL(1/3); CSNG(2.5); CDBL(7)
 1/3  2.5  7
```

CHDIR

A statement: changes the current directory.

SYNTAX

```
CHDIR path$
```

ARGUMENTS

path$

A string expression: the directory's name. One that doesn't start with `/` is taken from the current directory.

DESCRIPTION

Sets the current directory of HyBASIC's task, which is where a file name that doesn't start with `/` is looked for by `OPEN`, `LOAD`, `SAVE`, `FILES`, `KILL` and the rest. The change lasts until the next `CHDIR` or until HyBASIC ends. It doesn't change the directory of the shell that started HyBASIC.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no such name
76	`path not found`	The name is a file, not a directory
13	`type mismatch`	*path$* is a number

SEE ALSO

`DIR$`, `FILES`, `MKDIR`, `RMDIR`, `SHELL`.

EXAMPLE

```
> CHDIR "/ram" : OPEN "note.txt" FOR OUTPUT AS #1 : CLOSE : FILES
bin/            lib/            note.txt
> CHDIR "/nothere"
file not found
> CHDIR "/rom/bin/ls"
path not found
```

CHR$

A string function: a string of one byte.

SYNTAX

```
CHR$(n)
```

ARGUMENTS

n

A numeric expression from 0 to 255. One that isn't an integer is rounded to the nearest, a half going to the even one.

RESULT

A string one byte long, whose byte is *n*. Any value can be made, 0 included, and a string can hold any of them anywhere: `LEN(CHR$(0))` is 1.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 0 or more than 255
13	`type mismatch`	*n* is a string

NOTES

What the console shows for a byte depends on the byte and on the character set (Appendix E). The codes below 32 are controls to a terminal: `CHR$(7)` rings its bell, `CHR$(10)` starts a new line, and `CHR$(27)` begins an escape sequence (section 13.7).

SEE ALSO

`ASC`, `STRING$`, `SPACE$`.

EXAMPLE

```
> PRINT CHR$(72) + CHR$(105); LEN(CHR$(0)); ASC(CHR$(65.5)); ASC(CHR$(66.5))
Hi 1  66  66
> PRINT CHR$(256)
illegal function call
```

CINT, CLNG

Numeric functions: a real number rounded to the nearest integer.

SYNTAX

```
CINT(x)
CLNG(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The integer nearest to *x*. A value exactly half way between two integers goes to the even one: 2.5 gives 2, 3.5 gives 4, and −2.5 gives −2. This is the rounding QuickBASIC's `CINT` does.

The two functions are the same. An integer here can be of any length, so there is no range to pass, where QuickBASIC's `CINT` overflows beyond 32,767.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

`INT` (rounds down), `FIX` (rounds toward zero), `FIXED`.

EXAMPLE

```
> PRINT CINT(2.5); CINT(3.5); CINT(-2.5); CINT(7/2); CINT(2.4); CINT(-2.6)
 2  4 -2  4  2 -3
> PRINT CLNG(2 ^ 40 + 1/2)
 1099511627776
```

CIRCLE

A statement: draws a circle, or a filled disc, on the bitmap.

SYNTAX

```
CIRCLE [STEP] (x, y), r [, c [, start [, end [, aspect [, F]]]]]
```

ARGUMENTS

x, y
: Numeric expressions: the centre (see `SCREEN` for coordinates). With `STEP`, they are an offset from the last point referenced.

r
: A numeric expression, 0 or more: the radius.

c
: A numeric expression, 0 to 255: the colour. A mode with fewer colours uses the number's low bits. If it is left out, the pen's colour.

start, end, aspect
: Accepted, evaluated and ignored. QuickBASIC draws arcs and ellipses with them; HyBASIC always draws a whole circle.

F
: The letter F: fill the circle, making a disc.

DESCRIPTION

Needs a graphics mode. The radius is in pixels, rounded to the nearest; with a `WINDOW` in force it is in the window's x units. The centre becomes the last point referenced.

A radius of 0 draws the centre's one dot.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set; or *c* is not 0 to 255
258	`invalid argument`	The driver refused it: a negative radius, or a centre far off the bitmap
13	`type mismatch`	An argument is a string

NOTES

In HydraOS 1.0, a circle drawn while a `WINDOW` is in force is filled whether or not `F` is given. This is a fault, not a feature.

SEE ALSO

`SCREEN`, `COLOR`, `PSET`, `LINE`, `PAINT`, `WINDOW`, `POINT`.

EXAMPLE

With a Vera X card. (`SCREEN` clears the console, so the graphics examples are shown as they were typed, not as replayed transcripts.)

```
> SCREEN 13 : CIRCLE (100, 100), 20, 200
> PRINT POINT(120, 100); POINT(100, 80); POINT(100, 100)
 200  200  0
> CIRCLE (100, 100), 10, 3, , , , F : PRINT POINT(100, 100); POINT(105, 100)
 3  3
> SCREEN 0
```

CLEAR

A command of the prompt: forgets every variable.

SYNTAX

```
CLEAR
```

DESCRIPTION

Every variable and array of the program is dropped, so that each is 0, empty or not yet made when it is next used. The program's lines stay. A program that was stopped can no longer be continued with `CONT`.

`CLEAR` is one of the commands that work only when typed at the prompt, alone on its line. `RUN` clears the variables itself as it starts.

ERRORS

ERR	MESSAGE	CAUSE
89	`only at the prompt`	`CLEAR` in a program's line, or after other statements on a line typed at the prompt

SEE ALSO

`NEW`, `RUN`, `ERASE`, `CONT`.

CLOSE

A statement: closes files.

SYNTAX

```
CLOSE [[#]f [, [#]f] ...]
```

ARGUMENTS

f

A numeric expression: the number of a file that `OPEN` opened.

DESCRIPTION

Each file named is closed, and its number is free to be used by another `OPEN`. With no numbers, every open file is closed.

Closing a number that is not open is not an error: nothing happens. That holds for any number, 0 and numbers above 255 included.

What a program has written with `PRINT #`, `WRITE #` or `PUT` is in the file as each statement finishes, not held back until `CLOSE`; closing only gives up the file.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*f* is a string

NOTES

`RUN` closes every file as it starts, and files are closed when HyBASIC ends.

SEE ALSO

`OPEN`, `FREEFILE`.

CLS

A statement: clears the screen.

SYNTAX

```
CLS
```

DESCRIPTION

Clears the console's text and puts the cursor at the top left. In a graphics mode (`SCREEN` 1 or more) the bitmap is cleared to colour 0 as well, and the last point referenced becomes its middle.

If output is not a console, the bytes that would clear one (an escape sequence, section 13.7) go where output goes.

SEE ALSO

`LOCATE`, `SCREEN`, `COLOR`.

COLOR

A statement: sets the colours of text, or in a graphics mode the colour drawn with.

SYNTAX

```
COLOR [fg] [, bg]
```

ARGUMENTS

fg

A numeric expression: the foreground colour.

bg

A numeric expression: the background colour.

DESCRIPTION

With no graphics mode (`SCREEN 0`), `COLOR` sets the colours of the text written from then on. Each is 0 to 15: 0 to 7 are the terminal's eight colours (black, red, green, yellow, blue, magenta, cyan, white), and 8 to 15 are their bright forms. One that is left out is not changed. The colours are set by writing an escape sequence (section 13.7), so what they look like is the terminal's choice.

In a graphics mode (`SCREEN` 1 or more), *fg* sets the pen: the colour used by the drawing statements when they are given none. *bg* is accepted and ignored.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	A text colour that isn't 0 to 15
13	`type mismatch`	An argument is a string

SEE ALSO

`SCREEN`, `PALETTE`, `CLS`, `PSET`, `LINE`, `CIRCLE`.

COMMAND$

A string function: the arguments the program was started with, as one string.

SYNTAX

```
COMMAND$
```

RESULT

The arguments after the program's name, joined with one space between each, 255 bytes at most. The empty string if there were none, or if HyBASIC was started without a program.

An argument that held spaces of its own can't be told from two arguments here. `ARG$` gives each one whole.

SEE ALSO

`ARG$`, `ENVIRON$`.

EXAMPLE

```
% echo 'PRINT "["; COMMAND$; "]"' >cmd.bas
% basic cmd.bas one 'two words'
[one two words]
% basic cmd.bas
[]
```

COMPLEX

A numeric function: a complex number made from its two parts.

SYNTAX

```
COMPLEX(re, im)
```

ARGUMENTS

re, im

Numeric expressions whose values are real: the real part and the imaginary part.

RESULT

The number *re* + *im* i. The parts keep their kinds: a rational real part with a decimal imaginary part is a complex number of exactly those. If *im* is 0 the result is the real number *re*, not a complex number with nothing in its imaginary part.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*re* or *im* is itself complex
13	`type mismatch`	An argument is a string
2	`syntax error`	One argument only

NOTES

A complex number can also be written directly, as `1+2i` or `0.5i`: digits with an `i` after them are an imaginary number. The `i` must follow a number. `1+i` is 1 plus the variable `I`.

SEE ALSO

REAL, IMAG, SQR.

EXAMPLE

```
> PRINT COMPLEX(1, 2); COMPLEX(1/2, 0.5); COMPLEX(3, 0); COMPLEX(0, 1) ^ 2
 1+2i  1/2+0.5i  3 -1
> PRINT COMPLEX(1+2i, 3)
illegal function call
```

CONST

A statement: gives names to constant values.

SYNTAX

```
CONST name = value [, name = value] ...
```

ARGUMENTS

name

A name that has not been used: a string's if it ends in `$`, otherwise a number's.

value

A constant expression of the name's type: numbers and strings written out, other constants, and operators and built-in functions applied to them.

DESCRIPTION

The value is worked out when the program is compiled, and the name stands for it from then on. A constant can be used wherever a value can, and it is known in every `SUB` and `FUNCTION` without being `SHARED`.

A constant can't be assigned to, and its name can't be given to anything else.

ERRORS

ERR	MESSAGE	CAUSE
10	`duplicate definition`	*name* is already a constant or a variable; or an assignment to a constant
87	`not a constant`	*value* uses a variable, an array or a `FUNCTION`
13	`type mismatch`	A string for a numeric name, or a number for a string name

NOTES

An integer constant's name is also a symbol in the program's `ASM` blocks, where it stands for its value.

SEE ALSO

DIM, LET, ASM.

EXAMPLE

```
> CONST LIMIT = 5, UNIT$ = "cm", TWICE = LIMIT * 2 : PRINT LIMIT; UNIT$; TWICE
 5 cm 10
> LIMIT = 6
duplicate definition
> V = 3 : CONST LATER = V
not a constant
```

CONT

A command of the prompt: goes on with a program that was stopped.

SYNTAX

```
CONT
```

DESCRIPTION

A program stops, with its state kept, at a STOP statement or when CTRL-C is pressed. CONT carries on from the statement after the one it stopped at, with every variable as it is now: variables can be looked at, and changed, at the prompt in between.

A program can no longer be continued once it has ended, once one of its lines has been changed, added or removed, or after NEW, LOAD or CLEAR.

ERRORS

ERR	MESSAGE	CAUSE
17	cannot continue	No program is stopped
89	only at the prompt	CONT in a program's line

SEE ALSO

STOP, END, RUN.

EXAMPLE

```
> 10 PRINT "one" : STOP : PRINT "two"
> 20 PRINT "three"
> RUN
one
Break in line 10
> CONT
two
three
> CONT
cannot continue
```

COS

A numeric function: the cosine of a real number.

SYNTAX

```
COS(x)
```

ARGUMENTS

x

A numeric expression whose value is real: an angle in radians.

RESULT

The cosine of *x*, from −1 to 1. `COS(0)` is the integer 1. Any other result is a decimal, correct to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

An angle in degrees is turned into radians by multiplying by `PI / 180`.

`PI` is itself rounded to the digits set, so `COS(PI / 2)` is not exactly 0. It is a very small number: the cosine of the angle that `PI / 2` really is.

SEE ALSO

`SIN`, `TAN`, `ATN`, `PI`, `DIGITS`.

EXAMPLE

```
> PRINT COS(0); COS(1); COS(PI); COS(60 * PI / 180)
 1  0.540302305868 -1  0.5
```

CSRLIN

A numeric function: the row the cursor is on.

SYNTAX

```
CSRLIN
```

RESULT

The cursor's row, counted from 1 at the top of the screen. 0 if HyBASIC has no console (its input or output is a file or a pipe).

DESCRIPTION

HyBASIC does not keep track of the row. It asks the terminal each time, by writing a cursor position request (section 13.7) and reading the terminal's reply, so the answer is right whatever has been written to the screen, by this program or another.

NOTES

The column is `POS(0)`.

SEE ALSO

`POS`, `LOCATE`.

CVN

A numeric function: the number that a string of bytes holds, in the stored format.

SYNTAX

```
CVN(s$)
```

ARGUMENTS

s$

A string expression: a number's bytes, as `MKN$` makes them and as `PUT` writes a numeric variable to a file.

RESULT

The number. It is exactly the number that was stored, of the same kind: `CVN(MKN$(x))` is *x* for every number *x*.

DESCRIPTION

A number's stored form is as long as the number needs: a small integer takes one byte, a rational or a long integer more.

It replaces QuickBASIC's `CVI`, `CVL`, `CVS` and `CVD`, as `MKN$` replaces `MKI$` and the rest: there is one kind of number, so one pair of functions.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*s$* is not a number's stored form
13	`type mismatch`	*s$* is a number

NOTES

A string that was never a number may still be some number's stored form, and then there is no error: `CVN("x")` is –8.

SEE ALSO

MKN$, GET, PUT.

EXAMPLE

```
> PRINT CVN(MKN$(1/3)); CVN(MKN$(-7)); CVN(MKN$(2+3i)); LEN(MKN$(7)); LEN(MKN$(1/3))
 1/3 -7  2+3i  1  3
```

DATA

A statement: holds values in the program for READ to take.

SYNTAX

```
DATA item [, item] ...
```

ARGUMENTS

item

A value, written out. It is not an expression: nothing in it is evaluated.

DESCRIPTION

DATA does nothing when control passes over it. Its items are collected when the program is compiled: all the DATA statements of the program, in the order of its lines, make one list, and READ takes the next item from that list each time it fills a variable. RESTORE moves the place back.

An item is the text between two commas, with the spaces before and after it dropped. Text in double quotes is taken as it stands, commas and spaces included, without the quotes. An empty item, two commas with nothing between them, is allowed.

What an item means is decided by the variable that reads it. Read into a string variable, it is its text. Read into a numeric variable, it is read as a number in any form a program's numbers may take (12, -7/2, 2.5, &HFF); text that isn't a number, and an empty item, give 0.

ERRORS

ERR	MESSAGE	CAUSE
4	out of DATA	A READ when every item has been taken. The error is READ's, at its line

NOTES

Because the items aren't expressions, DATA 2 + 2 holds the five characters 2 + 2: nothing adds them up.

A DATA statement can be anywhere in the program, before or after the READ that uses it.

SEE ALSO

READ, RESTORE.

EXAMPLE

```
> 10 DATA 12, hello there , "a, b", 1/2, , &HFF
> 20 READ A, B$, C$, D, E$, F
> 30 PRINT A; "["; B$; "]["; C$; "]"; D; "["; E$; "]"; F
> 40 READ G
> RUN
 12 [hello there][a, b] 1/2 [] 255
line 40: out of DATA
```

DATE$

A string function: today's date.

SYNTAX

```
DATE$
```

RESULT

A string of ten characters, `yyyy-mm-dd`: the year, the month and the day of the system's clock. On a Hydra with no clock chip, the clock starts at 2000-01-01 when the system starts.

ERRORS

ERR	MESSAGE	CAUSE
73	`not done yet`	An assignment to `DATE$`. QuickBASIC sets the date that way; HyBASIC doesn't set the clock

NOTES

The form is not QuickBASIC's `mm-dd-yyyy`. Strings in this form sort in date order.

SEE ALSO

`TIME$`, `TIMER`.

DECLARE

A statement: accepted, and ignored.

SYNTAX

```
DECLARE SUB name [(parameters)]
DECLARE FUNCTION name [(parameters)]
```

DESCRIPTION

QuickBASIC needs a `DECLARE` for a procedure that is used before it is defined. HyBASIC compiles the whole program before it runs any of it, and finds every `SUB` and `FUNCTION` first, so a declaration tells it nothing. The statement is read past without being checked: its parameter list need not match the procedure's.

It is here so that programs written for QuickBASIC load unchanged.

SEE ALSO

`SUB`, `FUNCTION`.

DEFINT, DEFLNG, DEFSNG, DEFDBL, DEFSTR

Statements: set the type of names by their first letter.

SYNTAX

```
DEFSTR letters [, letters] ...
DEFINT letters [, letters] ...
```

ARGUMENTS

letters

One letter, or a range of letters with a hyphen between: `S`, `A-C`.

DESCRIPTION

A name normally belongs to a string if it ends in `$` and to a number if it doesn't. After `DEFSTR`, a name with no `$` whose first letter is one of those given is a string's too.

`DEFINT`, `DEFLNG`, `DEFSNG` and `DEFDBL` all do the opposite, and are the same statement: they make names that start with those letters numbers' again. (In QuickBASIC they choose between four kinds of number. HyBASIC has one.)

The statement takes effect where it stands in the program's text, for the lines after it, not when control reaches it.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	A number assigned to a name that `DEFSTR` has made a string's, or the reverse

SEE ALSO

`DIM`, `OPTION`.

EXAMPLE

```
> 10 DEFSTR S-T : S = "str" : T1 = "t" : PRINT S; T1
> 20 DEFINT S : S2 = 5 : PRINT S2
> RUN
strt
 5
```

DELETE

A command of the prompt: removes lines from the program.

SYNTAX

```
DELETE from[-to]
```

ARGUMENTS

from, to
: Line numbers.

DESCRIPTION

The lines numbered *from* to *to*, both included, are removed. With *from* alone, that one line is. Numbers in the range that no line has are passed over, so a range can name lines that aren't there, and a `DELETE` that finds none is not an error.

A program that was stopped can't be continued after a `DELETE`.

Typing a line number with nothing after it removes that one line, which is usually quicker.

ERRORS

ERR	MESSAGE	CAUSE
2	`syntax error`	No line number; or a range with no *from*
89	`only at the prompt`	`DELETE` in a program's line

SEE ALSO

`LIST`, `NEW`, `EDIT`.

EXAMPLE

```
> 10 PRINT 1
> 20 PRINT 2
> 30 PRINT 3
> 40 PRINT 4
> DELETE 20-30
> LIST
10 PRINT 1
40 PRINT 4
```

DENOMINATOR

A numeric function: the denominator of a real number written as a fraction in lowest terms.

SYNTAX

```
DENOMINATOR(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

A positive integer. A rational gives its denominator, with any common factor already taken out: 6/8 is kept as 3/4, so it gives 4. An integer gives 1. A decimal is turned into the fraction it exactly equals first: 0.75 is 3/4, and gives 4.

`NUMERATOR(x) / DENOMINATOR(x)` is *x* for every real *x*; the sign is the numerator's.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

`NUMERATOR`, `RATIONAL`, `FIXED`, `GCD`.

EXAMPLE

```
> PRINT NUMERATOR(-6/8); DENOMINATOR(-6/8); DENOMINATOR(7); NUMERATOR(0.75);
DENOMINATOR(0.75)
-3  4  1  3  4
```

DIGITS

A statement and a function: sets, or reports, how many significant digits the mathematical functions work to.

SYNTAX

```
DIGITS n
DIGITS
```

ARGUMENTS

n

A numeric expression from 1 to 100, rounded to the nearest integer.

RESULT

`DIGITS`, used as a function, is the current setting: 12 when HyBASIC starts.

DESCRIPTION

Addition, subtraction, multiplication and division of integers, rationals and decimals are exact, and `DIGITS` has nothing to do with them. It governs the functions whose results generally can't be written exactly: `SQR`, `EXP`, `LOG`, `SIN`, `COS`, `TAN`, `ATN`, `PI` and a power with a fractional exponent. Each of those

returns a decimal rounded to *n* significant digits. `RND` returns a decimal of *n* digits too.

A result that happens to be exact comes back exact whatever the setting: `SQR(16)` is the integer 4.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 1 or more than 100
13	`type mismatch`	*n* is a string

NOTES

A value that was worked out at one setting keeps the digits it has when the setting changes. To cut a number to some number of decimal places, use `FIXED`.

SEE ALSO

`BASE`, `FIXED`, `PI`, `SQR`, `RND`.

EXAMPLE

```
> PRINT DIGITS; SQR(2); 1/3
 12  1.41421356237  1/3
> DIGITS 5 : PRINT SQR(2); PI; 1/3 : DIGITS 12
 1.4142  3.1416  1/3
> DIGITS 0
illegal function call
```

DIM

A statement: makes arrays, and declares variables with a type.

SYNTAX

```
DIM [SHARED] name[(bounds [, bounds] ...)] [AS type] [, name ...] ...
```

where *bounds* is

```
upper
lower TO upper
```

ARGUMENTS

name

The array's or the variable's name.

lower, upper

Numeric expressions, each rounded to the nearest integer: the lowest and the highest index in that dimension. If *lower* is left out it is 0, or 1 after `OPTION BASE 1`. An array can have from 1 to 8 dimensions.

type

INTEGER, LONG, SINGLE, DOUBLE or NUMBER, which all mean a number; STRING; or the name of a TYPE, for a record. Without AS, the type is the name's own: a string if it ends in $.

DESCRIPTION

An array is made when the statement runs, so its bounds can be worked out by the program. Every element starts as 0, or as the empty string. DIM A(3) has the four elements A(0) to A(3).

An array that is used without having been made is made on the spot, with indexes 0 to 10 in each dimension it is used with (from 1 after OPTION BASE 1). DIM is needed only for other sizes.

A variable with no bounds is declared with its type: DIM Count AS INTEGER, DIM P AS Point. A numeric type is a statement of intent only. Every numeric variable holds any number, so a variable declared INTEGER can be given 1/3 and keeps it.

SHARED makes the names known inside every SUB and FUNCTION as well as in the main program: see SHARED.

ERRORS

ERR	MESSAGE	CAUSE
10	duplicate definition	The array has been made already. ERASE it first, or use REDIM
9	subscript out of range	A negative *upper*; *lower* above *upper*; more than 8 dimensions. Also: an index outside an array's bounds, wherever the array is used
7	out of memory	No room for the array
13	type mismatch	A bound that is a string

NOTES

A name followed by parentheses is taken to be an array's unless it is a FUNCTION's. A misspelled function name is therefore not an error: it is a new array, whose elements are all 0.

SEE ALSO

REDIM, ERASE, OPTION, LBOUND, UBOUND, TYPE, SHARED, STATIC.

EXAMPLE

```
> DIM A(3) : A(3) = 7 : PRINT A(0); A(3); LBOUND(A); UBOUND(A)
 0  7  0  3
> A(4) = 1
subscript out of range
> DIM A(5)
duplicate definition
> DIM C(1 TO 3, -2 TO 2) : C(3, -2) = 9 : PRINT C(3, -2); LBOUND(C, 2); UBOUND(C, 2)
 9 -2  2
> B(10) = 1 : PRINT UBOUND(B)
 10
> DIM Q AS INTEGER : Q = 1/3 : PRINT Q
 1/3
```

DIR$

A string function: the names in a directory, one at a time.

SYNTAX

```
DIR$(path$)
DIR$
```

ARGUMENTS

path$

A string expression: a directory's name.

RESULT

`DIR$(path$)` is the first name in the directory, and each `DIR$` after it, with no argument, is the next. A name that is itself a directory has a `/` after it. The empty string when there are no more, and for every call after that. The names come in the directory's own order, which is not alphabetical.

The empty string, too, if *path$* is a file and not a directory.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no such name as *path$*
13	`type mismatch`	*path$* is a number

NOTES

path$ names a directory. It is not a pattern: QuickBASIC's `DIR$("*.BAS")` has no meaning here. To pick out some of the names, test each one as it comes.

SEE ALSO

`FILES`, `CHDIR`, `OPEN`.

EXAMPLE

```
> CHDIR "/ram" : MKDIR "sub" : OPEN "a.txt" FOR OUTPUT AS #1 : CLOSE
> N$ = DIR$("/ram") : DO WHILE N$ <> "" : PRINT N$; " "; : N$ = DIR$ : LOOP : PRINT
bin/ lib/ sub/ a.txt
```

DO ... LOOP

A statement block: a loop that repeats while, or until, a condition holds.

SYNTAX

```
DO [{WHILE | UNTIL} condition]
    [statements]
LOOP

DO
    [statements]
LOOP [{WHILE | UNTIL} condition]
```

ARGUMENTS

condition

A numeric expression. It holds if its value is not 0.

DESCRIPTION

With the condition after `DO`, it is tested before each pass, the first included, so the statements may not run at all. With the condition after `LOOP`, it is tested after each pass, so the statements run at least once.

`WHILE` goes on for as long as the condition holds. `UNTIL` goes on for as long as it does not.

With no condition in either place the loop repeats for ever, until `EXIT DO` leaves it, an error stops it, or CTRL-C is pressed. `EXIT DO` leaves the innermost `DO` loop at once, whatever its condition, and goes on after its `LOOP`.

ERRORS

ERR	MESSAGE	CAUSE
80	`block without its start or end`	A `DO` with no `LOOP`, or a `LOOP` with no `DO`; a condition in both places; the loop and another block overlapping. Found when the program is compiled
13	`type mismatch`	The condition is a string

NOTES

At the prompt a loop must be whole on its line, since each line typed is compiled alone.

`WHILE` ... `WEND` is the same as `DO WHILE` ... `LOOP`.

SEE ALSO

`EXIT`, `WHILE`, `FOR`.

EXAMPLE

```
> I = 0 : DO : I = I + 1 : LOOP UNTIL I = 3 : PRINT I
 3
> I = 5 : DO WHILE I < 3 : I = I + 1 : LOOP : PRINT I
 5
> I = 5 : DO : I = I + 1 : LOOP WHILE I < 3 : PRINT I
 6
```

DRAW

A statement: draws with a string of pen movements, QuickBASIC's turtle language.

SYNTAX

```
DRAW commands$
```

ARGUMENTS

commands$

A string expression: movement commands, one after another. Spaces between them are optional, and capitals and small letters are the same.

DESCRIPTION

Needs a graphics mode. Drawing starts at the last point referenced, and each move draws a line from where the pen is to where it goes, in the pen's colour, and leaves the pen there.

COMMAND	MOVES THE PEN
`U` *n*, `D` *n*, `L` *n*, `R` *n*	Up, down, left, right by *n* units
`E` *n*, `F` *n*, `G` *n*, `H` *n*	Diagonally: up and right, down and right, down and left, up and left, *n* units each way
`M` *x*, *y*	To the point (*x*, *y*)
`M` +*x*, *y* or `M` –*x*, *y*	By that much from where it is: a sign on the first number makes both numbers offsets

n is 1 if it is left out. A unit is one pixel to begin with.

BEFORE A MOVE	EFFECT
`B`	Move without drawing
`N`	Draw, then put the pen back where it started

SETTING	EFFECT
`C` *n*	The colour to draw in. It becomes the pen's colour, and stays so after the `DRAW`
`S` *n*	The scale: a unit is *n* / 4 pixels. 4 to begin with
`A` *n*	Turn every direction by *n* × 90 degrees, anticlockwise: *n* is 0 to 3
`TA` *n*	Turn every direction by *n* degrees, anticlockwise

The scale and the turn stay in force for the `DRAW` statements that follow, until they are set again. A line that runs off the bitmap is drawn up to its edge.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set; or a letter in the string that is not a command
13	`type mismatch`	The argument is a number

NOTES

QuickBASIC's `X`, which runs another string, and `P`, which paints, are not here. Join strings with `+`, and use `PAINT`.

SEE ALSO

`LINE`, `PSET`, `COLOR`, `PAINT`, `SCREEN`.

EXAMPLE

A square of side 10, and a line at 45 degrees:

```
> SCREEN 13 : DRAW "BM150,100 C5 R10 D10 L10 U10"
> PRINT POINT(155, 100); POINT(160, 105); POINT(150, 105); POINT(155, 105)
 5  5  5  0
> DRAW "BM150,150 C7 TA45 R10" : PRINT POINT(157, 143)
 7
> SCREEN 0
```

EDIT

A command of the prompt: opens the program in the screen editor.

SYNTAX

```
EDIT [line]
```

ARGUMENTS

line

A line number or a label: where the editor's cursor starts.

DESCRIPTION

The program's text is written to a file in `/ram`, and the screen editor, `edit` (Chapter 11), is started on it with the cursor at *line*. With no *line*, the cursor starts at the line of the last error, if there has been one.

When the editor is left, the file is read back as the program and compiled, and the first error found, if there is one, is reported with its line. `EDIT` again goes to it.

ERRORS

ERR	MESSAGE	CAUSE
89	`only at the prompt`	`EDIT` in a program's line

SEE ALSO

LIST, LOAD, SAVE, DELETE.

ELSE, ELSEIF

Parts of IF: ELSE begins the statements for a condition that didn't hold, and ELSEIF tests another. CASE ELSE is the last choice of a SELECT CASE. See IF and SELECT.

END

A statement: ends the program. Also the first word of the lines that close a block.

SYNTAX

```
END
END {IF | SELECT | SUB | FUNCTION | TYPE | ASM}
```

DESCRIPTION

END alone stops the program and returns to the prompt, or, when HyBASIC was started with a program to run, ends HyBASIC. Nothing is printed. A program also ends when control runs off its last line, so END is needed only to stop sooner, typically before the lines of a subroutine that GOSUB reaches.

A program that has ended can't be continued with CONT. To stop in a way that can be, use STOP.

The two-word forms each close the block of that name: see IF, SELECT, SUB, FUNCTION, TYPE and ASM.

SEE ALSO

STOP, SYSTEM, CONT.

EXAMPLE

```
> 10 PRINT "a" : END : PRINT "b"
> RUN
a
> CONT
cannot continue
```

ENV$, ENVIRON$

String functions: the value of a variable of the environment.

SYNTAX

```
ENVIRON$(name$)
ENV$(name$)
```

ARGUMENTS

name$

A string expression: the variable's name, in the case it has. The shell's own variables are in small letters.

RESULT

The variable's value. A value is a list of words (section 8.4); they are returned joined with one space between each. The empty string if there is no such variable.

The two names are the same function.

DESCRIPTION

The environment is the set of named values a task starts with, copied from the task that started it: for HyBASIC, usually a shell, whose variables they are: `path`, `prompt` and the rest. Each is also a file in `/env` (section 17.7).

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*name$* is a number

SEE ALSO

`ENVIRON`, `ARG$`, `COMMAND$`, `SHELL`.

EXAMPLE

```
> PRINT ENVIRON$("path"); "|"; ENV$("nosuch"); "|"
. /bin||
```

ENVIRON

A statement: sets, or removes, a variable of the environment.

SYNTAX

```
ENVIRON "name=value"
```

ARGUMENTS

A string expression: the variable's name, an equals sign, and its value.

DESCRIPTION

The variable is set in HyBASIC's own environment. The value is one word, whatever spaces are in it. Programs that HyBASIC starts afterwards with `SHELL` get a copy of the environment and see the variable; the shell that started HyBASIC does not, since a task can't change its parent's environment.

With nothing after the equals sign, the variable is removed.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No equals sign in the string
13	`type mismatch`	The argument is a number

SEE ALSO

`ENVIRON$`, `SHELL`.

EXAMPLE

```
> ENVIRON "colour=red" : PRINT ENVIRON$("colour") : SHELL "cat /env/colour; echo"
red
red
> ENVIRON "colour=" : PRINT "["; ENVIRON$("colour"); "]"
[]
```

EOF

A numeric function: whether a file has been read to its end.

SYNTAX

```
EOF(f)
```

ARGUMENTS

f

A numeric expression: the number of an open file.

RESULT

−1 (true) if there is nothing more to read from file *f*, and 0 (false) if there is. For a file opened `FOR OUTPUT` or `FOR APPEND`, always −1.

DESCRIPTION

The usual loop reads while there is something to read: `DO UNTIL EOF(1) : LINE INPUT #1, L$ : ... : LOOP`. Testing first matters, because reading at the end is an error (62, `input past end of file`), and an empty file is at its end from the start.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	File *f* isn't open
13	`type mismatch`	*f* is a string

SEE ALSO

`OPEN`, `INPUT`, `LINE INPUT`, `LOF`, `LOC`.

EXAMPLE

```
> OPEN "/ram/two.txt" FOR OUTPUT AS #1 : PRINT #1, "one" : PRINT #1, "two" : CLOSE
> OPEN "/ram/two.txt" FOR INPUT AS #1
> DO UNTIL EOF(1) : LINE INPUT #1, L$ : PRINT L$; EOF(1) : LOOP : CLOSE
one 0
two-1
```

EQV

An operator: bitwise equivalence.

SYNTAX

```
a EQV b
```

RESULT

An integer. Each bit of it is 1 where the same bit of *a* and of *b* are alike (both 0 or both 1), and 0 where they differ. It is `NOT (a XOR b)`.

For two truth values, –1 and 0, the result is true when they are the same.

The operands are evaluated, rounded and treated as `AND`'s are: see `AND`. `EQV` binds less tightly than `AND`, `OR` and `XOR`, and more tightly than `IMP`.

EXAMPLE

```
> PRINT 5 EQV 3; (1 = 1) EQV (2 = 3); 0 EQV 0
-7  0 -1
```

ERASE

A statement: drops arrays.

SYNTAX

```
ERASE array [, array] ...
```

ARGUMENTS

array

An array's name, with no parentheses.

DESCRIPTION

Each array named is dropped, and the memory it took is free again. The name is then as if it had never been used: a `DIM` can make it again with other bounds, and using it without one makes it afresh with indexes 0 to 10, every element 0 or empty.

A name that isn't an array's is passed over without an error.

SEE ALSO

DIM, REDIM, CLEAR.

EXAMPLE

```
> DIM A(3) : A(3) = 7 : ERASE A : DIM A(50) : PRINT A(3); UBOUND(A)
 0  50
```

ERL, ERR, ERR$

Functions: the line, the number and the message of the last error.

SYNTAX

```
ERR
ERR$
ERL
```

RESULT

`ERR` is the number of the most recent error, and 0 if there has been none. The numbers are QuickBASIC's where it has an error of the same meaning (5 is `illegal function call`, 53 is `file not found`); the Hydra's own start at 80. An error reported by the system that has no number of HyBASIC's is 256 plus the system's code (Appendix G).

`ERR$` is that error's message, as it is printed: in small letters, with no line number. `error 0` if there has been none.

`ERL` is the number of the line the error was on. If that line has no number, it is the number of the nearest numbered line before it; in a program with no line numbers at all, it is the line's position in its file, counted from 1. 0 if the error was in a line typed at the prompt, or if there has been none.

DESCRIPTION

The three keep their values until the next error: a statement that succeeds doesn't clear them. They are meant to be read in an error handler (`ON ERROR GOTO`), where they tell it what happened and where.

SEE ALSO

ON, RESUME, ERROR.

EXAMPLE

```
> PRINT ERR; ERL; ERR$
 0  0 error 0
> 10 X = 1 / 0
> RUN
line 10: division by zero
> PRINT ERR; ERL; ERR$
 11  10 division by zero
```

ERROR

A statement: raises an error.

SYNTAX

```
ERROR n
```

ARGUMENTS

n

A numeric expression from 1 to 255: the error's number.

DESCRIPTION

The program behaves exactly as if error *n* had happened at this statement. If an `ON ERROR GOTO` handler is in force, control goes to it, with `ERR` equal to *n* and `ERL` this line; otherwise the program stops and the error's message is printed.

If *n* is one of HyBASIC's error numbers, the message is that error's. Any other number from 1 to 255 can be used for an error of the program's own; its message is `error` and the number.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 1 or more than 255

SEE ALSO

`ON`, `RESUME`, `ERR`.

EXAMPLE

```
> 10 ON ERROR GOTO 50
> 20 ERROR 200
> 30 PRINT "after"
> 40 END
> 50 PRINT "caught"; ERR; ERL; ERR$ : RESUME NEXT
> RUN
caught 200  20 error 200
after
```

EXIT

A statement: leaves a loop or a procedure early.

SYNTAX

```
EXIT {FOR | DO | SUB | FUNCTION}
```

DESCRIPTION

`EXIT FOR` and `EXIT DO` leave the innermost loop of that kind that the statement is inside, and the program goes on with the statement after the loop's `NEXT` or `LOOP`. A `FOR` loop's variable keeps the value it has. An `EXIT DO` inside a `FOR` loop that is itself inside a `DO` loop leaves both.

`EXIT SUB` and `EXIT FUNCTION` return from the procedure at once. A function returns the value last assigned to its name, or 0 or the empty string if none has been.

ERRORS

ERR	MESSAGE	CAUSE
80	`block without its start or end`	`EXIT FOR` or `EXIT DO` that is not inside such a loop
85	`not in a procedure`	`EXIT SUB` or `EXIT FUNCTION` outside a `SUB` or a `FUNCTION`

NOTES

There is no `EXIT WHILE`, and `EXIT DO` doesn't leave a `WHILE` ... `WEND` loop. A loop that needs an early way out should be written with `DO`.

SEE ALSO

`FOR`, `DO`, `SUB`, `FUNCTION`.

EXAMPLE

```
> 10 FOR I = 1 TO 100 : IF I * I > 50 THEN EXIT FOR
> 20 NEXT : PRINT I
> RUN
 8
```

EXP

A numeric function: e raised to a power.

SYNTAX

```
EXP(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

e to the power *x*, where e is 2.71828... `EXP(0)` is the integer 1. Any other result is a decimal, rounded to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

There is no overflow. A large result is a decimal of that many significant digits followed by zeros, and is printed in full.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

`LOG`, `SQR`, `DIGITS`.

EXAMPLE

```
> PRINT EXP(0); EXP(1); EXP(-1); EXP(1/2)
 1  2.71828182846  0.367879441171  1.6487212707
> PRINT EXP(100)
 26881171418200000000000000000000000000000000
```

FIB

A numeric function: a Fibonacci number.

SYNTAX

```
FIB(n)
```

ARGUMENTS

n

A numeric expression whose value is an integer, 0 or more.

RESULT

The *n*th Fibonacci number, an exact integer of whatever length: `FIB(0)` is 0, `FIB(1)` is 1, and each one after is the sum of the two before it.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is negative, or is not an integer
13	`type mismatch`	*n* is a string

SEE ALSO

`GCD`.

EXAMPLE

```
> PRINT FIB(0); FIB(1); FIB(2); FIB(30); FIB(100)
 0  1  1  832040  354224848179261915075
```

FILES

A statement: lists the names in a directory.

SYNTAX

```
FILES [path$]
```

ARGUMENTS

path$

A string expression: a directory's name. The current directory if it is left out.

DESCRIPTION

The names are printed across the screen, each in a column 16 characters wide, five to a line, in the directory's own order. A name that is a directory has a `/` after it. Nothing else is shown: for sizes and dates, use `SHELL "ls -l"`.

If *path$* is a file and not a directory, nothing is listed.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no such name as *path$*
13	`type mismatch`	*path$* is a number

NOTES

path$ is a directory, not a pattern: `FILES "*.BAS"` looks for a directory of that name.

SEE ALSO

`DIR$`, `CHDIR`, `KILL`, `NAME`, `SHELL`.

EXAMPLE

```
> FILES "/rom/lib"
as/             basic/          edit/           font/           forth/
hylang/         namespace       profile         shell           windows
```

FIX

A numeric function: a real number with its fraction removed.

SYNTAX

```
FIX(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The integer part of *x*: *x* rounded toward zero. For a positive number this is `INT(x)`. For a negative number that isn't whole it is one more: `FIX(-2.5)` is −2, where `INT(-2.5)` is −3. The result is an exact integer of whatever length.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

`INT`, `CINT`, `FIXED`.

EXAMPLE

```
> PRINT FIX(2.5); FIX(-2.5); FIX(7/2); FIX(-7/2); INT(-2.5)
 2 -2  3 -3 -3
```

FIXED

A numeric function: a real number as a decimal with a given number of places.

SYNTAX

```
FIXED(x, n)
```

ARGUMENTS

x

A numeric expression whose value is real.

n

A numeric expression, 0 or more: the number of decimal places.

RESULT

A decimal: *x* with the digits after its *n*th decimal place cut off. The digits are cut, not rounded, so the result is never further from zero than *x*: `FIXED(2/3, 2)` is 0.66, and `FIXED(-2/3, 2)` is −0.66. Zeros at the end are not kept, so `FIXED(5, 2)` is 5.

DESCRIPTION

This is the way to turn a rational into a decimal. `1/3` is kept and printed as `1/3`; `FIXED(1/3, 6)` is 0.333333. *n* is not limited by `DIGITS`, which is about the mathematical functions.

To round instead of cutting, add half of the last place first: `FIXED(x + 0.005, 2)` rounds a positive *x* to two places.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	An argument is a string

SEE ALSO

`RATIONAL`, `FIX`, `CINT`, `DIGITS`, `PRINT` (`USING`).

EXAMPLE

```
> PRINT FIXED(22/7, 4); FIXED(2/3, 2); FIXED(-2/3, 2); FIXED(5, 2); FIXED(1.239, 2)
 3.1428  0.66 -0.66  5  1.23
> PRINT FIXED(1/3, 30)
 0.333333333333333333333333333333
```

FOR ... NEXT

A statement block: a loop that counts a variable from one value to another.

SYNTAX

```
FOR v = start TO limit [STEP step]
    [statements]
NEXT [v [, v] ...]
```

ARGUMENTS

v

The loop's counter: a simple numeric variable. An array's element can't be one.

start, limit, step

Numeric expressions. *step* is 1 if `STEP` is left out.

DESCRIPTION

1. *start* is assigned to *v*, and *limit* and *step* are evaluated and kept. They are evaluated this once only: a variable they were worked out from can change inside the loop without changing them.
2. The test: if *step* is zero or more and *v* is greater than *limit*, or *step* is negative and *v* is less than *limit*, the loop ends, and the program goes on after its `NEXT`. The test is made before the first pass too, so the statements may not run at all; *v* then keeps the value *start*.
3. Otherwise the statements run.
4. At `NEXT`, *step* is added to *v*, and the loop goes back to the test at step 2.

When the loop ends at the test, *v* holds the first value that failed it: after `FOR I = 1 TO 3`, `I` is 4.

The addition is exact. A loop with a fractional step, `STEP 0.1` or `STEP 1/3`, reaches its limit on the last pass, where a binary floating-point BASIC may stop one short.

The statements may assign to *v*; `NEXT` adds *step* to whatever it then holds.

`EXIT FOR` leaves the innermost loop at once, with *v* as it is.

`NEXT` with no variable closes the innermost open loop. `NEXT v` does the same, and the compiler checks that *v* is that loop's counter. `NEXT j, i` is `NEXT j : NEXT i`.

ERRORS

ERR	MESSAGE	CAUSE
26	`FOR without NEXT`	A `FOR` whose `NEXT` isn't there. Found when the program is compiled: nothing has run
1	`NEXT without FOR`	A `NEXT` with no loop open, or naming a variable that isn't the innermost loop's counter
80	`block without its start or end`	The loop and another block overlap: a `NEXT` inside a one-line `IF`, say
13	`type mismatch`	*v* is a string variable, or *start*, *limit* or *step* a string
2	`syntax error`	*v* is not a simple variable

NOTES

With a *step* of 0 the counter never changes and the loop doesn't end by itself: only `EXIT FOR`, an error or CTRL-C ends it.

At the prompt, a line is compiled alone, so a loop typed there must be whole on its line: `FOR I = 1 TO 3` by itself is `FOR without NEXT`.

SEE ALSO

`EXIT`, `DO`...`LOOP`, `WHILE`...`WEND`.

EXAMPLE

```
> FOR I = 1 TO 3 : PRINT I; : NEXT : PRINT "then"; I
 1  2  3 then 4
> FOR I = 5 TO 1 : PRINT I; : NEXT : PRINT "then"; I
then 5
> FOR X = 0 TO 1 STEP 1/3 : PRINT X; : NEXT : PRINT
 0  1/3  2/3  1
> N = 3 : FOR I = 1 TO N : N = 10 : PRINT I; : NEXT : PRINT
 1  2  3
> FOR I = 1 TO 2 : FOR J = 1 TO 2 : PRINT I; J; : NEXT J, I : PRINT
 1  1  1  2  2  1  2  2
> FOR I = 1 TO 2 : FOR J = 1 TO 2 : NEXT I
NEXT without FOR
```

FRE

A numeric function: how much memory is free.

SYNTAX

```
FRE()
FRE(x)
```

ARGUMENTS

x

Any expression. It is evaluated and ignored; QuickBASIC's forms `FRE(0)` and `FRE("")` both work.

RESULT

The number of bytes left for the program's data: its strings, its arrays and its numbers too long for a variable's own five bytes. The figure is the room in the RAM banks HyBASIC has not yet used, so it falls as arrays are made and strings grow, and rises when they are dropped.

NOTES

With three RAM modules a new HyBASIC has about 300K free.

SEE ALSO

`DIM`, `ERASE`, `CLEAR`.

EXAMPLE

The figure itself depends on the machine. An array of 5,001 numbers takes five bytes for each, and twelve for the array:

```
> A = FRE() : DIM BIG(5000) : PRINT A - FRE()
 25017
```

FREEFILE

A numeric function: a file number that is not in use.

SYNTAX

```
FREEFILE
```

RESULT

The lowest number, from 1 up, that no open file has. It is the number to give the next `OPEN` in a routine that can't know which numbers the rest of the program is using.

NOTES

`FREEFILE` finds a number. It doesn't reserve it: call it again only after the `OPEN`, or it returns the same one.

Since 8 files can be open at once and numbers go to 255, there is always a free number. When 8 are open the `OPEN` fails instead, with `too many files`.

SEE ALSO

`OPEN`, `CLOSE`.

EXAMPLE

```
> PRINT FREEFILE : OPEN "/ram/x" FOR OUTPUT AS #1 : OPEN "/ram/y" FOR OUTPUT AS #3
 1
> PRINT FREEFILE : CLOSE #1 : PRINT FREEFILE : CLOSE
 2
 1
```

FUNCTION ... END FUNCTION

A block: defines a function, which returns a value and is used in expressions.

SYNTAX

```
FUNCTION name [(parameter [, parameter] ...)] [STATIC]
    [statements]
    name = value
END FUNCTION
```

ARGUMENTS

name

The function's name. It returns a string if the name ends in `$`, and a number otherwise.

parameter

A variable's name, optionally followed by `AS type`; or an array's, written with empty parentheses.

STATIC

The function's variables keep their values from one call to the next. See `STATIC`.

DESCRIPTION

The function is called by using its name in an expression, with an argument for each parameter: `PRINT Area(3, 4)`. A function with no parameters is used as a bare name.

The value returned is whatever was last assigned to the function's name when the function ends, at `END FUNCTION` or at an `EXIT FUNCTION`. If nothing was assigned, it is 0, or the empty string.

Arguments are passed as a `SUB`'s are (see `CALL`): a variable or an array's element by reference, so that assigning to the parameter changes it; any other expression by value. To protect a variable, pass it in parentheses of its own.

The variables used inside the function are its own. A name there is a new variable, 0 or empty at each call, even if the main program has a variable of the same name; the main program's is not touched. Names declared `SHARED`, and constants, are the exceptions.

A function may call itself. Each call has its own parameters and variables.

Functions are found when the program is compiled, wherever they are in its text, so one can be used on a line before its definition. When control reaches a `FUNCTION` line in the ordinary run of the program, it skips to after `END FUNCTION`.

ERRORS

ERR	MESSAGE	CAUSE
37	`argument-count mismatch`	A call with more arguments, or fewer, than the function has parameters
10	`duplicate definition`	Two procedures of one name
13	`type mismatch`	An argument of the wrong type; a string assigned to a numeric function's name
80	`block without its start or end`	`FUNCTION` with no `END FUNCTION`
2	`syntax error`	`FUNCTION` typed at the prompt: a function can be defined only in a program

NOTES

A name followed by parentheses that is not a function's is an array's. Calling a function that doesn't exist, or misspelling one, is therefore not an error: it makes an array of that name and returns its element, 0.

SEE ALSO

`SUB`, `CALL`, `EXIT`, `SHARED`, `STATIC`, `DECLARE`.

EXAMPLE

```
> 10 FUNCTION Fact (N)
> 20 IF N <= 1 THEN Fact = 1 ELSE Fact = N * Fact(N - 1)
> 30 END FUNCTION
> 40 FUNCTION Bump (V) : V = V + 1 : X = 99 : Bump = V : END FUNCTION
> 50 PRINT Fact(5); Fact(25)
> 60 X = 1 : Z = 1 : PRINT Bump(Z); Z; Bump((Z)); Z; X
> 70 PRINT Fcat(5)
> RUN
 120  15511210043330985984000000
 2  2  3  2  1
 0
```

GCD

A numeric function: the greatest common divisor of two integers.

SYNTAX

```
GCD(a, b)
```

ARGUMENTS

a, b

Numeric expressions whose values are real. Each is rounded to the nearest integer, a half going to the even one.

RESULT

The largest integer that divides both *a* and *b* exactly. It is never negative: the signs of *a* and *b* make no difference. `GCD(a, 0)` is the size of *a*, and `GCD(0, 0)` is 0. The integers can be of any length.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	An argument is complex
13	`type mismatch`	An argument is a string
2	`syntax error`	One argument only

NOTES

The least common multiple of two positive integers is `a * b / GCD(a, b)`.

Rationals are always kept in lowest terms, so there is no need for `GCD` to reduce a fraction.

SEE ALSO

`MOD`, `NUMERATOR`, `DENOMINATOR`, `FIB`.

EXAMPLE

```
> PRINT GCD(12, 18); GCD(-12, 18); GCD(0, 5); GCD(17, 5); GCD(2 ^ 64, 2 ^ 40)
 6  6  5  1  1099511627776
```

GET

A statement: reads a value from a `BINARY` file, at a given position.

SYNTAX

```
GET [#]f, [position], v
```

ARGUMENTS

f

A numeric expression: the number of a file opened `FOR BINARY`.

position

A numeric expression: where to read, counted in bytes from 1 for the file's first byte. If it is left out (the two commas are still written) or is less than 1, the read starts where the last `GET` or `PUT` on the file ended.

v

A variable: where the value goes. Its type and, for a string, its present length decide how many bytes are read.

DESCRIPTION

A string variable is filled with as many bytes from the file as it holds now. To read ten bytes, make the variable ten bytes long first, with `SPACE$(10)` say. If the file ends first, the bytes that weren't there are left as zero bytes; the variable keeps its length.

A numeric variable reads one number in the stored format, the form `PUT` writes and `MKN$` makes. A number takes as many bytes as it needs, which the number's own first bytes say, so numbers written one after another are read back one after another without their lengths being known.

After the read, the file's position is just past what was read.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* was not opened `FOR BINARY`
62	`input past end of file`	A number was to be read, and the file had ended

NOTES

There are no records and no `FIELD`: QuickBASIC's random files are not here. `GET` and `PUT` always work in bytes, and a program that wants records of a fixed size works out each one's position itself.

SEE ALSO

`PUT`, `OPEN`, `SEEK`, `LOC`, `LOF`, `CVN`, `MKN$`, `INPUT$`.

EXAMPLE

```
> OPEN "/ram/b.dat" FOR BINARY AS #1 : A$ = "HELLO" : PUT #1, 1, A$ : N = 1/3 : PUT #1,
, N
> PRINT LOF(1) : B$ = SPACE$(3) : GET #1, 2, B$ : PRINT B$
 8
ELL
> GET #1, 6, M : PRINT M; EOF(1) : CLOSE
 1/3 -1
```

GOSUB

A statement: calls a subroutine, which is a part of the program that ends with `RETURN`.

SYNTAX

```
GOSUB {line | label}
```

ARGUMENTS

line

A line number.

label

A label: a name with a colon after it at the start of a line that has no number.

DESCRIPTION

Control goes to the line named, and the place to come back to is remembered. The next `RETURN` that runs comes back to the statement after the `GOSUB`.

Subroutines can call others, and themselves; each `RETURN` goes back to the most recent `GOSUB` that hasn't been returned from.

A subroutine is not a block: it is only lines of the program, and nothing stops control from running into them from above. The usual arrangement is to put the subroutines at the end of the program, with an `END` before them.

A subroutine shares every variable with the code that called it. A `SUB` or a `FUNCTION`, with parameters and variables of its own, is nearly always the better tool.

ERRORS

ERR	MESSAGE	CAUSE
8	`label not defined`	No line has that number or that label. Found when the program is compiled
3	`RETURN without GOSUB`	A `RETURN` ran with no `GOSUB` to go back to
81	`out of stack space`	Too many `GOSUB`s without a `RETURN`

NOTES

A `GOSUB` inside a `SUB` or `FUNCTION` can only reach lines of that same procedure.

SEE ALSO

`RETURN`, `GOTO`, `ON`, `SUB`, `FUNCTION`.

EXAMPLE

```
> 10 GOSUB 100 : PRINT "back" : END
> 100 PRINT "in the subroutine" : RETURN
> RUN
in the subroutine
back
```

GOTO

A statement: goes to another line.

SYNTAX

```
GOTO {line | label}
```

DESCRIPTION

Control goes to the first statement of the line with that number or that label, and carries on from there.

The same jump is written inside an `IF` as `IF c THEN 100` or `IF c GOTO 100`.

ERRORS

ERR	MESSAGE	CAUSE
8	`label not defined`	No line has that number or that label. Found when the program is compiled

NOTES

A label marks a line that has no number. In a program typed with line numbers, the targets are numbers; in a program written in a file without them, they are labels, and a line may still be given a number so that an old listing's `GOTO 100` works.

`GOTO` typed at the prompt can't jump into the program: a line typed there is compiled alone, and the program's lines are not known to it (`label not defined`).

SEE ALSO

`GOSUB`, `ON`, `IF`, `DO`, `EXIT`.

GPRINT

A statement: writes text on the bitmap.

SYNTAX

```
GPRINT [STEP] (x, y), text$
```

ARGUMENTS

x, y
: Numeric expressions: where the top left corner of the first character goes. With `STEP`, an offset from the last point referenced.

text$
: A string expression. Its first 40 characters are drawn.

DESCRIPTION

Needs a graphics mode. The characters are drawn as dots, in the console's font and the pen's colour.

This is text that is part of the picture. It is not the console's text, which `PRINT` writes and which lies over the bitmap in its own layer; `CLS` clears both.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set
258	`invalid argument`	The point is far off the bitmap
13	`type mismatch`	*text$* is a number

SEE ALSO

`PRINT`, `LOCATE`, `COLOR`, `SCREEN`.

HEX$

A string function: a number written in hexadecimal.

SYNTAX

```
HEX$(x)
```

ARGUMENTS

x

A numeric expression of any kind.

RESULT

The text of *x* in base 16, as `STR$(x, "x")` would give it, with no space in front. The digits above 9 are the capitals A to F.

This is the number itself written in another base, not the contents of a machine word. A negative number has a minus sign, so `HEX$(-1)` is `-1`, where QuickBASIC gives `FFFF`. A number that isn't an integer is written as what it is: 2.5 is `2.8` (two and eight sixteenths), and 1/3 is `1/3`. An integer can be of any length.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*x* is a string

NOTES

For the two's complement form of a negative number in some number of bits, mask it first: `HEX$(-1 AND &HFFFF)` is `FFFF`.

To read hexadecimal text back, put `&H` before it and use `VAL`.

SEE ALSO

`OCT$`, `STR$`, `BASE`, `VAL`.

EXAMPLE

```
> PRINT HEX$(255); " "; HEX$(-1); " "; HEX$(2 ^ 40); " "; HEX$(2.5); " "; HEX$(-1 AND
&HFFFF)
FF -1 10000000000 2.8 FFFF
```

IF

A statement: runs statements only if a condition holds. It has a one-line form and a block form.

SYNTAX

```
IF condition THEN statements [ELSE statements]
IF condition THEN line
IF condition GOTO line

IF condition THEN
    [statements]
[ELSEIF condition THEN
    [statements]] ...
[ELSE
    [statements]]
END IF
```

ARGUMENTS

condition

A numeric expression. It holds if its value is not 0. A comparison gives −1 or 0, but any number will do: 1/2 holds.

line

A line number or a label, which is jumped to.

DESCRIPTION

The one-line form is all on one line. If the condition holds, the statements after `THEN` run, up to the `ELSE` or the end of the line; if it doesn't, the statements after `ELSE` run, if there is an `ELSE`. Either part can be several statements with colons between them, and they all belong to the `IF`: nothing on the line after `THEN` runs when the condition fails, except what follows `ELSE`.

A one-line `IF` can hold another. An `ELSE` belongs to the nearest `IF` before it that has none.

The block form has nothing after `THEN` on the first line. The conditions are tested in order, the `IF`'s and then each `ELSEIF`'s, and the statements under the first one that holds are run; the rest are skipped, their conditions not evaluated. If none holds, the statements under `ELSE` run. Control then goes on after `END IF`.

Block `IF`s nest to any depth, and can hold loops and other blocks, each closed inside the part it began in.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	The condition is a string. (Two strings can be compared: the comparison is a number)
80	`block without its start or end`	A block `IF` with no `END IF`; `ELSE`, `ELSEIF` or `END IF` with no block `IF`
8	`label not defined`	`THEN` or `GOTO` with a line that isn't there

NOTES

At the prompt only the one-line form can be used, since a line typed there is compiled alone.

Both sides of an `AND` or an `OR` in a condition are always evaluated. To test one thing only if another holds, use two `IF`s.

SEE ALSO

`SELECT`, `AND`, `OR`, `NOT`, `GOTO`.

EXAMPLE

```
> 10 IF 1 THEN PRINT "a"; : PRINT "b" ELSE PRINT "c"; : PRINT "d"
> 20 IF 0 THEN PRINT "a"; : PRINT "b" ELSE PRINT "c"; : PRINT "d"
> 30 IF 5 THEN IF 0 THEN PRINT "x" ELSE PRINT "inner else"
> 40 X = 3
> 50 IF X = 1 THEN
> 60   PRINT "one"
> 70 ELSEIF X = 3 THEN
> 80   PRINT "three"
> 90 ELSE
> 100  PRINT "other"
> 110 END IF
> RUN
ab
cd
inner else
three
```

IMAG

A numeric function: the imaginary part of a number.

SYNTAX

```
IMAG(z)
```

ARGUMENTS

z

A numeric expression of any kind.

RESULT

The imaginary part of *z*, a real number of the kind it has in *z*. 0 if *z* is real.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*z* is a string

SEE ALSO

`REAL`, `COMPLEX`.

EXAMPLE

```
> PRINT IMAG(1+2i); REAL(1+2i); IMAG(5); IMAG(1/2+0.5i)
 2  1  0  0.5
```

IMP

An operator: bitwise implication.

SYNTAX

```
a IMP b
```

RESULT

An integer. Each bit of it is 0 where that bit is 1 in *a* and 0 in *b*, and 1 everywhere else. It is `(NOT a) OR b`.

For two truth values, −1 and 0, the result is false only when *a* is true and *b* is false: "if *a*, then *b*".

The operands are evaluated, rounded and treated as `AND`'s are: see `AND`. `IMP` binds less tightly than every other operator.

EXAMPLE

```
> PRINT -1 IMP 0; 0 IMP 0; 0 IMP -1; -1 IMP -1; 5 IMP 3
 0 -1 -1 -1 -5
```

INCLUDE

A statement: reads another file's lines into the program at this point.

SYNTAX

```
INCLUDE "file"
'$INCLUDE: 'file'
REM $INCLUDE: 'file'
```

ARGUMENTS

file

The file's name, written out: a literal, not an expression.

DESCRIPTION

The lines of *file* are compiled as if they were written where the `INCLUDE` line is. The statement must be on a line of its own. The second and third forms are QuickBASIC's, in which the statement is disguised as a comment; they do the same.

A name that starts with `/` is used as it is. Any other name is looked for in the directory the program itself came from, and then in `/lib/basic`.

An included file can include others, four deep. A program can include eight files in all.

An error in an included file is reported with that file's name and its own line number there.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	The file is in neither place
12	`illegal in direct mode`	`INCLUDE` typed at the prompt
2	`syntax error`	`INCLUDE` with other statements on its line

NOTES

Including is done when the program is compiled, not when control reaches the line, so a file can't be chosen by the running program. To run another program from a program, use `SHELL "basic file"`.

SEE ALSO

`LOAD`, `SHELL`, `DECLARE`.

EXAMPLE

```
% echo 'PRINT "included"' >inc.bas
% echo 'PRINT "main"' >main.bas
% echo 'INCLUDE "inc.bas"' >>main.bas
% echo 'PRINT "after"' >>main.bas
% basic main.bas
main
included
after
```

INKEY$

A string function: a key that has been pressed, without waiting for one.

SYNTAX

```
INKEY$
```

RESULT

The empty string if no key is waiting. Otherwise one key, which is taken out of the keyboard's queue:

- An ordinary key is a string of one byte, the key's character.
- A cursor key or a function key is a string of two bytes: `CHR$(0)`, then a code for the key. The codes are QuickBASIC's: 72 is up, 80 is down, 59 is F1, and so on.

DESCRIPTION

`INKEY$` never waits. The usual use is in a loop that does something else as well, or one that waits for any key: `DO : K$ = INKEY$ : LOOP UNTIL K$ <> ""`.

NOTES

A loop that only polls `INKEY$` takes all the processor time it can get. Put a `SLEEP` of a short time in it if other tasks are running.

To wait for a set number of keys, use `INPUT$`.

SEE ALSO

`INPUT$`, `INPUT`, `LINE INPUT`, `SLEEP`.

INPUT

A statement: reads a line and assigns its fields to variables. The line comes from the console, or from a file.

SYNTAX

```
INPUT [;] ["prompt" {; | ,}] v [, v] ...
INPUT #f, v [, v] ...
```

ARGUMENTS

prompt

A string in quotes, written before the line is read. It must be a literal: a string variable there is a syntax error. After it, `;` adds a question mark and a space, and `,` adds nothing. With no prompt at all, `?` is written.

v

A numeric or string variable, or an array's element. Each takes one field of the line.

f

The number of a file opened `FOR INPUT` or `FOR BINARY`.

DESCRIPTION

From the console. The prompt is written and one line is read, with the console's line editing (section 13.2). Only its first 255 bytes are used. The line is split into fields at its commas, and each variable takes a field, in order:

- A numeric variable's field is read as a number, in any form a program's own numbers may take: `12`, `-7/2`, `2.5`, `1e3`, `&HFF`. An empty field is 0.
- A string variable's field is the text up to the next comma, with the spaces before and after it dropped. A field that starts with a double quote runs to the closing quote instead, and can hold commas and spaces.

If the line has fewer fields than there are variables, or more, or a numeric field that isn't a number, HyBASIC writes `?Redo from start` and the statement starts again from its prompt. It is never an error.

From a file. One line of file *f* is read, and split and assigned in the same way, with three differences. There is no prompt and nothing is asked again: a numeric field that isn't a number is 0. If the line runs out of fields before the variables do, the next line is read and its fields carry on. Fields left on the line after the last variable is filled are dropped: the next `INPUT #` starts at the next line.

The semicolon after `INPUT` matters only when input doesn't come from the console (`basic prog <answers`). The console echoes the Return that ends a line; a file doesn't, so HyBASIC writes a new line itself after reading one. `INPUT ;` leaves it out, and what is printed next follows the prompt on the same line.

ERRORS

ERR	MESSAGE	CAUSE
62	`input past end of file`	No line is left: in file *f*, or in standard input when it isn't the console
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR OUTPUT` or `FOR APPEND`
2	`syntax error`	The prompt isn't a string literal

CTRL-C at the prompt stops the program, as it does anywhere else (`Break`).

NOTES

To read a whole line, commas and all, into one string, use `LINE INPUT`. To read keys without waiting for Return, use `INKEY$` or `INPUT$`.

SEE ALSO

`LINE INPUT`, `INPUT$`, `INKEY$`, `OPEN`, `EOF`, `READ`.

EXAMPLE

```
> 10 INPUT "Two numbers"; A, B
> 20 PRINT A + B
> 30 INPUT "Name, age: ", N$, G
> 40 PRINT "["; N$; "]"; G
> 50 INPUT X, Y$
> 60 PRINT X; "["; Y$; "]"
> RUN
Two numbers? 3
?Redo from start
Two numbers? 1/2, 2.5
 3
Name, age: Sam Smith , 12
[Sam Smith] 12
? , "x, y"
 0 [x, y]
```

Fields that carry on from one line of a file to the next:

```
> 10 OPEN "/ram/n.txt" FOR OUTPUT AS #1
> 20 PRINT #1, "1" : PRINT #1, "2, 3" : PRINT #1, "4"
> 30 CLOSE #1 : OPEN "/ram/n.txt" FOR INPUT AS #1
> 40 INPUT #1, A, B : PRINT A; B
> 50 INPUT #1, C : PRINT C
> 60 INPUT #1, D
> RUN
 1  2
 4
line 60: input past end of file
```

INPUT$

A string function: a set number of keys from the keyboard, or of bytes from a file.

SYNTAX

```
INPUT$(n)
INPUT$(n, [#]f)
```

ARGUMENTS

n

A numeric expression: how many keys or bytes.

f

A numeric expression: the number of a file opened `FOR INPUT` or `FOR BINARY`.

RESULT

From the keyboard, `INPUT$(n)` waits until *n* keys have been typed and returns them as a string of *n* bytes. Nothing is echoed and Return is a key like any other, so this is the way to read a key press, a password, or a menu choice without the user pressing Return. When standard input is not a console, it

reads *n* bytes from it instead.

From a file, `INPUT$(n, #f)` returns the next *n* bytes of file *f*, whatever they are, line ends included, and moves the file's position past them.

ERRORS

ERR	MESSAGE	CAUSE
62	`input past end of file`	The file, or standard input, ends before *n* bytes
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR OUTPUT` or `FOR APPEND`

NOTES

To read a whole file into a string, ask for its length: `INPUT$(LOF(1), #1)`.

SEE ALSO

`INKEY$`, `INPUT`, `LINE INPUT`, `GET`, `LOF`.

EXAMPLE

```
> OPEN "/ram/t.txt" FOR OUTPUT AS #1 : PRINT #1, "abcdef" : CLOSE
> OPEN "/ram/t.txt" FOR INPUT AS #1 : PRINT INPUT$(3, #1); "|"; INPUT$(2, 1); "|";
LOC(1)
abc|de| 5
> CLOSE
```

INSTR

A numeric function: where one string is found inside another.

SYNTAX

```
INSTR([start,] s$, t$)
```

ARGUMENTS

start

A numeric expression, an integer from 1 up: the position in *s$* to start looking from. 1 if it is left out.

s$

A string expression: the string to look in.

t$

A string expression: the string to look for.

RESULT

The position in *s$*, counting its first byte as 1, of the first place at or after *start* where *t$* appears whole. 0 if it appears nowhere from there on, which includes the cases where *start* is beyond the end of *s$* and where *t$* is longer than what is left.

If *t$* is empty, the result is *start*, as long as *start* is within *s$*.

The comparison is of bytes: capitals and small letters are different.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*start* is less than 1, or is not an integer
13	`type mismatch`	*s$* or *t$* is a number

NOTES

To find something without regard to case, compare capitals: `INSTR(UCASE$(s$), UCASE$(t$))`.

To find every place, call it again with *start* one past the last result.

SEE ALSO

`MID$`, `LEFT$`, `RIGHT$`, `LEN`.

EXAMPLE

```
> PRINT INSTR("hello", "l"); INSTR(4, "hello", "l"); INSTR("hello", "z"); INSTR("Hello",
"h")
 3  4  0  0
> PRINT INSTR("hello", ""); INSTR(9, "hello", "l")
 1  0
```

INT

A numeric function: a real number rounded down.

SYNTAX

```
INT(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The largest integer that is not more than *x*. For a negative number that isn't whole this is the next integer *below* it: `INT(-2.5)` is –3. The result is an exact integer of whatever length.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

The three ways to make an integer differ only for numbers that aren't whole: `INT` rounds down, `FIX` rounds toward zero, and `CINT` rounds to the nearest.

SEE ALSO

`FIX`, `CINT`, `MOD`, `FIXED`.

EXAMPLE

```
> PRINT INT(2.5); INT(-2.5); INT(7/2); INT(-7/2); INT(9)
 2 -3  3 -4  9
```

IS

Part of `SELECT CASE`: `CASE IS < 10` tests the selected value with a comparison. See `SELECT`.

KILL

A statement: removes a file.

SYNTAX

```
KILL path$
```

ARGUMENTS

path$

A string expression: the file's name. It is one name, not a pattern.

DESCRIPTION

The file is removed from its directory, and the space it took is free. There is no way to get it back.

A directory can be removed with `KILL` too, if it is empty, though `RMDIR` says more clearly what is meant.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no such file
263	`busy`	The file is open, in this program or any other
70	`permission denied`	The file is read-only, or its disk is
13	`type mismatch`	*path$* is a number

SEE ALSO

`NAME`, `RMDIR`, `FILES`, `OPEN`.

EXAMPLE

```
> OPEN "/ram/u.txt" FOR OUTPUT AS #1 : KILL "/ram/u.txt"
busy
> CLOSE : KILL "/ram/u.txt" : KILL "/ram/u.txt"
file not found
```

LBOUND, UBOUND

Numeric functions: the lowest and the highest index of an array.

SYNTAX

```
LBOUND(array [, dimension])
UBOUND(array [, dimension])
```

ARGUMENTS

array

An array's name, with no parentheses.

dimension

A numeric expression: which dimension, counting the first as 1. 1 if it is left out.

RESULT

The lowest, or the highest, index the array allows in that dimension, as `DIM` set them.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*dimension* is less than 1 or more than the array has

NOTES

If the name has not been used as an array, asking for its bounds makes it one, with the usual indexes 0 to 10: `UBOUND` of a misspelled name is 10, not an error.

A loop over a whole array is best written `FOR I = LBOUND(A) TO UBOUND(A)`, which goes on being right if the `DIM` is changed.

SEE ALSO

`DIM`, `REDIM`, `OPTION`.

EXAMPLE

```
> DIM A(2 TO 5, 7) : PRINT LBOUND(A); UBOUND(A); LBOUND(A, 2); UBOUND(A, 2)
 2  5  0  7
```

LCASE$, UCASE$

String functions: a string with its letters made small, or made capital.

SYNTAX

```
LCASE$(s$)
UCASE$(s$)
```

ARGUMENTS

s$

A string expression.

RESULT

A string of the same length. `LCASE$` turns each of the capitals A to Z into its small letter; `UCASE$` turns each of a to z into its capital. Every other byte is copied as it is, including the accented letters above 127 in the console's character sets.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*s$* is a number

SEE ALSO

`INSTR`, `LTRIM$`.

EXAMPLE

```
> PRINT LCASE$("Hello World 123"); " "; UCASE$("Hello World 123")
hello world 123 HELLO WORLD 123
```

LEFT$, RIGHT$

String functions: the first, or the last, bytes of a string.

SYNTAX

```
LEFT$(s$, n)
RIGHT$(s$, n)
```

ARGUMENTS

s$

A string expression.

n

A numeric expression, 0 or more: how many bytes. One that isn't an integer is rounded to the nearest, a half going to the even one.

RESULT

`LEFT$` is the first *n* bytes of *s$*, and `RIGHT$` the last *n*. If *s$* has fewer than *n* bytes, the result is all of it. If *n* is 0, the result is the empty string.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is negative
13	`type mismatch`	*s$* is a number, or *n* a string

SEE ALSO

`MID$`, `LEN`, `INSTR`.

EXAMPLE

```
> PRINT "["; LEFT$("hello", 2); "]["; RIGHT$("hello", 2); "]["; LEFT$("hello", 9); "][";
RIGHT$("hi", 0); "]"
[he][lo][hello][]
```

LEN

A numeric function: the length of a string.

SYNTAX

```
LEN(s$)
```

ARGUMENTS

s$

A string expression.

RESULT

The number of bytes in *s$*: 0 for the empty string. Every byte counts, zero bytes and line ends included.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*s$* is a number. `LEN` does not give the size of a numeric variable

NOTES

`LEN = n` at the end of an `OPEN` statement is a different thing: see `OPEN`.

SEE ALSO

`LEFT$`, `MID$`, `INSTR`, `STR$`.

EXAMPLE

```
> PRINT LEN("hello"); LEN(""); LEN(CHR$(0) + CHR$(0)); LEN(STRING$(1000, "x"))
 5  0  2  1000
```

LET

A statement: gives a variable a value.

SYNTAX

```
[LET] variable = expression
```

ARGUMENTS

variable

A variable, an array's element, a record or one of its fields.

expression

An expression of the variable's type.

DESCRIPTION

The expression is evaluated, and its value becomes the variable's. The word `LET` is optional and changes nothing.

A numeric variable takes any number as it is: an integer of any length, a rational, a decimal or a complex number. Nothing is rounded or converted on the way in. A string variable takes a string of any length up to 64K.

Assigning to a function's own name, inside the function, sets the value it will return.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	A string for a numeric variable, or a number for a string variable
10	`duplicate definition`	The name is a constant's
9	`subscript out of range`	An array's element that isn't there

NOTES

= is also the comparison "is equal to". Which one is meant depends on where it stands: the first = of an assignment assigns, and any after it compare, so `A = B = C` gives `A` the truth of `B = C`.

A line typed at the prompt that starts with a number is taken as a line of the program, whatever follows the number.

SEE ALSO

`CONST`, `DIM`, `SWAP`, `READ`.

EXAMPLE

```
> B = 2 : C = 2 : A = B = C : PRINT A : A = B = 3 : PRINT A
-1
 0
> LET N = 2 ^ 100 / 3 : PRINT N
 12676506002282294014967032053 76/3
```

LINE

A statement: draws a line, a box, or a filled box. (`LINE INPUT`, which reads a line of text, is the next entry.)

SYNTAX

```
LINE [[STEP] (x1, y1)] - [STEP] (x2, y2) [, [c] [, {B | BF}]]
```

ARGUMENTS

x1, y1
: Numeric expressions: where the line starts. If the first point is left out, the line starts at the last point referenced: `LINE -(x, y)` carries on from where the last statement stopped.

x2, y2
: Numeric expressions: where it ends.

STEP
: Before either point: its coordinates are offsets from the last point referenced. For the second point, that is the first point.

c
: A numeric expression, 0 to 255: the colour. The pen's colour if it is left out.

B, BF
: `B` draws the box that has the two points as opposite corners, in outline. `BF` draws it filled.

DESCRIPTION

Needs a graphics mode. Both end points are drawn. The second point becomes the last point referenced, so a chain of `LINE -(x, y)` statements draws a connected path.

A line that goes off the bitmap is drawn as far as the edge.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set; or *c* is not 0 to 255
13	`type mismatch`	An argument is a string

SEE ALSO

`PSET`, `CIRCLE`, `DRAW`, `PAINT`, `COLOR`, `SCREEN`.

EXAMPLE

```
> SCREEN 13 : LINE (0, 0)-(10, 0), 3 : LINE -(10, 10), 4 : PRINT POINT(5, 0); POINT(10,
5)
 3  4
> LINE (30, 30)-(40, 40), 6, B : PRINT POINT(30, 35); POINT(35, 35)
 6  0
> LINE (50, 50)-(60, 60), 7, BF : PRINT POINT(55, 55)
 7
> SCREEN 0
```

LINE INPUT

A statement: reads a whole line into a string variable.

SYNTAX

```
LINE INPUT [;] ["prompt";] v$
LINE INPUT #f, v$
```

ARGUMENTS

prompt

A string in quotes, written before the line is read. Nothing is added to it: no question mark.

v$

A string variable, or an array's element.

f

The number of a file opened `FOR INPUT` or `FOR BINARY`.

DESCRIPTION

One line is read and assigned to *v$* as it stands: commas, quotes and spaces at either end are all kept, where `INPUT` would split the line at its commas and trim it. The line's end is not part of the string. A line longer than 255 bytes is cut to 255.

From the console the line is typed with the console's line editing (section 13.2). From a file, the next line of the file is read, and the file's position moves past its line end.

The semicolon after `INPUT` has the meaning it has for `INPUT`: it matters only when input is not a console.

ERRORS

ERR	MESSAGE	CAUSE
62	`input past end of file`	No line is left: in file *f*, or in standard input when it isn't the console
13	`type mismatch`	The variable is numeric
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR OUTPUT` or `FOR APPEND`

NOTES

`LINE` with an opening parenthesis after it is the graphics statement that draws a line.

SEE ALSO

`INPUT`, `INPUT$`, `EOF`, `OPEN`.

EXAMPLE

```
> Q$ = CHR$(34) : OPEN "/ram/t.txt" FOR OUTPUT AS #1 : PRINT #1, "  two, "; Q$; "words";
Q$; "  " : CLOSE
> OPEN "/ram/t.txt" FOR INPUT AS #1 : LINE INPUT #1, L$ : PRINT "["; L$; "]"; EOF(1)
[  two, "words"  ]-1
> LINE INPUT #1, L$
input past end of file
```

LIST

A command of the prompt: shows the program's lines.

SYNTAX

```
LIST [from][-[to]]
```

ARGUMENTS

from, to

Line numbers or labels.

DESCRIPTION

TYPED	LISTS
`LIST`	The whole program
`LIST 100`	Line 100 alone
`LIST 100-`	From line 100 to the end
`LIST -100`	From the start to line 100
`LIST 100-200`	Lines 100 to 200

A number need not be a line that exists: the lines in the range are listed, and if there are none nothing is. The lines are shown as they were typed or loaded, with their spacing and their case.

CTRL-C stops a long listing.

ERRORS

ERR	MESSAGE	CAUSE
89	`only at the prompt`	`LIST` in a program's line

SEE ALSO

`EDIT`, `DELETE`, `SAVE`, `LOAD`.

EXAMPLE

```
> 10 PRINT "one"
> 20 PRINT "two"
> 30 PRINT "three"
> LIST 20-
20 PRINT "two"
30 PRINT "three"
> LIST 15-25
20 PRINT "two"
```

LOAD

A command of the prompt: reads a program from a file, in place of the one in memory.

SYNTAX

```
LOAD "file"
```

ARGUMENTS

file

The file's name, in quotes. Nothing is added to it: a program saved as `game.bas` is loaded as `"game.bas"`.

DESCRIPTION

The program in memory is dropped first, with its variables and any files it had open, as `NEW` does; then the file's lines become the program. The file is plain text, a line of the program to each line, with or without line numbers. A carriage return before a line feed is dropped, so a file written on a PC loads as it is.

The program is not run. `RUN` runs it; `basic file`, typed at the shell, loads and runs a file in one step.

ERRORS

ERR	MESSAGE	CAUSE
288	`not found`	There is no such file. The message is preceded by the file's name
89	`only at the prompt`	`LOAD` in a program's line

NOTES

The old program is dropped before the file is opened. If the `LOAD` then fails, there is no program in memory at all: `SAVE` first.

SEE ALSO

`SAVE`, `NEW`, `RUN`, `INCLUDE`, `EDIT`.

EXAMPLE

```
> 10 PRINT "saved"
> SAVE "/ram/s.bas"
> NEW
> LOAD "/ram/s.bas"
> LIST
10 PRINT "saved"
> LOAD "/ram/nothere.bas"
/ram/nothere.bas: not found
> LIST
```

LOC, LOF

Numeric functions: an open file's position, and its length.

SYNTAX

```
LOC(f)
LOF(f)
```

ARGUMENTS

f

A numeric expression: the number of an open file, in any mode.

RESULT

`LOC(f)` is the number of bytes before the file's position: how many have been read or written so far, in a file used from start to end. It is 0 when a file has just been opened, except `FOR APPEND`, where it is the file's length.

`LOF(f)` is the file's length in bytes, as it is now, including what this program has written to it.

`SEEK(f)` is `LOC(f) + 1`: the position counted from 1, which is the form the `SEEK` and `GET` statements take.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	File *f* isn't open

SEE ALSO

`SEEK`, `EOF`, `OPEN`, `GET`, `PUT`.

EXAMPLE

```
> OPEN "/ram/l.txt" FOR OUTPUT AS #1 : PRINT #1, "abcdef" : PRINT LOC(1); LOF(1);
SEEK(1) : CLOSE
 7  7  8
```

LOCATE

A statement: moves the cursor, and shows or hides it.

SYNTAX

```
LOCATE [row] [, [column] [, cursor]]
```

ARGUMENTS

row, column

Numeric expressions, from 1: the top line of the screen is row 1 and its left edge column 1. Each is rounded to the nearest integer. One that is left out keeps its present value.

cursor

A numeric expression: 0 hides the cursor and 1 shows it.

DESCRIPTION

The cursor is moved by writing an escape sequence to standard output (section 13.7), and the next thing printed starts there. `POS(0)` follows the move.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*row* or *column* is less than 1, or more than 255
13	`type mismatch`	An argument is a string

NOTES

The screen's real size isn't checked. A position beyond the edge of the terminal is the terminal's to deal with; most stop at the edge.

SEE ALSO

`CSRLIN`, `POS`, `CLS`, `PRINT`, `TAB`.

EXAMPLE

The transcript shows what was printed, not where on the screen it went. `POS(0)` is read after `CSRLIN` has been printed, so it is 17, not 14:

```
> LOCATE 5, 10 : PRINT "here"; : PRINT CSRLIN; POS(0)
here 5  17
```

LOG

A numeric function: the natural logarithm.

SYNTAX

```
LOG(x)
```

ARGUMENTS

x

A numeric expression whose value is real and not 0.

RESULT

The logarithm of *x* to the base e. `LOG(1)` is the integer 0. Any other result is a decimal, rounded to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

For a negative *x* the result is complex: the logarithm of the size of *x*, plus π i. `LOG(-1)` is π i.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is 0, or is complex
13	`type mismatch`	*x* is a string

NOTES

A logarithm to another base is a quotient: `LOG(x) / LOG(10)` is the logarithm to base 10.

SEE ALSO

`EXP`, `SQR`, `DIGITS`.

EXAMPLE

```
> PRINT LOG(1); LOG(10); LOG(1/2); LOG(2 ^ 100); LOG(-1)
 0  2.30258509299 -0.69314718056  69.314718056  3.14159265359i
> PRINT LOG(0)
illegal function call
```

LOOP

The line that ends a `DO` loop, with or without a condition. See `DO`.

LTRIM$, RTRIM$

String functions: a string without the spaces at its start, or at its end.

SYNTAX

```
LTRIM$(s$)
RTRIM$(s$)
```

ARGUMENTS

s$

A string expression.

RESULT

`LTRIM$` is *s$* from its first byte that is not a space; `RTRIM$` is *s$* up to its last byte that is not a space. Only the space, byte 32, is removed: tabs and line ends are kept. A string that is all spaces gives the empty string.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*s$* is a number

NOTES

`STR$` puts a space in front of a number that isn't negative. `LTRIM$(STR$(n))` is the number's digits alone.

SEE ALSO

`STR$`, `LEFT$`, `RIGHT$`, `SPACE$`.

EXAMPLE

```
> PRINT "["; LTRIM$("  a b  "); "]["; RTRIM$("  a b  "); "]["; LTRIM$("   "); "]";
LTRIM$(STR$(42)); "|"
[a b  ][  a b][]42|
```

MID$

A string function, and a statement: a part of a string, taken out or replaced.

SYNTAX

```
MID$(s$, start [, length])
MID$(v$, start [, length]) = new$
```

ARGUMENTS

s$

A string expression.

v$

A string variable, or an array's element.

start

A numeric expression, 1 or more: the position of the first byte, counting the string's first byte as 1.

length

A numeric expression, 0 or more: how many bytes. If it is left out, all the rest of the string.

new$

A string expression.

Numbers that aren't integers are rounded to the nearest, a half going to the even one.

RESULT

The function is the *length* bytes of *s$* that start at *start*, or as many as there are if the string ends sooner. It is the empty string if *start* is past the end, or *length* is 0.

DESCRIPTION

The statement overwrites part of the variable's string with the bytes of *new$*, starting at *start*. It never changes the string's length. The number of bytes replaced is the smallest of three: *length* (if it is given), the length of *new$*, and what is left of *v$* from *start* to its end. Bytes of *new$* that don't fit are not used.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*start* is less than 1, or *length* is negative; in the statement, *start* is past the end of *v$*
13	`type mismatch`	A number where a string is wanted, or a string where a number is

SEE ALSO

`LEFT$`, `RIGHT$`, `INSTR`, `LEN`.

EXAMPLE

```
> PRINT "["; MID$("hello", 2, 3); "]["; MID$("hello", 4); "]["; MID$("hello", 4, 9); "]
["; MID$("hello", 9); "]"
[ell][lo][lo][]
> A$ = "hello" : MID$(A$, 2, 2) = "XYZ" : PRINT A$ : MID$(A$, 4) = "12345" : PRINT A$;
LEN(A$)
hXYlo
hXY12 5
```

MKDIR, RMDIR

Statements: make a directory, and remove one.

SYNTAX

```
MKDIR path$
RMDIR path$
```

ARGUMENTS

path$

A string expression: the directory's name.

DESCRIPTION

`MKDIR` makes a new, empty directory. The directory it is to go in must exist already: one level is made at a time.

`RMDIR` removes a directory, which must be empty.

ERRORS

ERR	MESSAGE	CAUSE
58	`file already exists`	`MKDIR`: there is a file or a directory of that name
53	`file not found`	`MKDIR`: the directory it would go in isn't there. `RMDIR`: there is no such directory
292	`directory not empty`	`RMDIR`: the directory still has something in it
70	`permission denied`	The disk can't be written

SEE ALSO

`CHDIR`, `FILES`, `KILL`, `NAME`.

EXAMPLE

```
> MKDIR "/ram/d1" : MKDIR "/ram/d1/sub" : RMDIR "/ram/d1"
directory not empty
> RMDIR "/ram/d1/sub" : RMDIR "/ram/d1" : RMDIR "/ram/d1"
file not found
```

MKN$

A string function: a number's bytes in the stored format.

SYNTAX

```
MKN$(x)
```

ARGUMENTS

x

A numeric expression of any kind.

RESULT

A string holding *x* in the form numbers are stored in: the form `PUT` writes for a numeric variable, and `CVN` reads back. Its length depends on the number. A small integer takes one byte; larger integers, rationals, decimals and complex numbers take as many as they need.

DESCRIPTION

`MKN$` and `CVN` are the pair that stands in place of QuickBASIC's `MKI$`, `MKL$`, `MKS$`, `MKD$` and their `CV` functions. `CVN(MKN$(x))` is exactly *x*, for every number.

Because the length varies, a number can't be given a fixed place in a record by `MKN$` alone. Pad the string to the width wanted, or keep the number as text.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*x* is a string

SEE ALSO

`CVN`, `PUT`, `GET`, `STR$`.

EXAMPLE

```
> PRINT LEN(MKN$(7)); LEN(MKN$(1/3)); LEN(MKN$(2 ^ 100)); CVN(MKN$(2 ^ 100))
 1  3  14  126765060022822940149670320 5376
```

MOD

An operator: the remainder of an integer division. The entry covers `\`, integer division, too.

SYNTAX

```
a MOD b
a \ b
```

ARGUMENTS

a, b

Numeric expressions whose values are real.

RESULT

Both operands are first rounded to the nearest integer, a half going to the even one. Then `a \ b` is the quotient with its fraction dropped, which rounds toward zero, and `a MOD b` is what is left over: `a - (a \ b)

- b`. The remainder has the sign of *a*, or is 0.

The integers can be of any length.

DESCRIPTION

`\` binds more tightly than `MOD`, and both less tightly than `*` and `/` and more tightly than `+` and –. So `A + B MOD C \ D` is `A + (B MOD (C \ D))`.

ERRORS

ERR	MESSAGE	CAUSE
11	`division by zero`	*b* is 0, or rounds to 0
13	`type mismatch`	An operand is a string

NOTES

Because the operands are rounded first, `7.5 MOD 2` is 0: 7.5 rounds to 8. For the remainder of a division of numbers that aren't whole, work it out: `x - INT(x / y) * y`.

`/` is exact division, and gives a rational when the integers don't divide: `7 / 2` is 7/2.

SEE ALSO

`INT`, `FIX`, `GCD`.

EXAMPLE

```
> PRINT 7 MOD 3; -7 MOD 3; 7 MOD -3; 7 \ 2; -7 \ 2; 7.5 MOD 2; 7 / 2
 1 -1  1  3 -3  0  7/2
> PRINT 2 ^ 70 MOD 1000; 2 + 3 * 4 ^ 2 MOD 5 \ 2
 424  2
```

NAME

A statement: renames a file.

SYNTAX

```
NAME old$ AS new$
```

ARGUMENTS

old$

A string expression: the file's present name.

new$

A string expression: its new name.

DESCRIPTION

The file, or directory, *old$* is given the name *new$* in the directory it is already in. Only the last part of *new$* is used, the name after its last `/`: a rename can't move a file to another directory, and a directory written in *new$* is ignored without an error. To move a file, copy it and remove the original.

Renaming a file to the name it has does nothing.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no *old$*
58	`file already exists`	There is a *new$* in that directory already
70	`permission denied`	The disk can't be written

SEE ALSO

`KILL`, `FILES`, `MKDIR`, `SHELL`.

EXAMPLE

```
> OPEN "/ram/n1" FOR OUTPUT AS #1 : CLOSE : MKDIR "/ram/dd"
> NAME "/ram/n1" AS "/ram/dd/moved" : FILES "/ram" : FILES "/ram/dd"
bin/            lib/            moved           dd/
```

NEW

A command of the prompt: drops the program.

SYNTAX

```
NEW
```

DESCRIPTION

The program's lines are removed, with everything that belonged to it: its variables and arrays, its procedures and labels, and any files it had open, which are closed. A stopped program is gone, so there is nothing to `CONT`. The settings that are HyBASIC's and not the program's stay as they are: `BASE`, `DIGITS`, the current directory.

There is no question first, and no way back. `SAVE` what is wanted.

ERRORS

ERR	MESSAGE	CAUSE
89	`only at the prompt`	`NEW` in a program's line

SEE ALSO

`LOAD`, `SAVE`, `CLEAR`, `DELETE`.

NEXT

The line that ends a `FOR` loop. See `FOR`.

NOT

An operator: the bitwise complement of an integer, which is also the logical NOT of a truth value.

SYNTAX

```
NOT a
```

RESULT

An integer: a, rounded to the nearest integer if it isn't one, with every bit turned over. In two's complement that is $-a-1$: `NOT 0` is −1, `NOT -1` is 0, and `NOT 5` is −6.

For the truth values −1 and 0, `NOT` gives the other one. For any other number it does not: 5 counts as true in an `IF`, and so does `NOT 5`.

DESCRIPTION

`NOT` binds less tightly than the comparisons and more tightly than `AND`: `NOT A = B` is `NOT (A = B)`, and `NOT A AND B` is `(NOT A) AND B`.

The operand is rounded and treated as `AND`'s are: see `AND`.

EXAMPLE

```
> PRINT NOT 0; NOT -1; NOT 5; NOT 1 = 2
-1  0 -6 -1
```

NUMERATOR

A numeric function: the numerator of a real number written as a fraction in lowest terms.

SYNTAX

```
NUMERATOR(x)
```

RESULT

An integer with the sign of *x*. A rational gives its numerator, an integer gives itself, and a decimal is turned into the fraction it exactly equals first: −0.75 is −3/4, and gives −3. See `DENOMINATOR`, which has the errors and an example.

OCT$

A string function: a number written in octal.

SYNTAX

```
OCT$(x)
```

RESULT

The text of *x* in base 8, with no space in front, as `STR$(x, "o")` would give it. As with `HEX$`, it is the number itself in another base: a negative number has a minus sign, and a number that isn't an integer is written as what it is. See `HEX$`.

EXAMPLE

```
> PRINT OCT$(8); " "; OCT$(255); " "; OCT$(-8); " "; OCT$(0.5)
10 377 -10 0.4
```

ON

A statement: jumps to one of several lines by a number; or sets where errors go.

SYNTAX

```
ON n GOTO line [, line] ...
ON n GOSUB line [, line] ...
ON ERROR GOTO line
ON ERROR GOTO 0
```

ARGUMENTS

n

A numeric expression, rounded to the nearest integer.

line

A line number or a label.

DESCRIPTION

ON n GOTO goes to the first line in the list if *n* is 1, to the second if it is 2, and so on. If *n* is less than 1 or more than the number of lines listed, nothing happens: the program goes on with the next statement. **ON n GOSUB** is the same, but calls the line as a subroutine, and its RETURN comes back to the statement after the ON.

ON ERROR GOTO line names an error handler. From then on, an error that would have stopped the program makes it go to *line* instead, with ERR, ERL and ERR$ set. The handler ends with a RESUME, which says where to carry on: at the statement that failed, at the one after it, or at another line.

While the handler is running, a second error is not sent to it again. It stops the program in the ordinary way.

ON ERROR GOTO 0 turns the handler off. Used inside a handler, it gives up: the error being handled stops the program, with its message, as if there had been no handler.

ERRORS

ERR	MESSAGE	CAUSE
8	`label not defined`	A line in the list, or the handler's, isn't in the program
13	`type mismatch`	*n* is a string

NOTES

CTRL-C is not an error, and no handler sees it.

SEE ALSO

RESUME, ERR, ERROR, GOTO, GOSUB, SELECT.

EXAMPLE

```
> 10 ON ERROR GOTO 100
> 20 X = 1 / 0 : PRINT "after the error"
> 30 OPEN "/ram/nothere" FOR INPUT AS #1
> 40 PRINT "line 40" : ON ERROR GOTO 0 : ERROR 5
> 50 PRINT "not reached"
> 100 PRINT "handler:"; ERR; ERL
> 110 IF ERR = 11 THEN RESUME NEXT
> 120 RESUME 40
> RUN
handler: 11  20
after the error
handler: 53  30
line 40
line 40: illegal function call
```

OPEN

A statement: opens a file, and gives it a number for the statements that read and write it.

SYNTAX

```
OPEN path$ FOR {INPUT | OUTPUT | APPEND | BINARY} AS [#]f [LEN = n]
```

ARGUMENTS

path$

A string expression: the file's name. A name that doesn't start with `/` is taken from the current directory (`CHDIR`). It can be any name in the program's namespace: a file on a disk, or a device such as `/dev/cons`.

f

A numeric expression from 1 to 255: the number the file is known by until it is closed. The `#` is optional.

n

Accepted, evaluated and ignored. QuickBASIC's record length has no meaning here: `GET` and `PUT` take a position in bytes.

DESCRIPTION

The mode decides how the file is opened, and which statements may then use it:

MODE	A FILE THAT EXISTS	A FILE THAT DOESN'T	STARTS AT	MAY THEN BE USED BY
`INPUT`	Opened for reading	Error 53	Its first byte	`INPUT #`, `LINE INPUT #`, `INPUT$`, `EOF`, `LOF`, `LOC`, `SEEK`
`OUTPUT`	Emptied, and opened for writing	Made	Its first byte	`PRINT #`, `WRITE #`, `LOF`, `LOC`, `SEEK`
`APPEND`	Opened for writing, its contents kept	Made	Its end	`PRINT #`, `WRITE #`, `LOF`, `LOC`, `SEEK`
`BINARY`	Opened for reading and writing	Made	Its first byte	All of the above, and `GET` and `PUT`

Up to 8 files can be open at once. A file stays open until `CLOSE` names it, `CLOSE` with no number closes them all, or a `RUN` starts (which closes every file first). Leaving HyBASIC closes them too.

`EOF(f)` is always true (–1) for a file opened `FOR OUTPUT` or `FOR APPEND`.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	*f* is less than 1 or more than 255
55	`file already open`	Number *f* is in use
67	`too many files`	8 files are open already
53	`file not found`	`FOR INPUT`: there's no such file. Any mode: a directory in the path isn't there
70	`permission denied`	The file, or its disk (`/rom`), can't be written
13	`type mismatch`	*path$* is a number, or *f* a string

The system's other refusals come through as its own errors (a full disk is 61, `disk full`).

NOTES

The limit of 8 is HyBASIC's own. Each open file also takes one of the task's 16 file descriptors, three of which are the console's.

One file can be open under two numbers at once, each with its own position: opening it `FOR INPUT` as `#2` while it is open `FOR OUTPUT` as `#1` is allowed, and reads what has been written so far.

A file written with `PRINT #` holds exactly the bytes `PRINT` would have sent to the screen, each line ended by a line feed (byte 10).

SEE ALSO

`CLOSE`, `PRINT`, `WRITE`, `INPUT`, `LINE INPUT`, `GET`, `PUT`, `SEEK`, `EOF`, `LOF`, `LOC`, `KILL`, `NAME`.

EXAMPLE

```
> OPEN "/ram/log.txt" FOR OUTPUT AS #1 : PRINT #1, "one" : PRINT LOF(1); EOF(1) : CLOSE
#1
 4 -1
> OPEN "/ram/log.txt" FOR APPEND AS #1 : PRINT #1, "two" : PRINT LOF(1) : CLOSE #1
 8
> OPEN "/ram/log.txt" FOR INPUT AS #1 : LINE INPUT #1, L$ : PRINT L$; EOF(1)
one 0
> PRINT #1, "three"
bad file mode
> OPEN "/ram/other.txt" FOR OUTPUT AS #1
file already open
> CLOSE : OPEN "/ram/nothere.txt" FOR INPUT AS #1
file not found
> PRINT ERR
 53
```

OPTION

A statement: sets the lowest index of arrays.

SYNTAX

```
OPTION BASE {0 | 1}
```

DESCRIPTION

An array made with only its highest index given, as in `DIM A(10)`, has indexes from 0. After `OPTION BASE 1` they are from 1, so that `DIM A(10)` has ten elements and no `A(0)`. `OPTION BASE 0` puts it back.

The setting applies to arrays made after the statement, whether by `DIM`, by `REDIM`, or by being used without either (whose indexes then run from 1 to 10). Arrays that exist already keep the bounds they were made with. A `DIM` that states its own lowest index, `DIM A(5 TO 9)`, is not affected.

The digit must be written out as 0 or 1: it is part of the statement, not an expression.

ERRORS

ERR	MESSAGE	CAUSE
2	`syntax error`	Anything but 0 or 1 after `BASE`

NOTES

This `BASE` has nothing to do with the `BASE` statement, which sets the base numbers are written in.

SEE ALSO

`DIM`, `REDIM`, `LBOUND`, `UBOUND`.

EXAMPLE

```
> OPTION BASE 1 : DIM A(3) : B(1) = 1 : PRINT LBOUND(A); UBOUND(A); LBOUND(B); UBOUND(B)
 1  3  1  10
> OPTION BASE 0 : DIM C(3) : PRINT LBOUND(C); LBOUND(A)
 0  1
```

OR

An operator: the bitwise OR of two integers, which is also the logical OR of two truth values.

SYNTAX

```
a OR b
```

RESULT

An integer. Each bit of it is 1 where the same bit is 1 in *a*, in *b*, or in both.

For two truth values, −1 and 0, the result is true if either is.

Both operands are always evaluated, and they are rounded and treated as `AND`'s are: see `AND`. `OR` binds less tightly than `AND` and more tightly than `XOR`.

EXAMPLE

```
> PRINT 5 OR 3; 0 OR 0; (1 = 2) OR (2 = 2); 1.5 OR 0
 7  0 -1  2
```

OUTPUT

A mode of `OPEN`: `OPEN path$ FOR OUTPUT AS #f` makes a file, or empties one, and opens it for writing. See `OPEN`.

PAINT

A statement: fills an area of the bitmap with a colour.

SYNTAX

```
PAINT [STEP] (x, y) [, c [, border]]
```

ARGUMENTS

x*, *y
Numeric expressions: a point inside the area.

c
A numeric expression, 0 to 255: the colour to fill with. The pen's colour if it is left out.

border

A numeric expression, 0 to 255: the colour that bounds the area. If it is left out, it is *c*.

DESCRIPTION

Needs a graphics mode. Starting from the point, the fill spreads to every pixel it can reach by moving up, down, left and right without crossing a pixel of the *border* colour, or one that is already colour *c*. It stops at the edges of the bitmap. If the starting pixel is itself the border's colour or *c*, nothing is filled.

The outline must be closed. A gap of a single pixel lets the colour out, and it fills everything outside too.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set; or a colour is not 0 to 255
13	`type mismatch`	An argument is a string

NOTES

The fill is done by HyBASIC, a row at a time, reading the bitmap back from the screen's memory. A large or intricate area takes a noticeable time. A box is filled far faster by `LINE` with `BF`, and a disc by `CIRCLE` with `F`.

A starting point off the bitmap fills nothing, without an error.

SEE ALSO

`LINE`, `CIRCLE`, `DRAW`, `COLOR`, `SCREEN`.

EXAMPLE

A box outlined in colour 6, filled with colour 2 up to that border:

```
> SCREEN 13 : LINE (30, 30)-(40, 40), 6, B : PAINT (35, 35), 2, 6
> PRINT POINT(35, 35); POINT(31, 31); POINT(30, 30); POINT(45, 45)
 2  2  6  0
> SCREEN 0
```

PALETTE

A statement: sets what colour one of the colour numbers shows.

SYNTAX

```
PALETTE n, rgb
```

ARGUMENTS

n

A numeric expression, 0 to 255: a colour number.

rgb

A numeric expression, 0 to 4095: the colour, as three hex digits, one each for red, green and blue. `&HF00` is the brightest red, `&HFFF` white and `&H000` black.

DESCRIPTION

A pixel holds a colour number, and the palette says what colour each number is shown as. `PALETTE` changes one entry. Everything on the screen that has that number changes colour at once, without being drawn again: the text as well as the bitmap, since they share the one palette.

It does not need a graphics mode, only a Vera X card.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is not 0 to 255, or *rgb* is not 0 to 4095
13	`type mismatch`	An argument is a string

NOTES

The palette is the screen's, not HyBASIC's: a change stays after the program ends, and after HyBASIC does.

SEE ALSO

`COLOR`, `SCREEN`.

PEEK, POKE

A function and a statement: read, and write, one byte of memory.

SYNTAX

```
PEEK(address)
POKE address, value
```

ARGUMENTS

address

A numeric expression: an address in HyBASIC's task, 0 to 65,535. A negative number from −32,768 to −1 is accepted too, and means the address 65,536 more: −1 is `$FFFF`.

value

A numeric expression from 0 to 255, rounded to the nearest integer.

RESULT

`PEEK` is the byte at the address, 0 to 255.

DESCRIPTION

The addresses are those of HyBASIC's own task: its zero page and stack, the RAM that holds its state, the bank window at `$8000` to `$9FFF`, the paged ROM at `$A000` to `$DFFF` and the BIOS above (Appendix F). Another task's memory can't be reached.

For an address in the bank window, the bank is the one `BANK` has set. HyBASIC puts it there for the read or the write and takes it away again, so the window is of no use to a program between the two: see `BANK`.

`POKE` does no checking beyond the two ranges. Writing into HyBASIC's own memory, which is most of what is below `$8000`, will usually stop it working; and a write to an address that is ROM does nothing. The hardware's registers are reached through the drivers' files (Chapter 12), not through `POKE`.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	The address is outside −32,768 to 65,535; or the value is outside 0 to 255
13	`type mismatch`	An argument is a string

SEE ALSO

`BANK`, `SYS`, `CALL`, `ASM`.

EXAMPLE

```
> BANK 40 : POKE &H8000, 65 : POKE &H8001, 66.5 : PRINT PEEK(&H8000); PEEK(&H8001);
PEEK(-32768)
 65  66  65
> POKE &H8000, 256
illegal function call
```

PI

A numeric function: π.

SYNTAX

```
PI
```

RESULT

The ratio of a circle's circumference to its diameter, as a decimal rounded to the number of significant digits that `DIGITS` has set: 3.14159265359 at the usual 12.

It takes no argument and no parentheses.

SEE ALSO

`DIGITS`, `SIN`, `COS`, `TAN`, `ATN`.

EXAMPLE

```
> PRINT PI : DIGITS 30 : PRINT PI : DIGITS 12
 3.14159265359
 3.14159265358979323846264338328
```

PLAY

A statement: plays music, and waits until it has finished.

SYNTAX

```
PLAY [channel,] score$
PLAY file$
```

ARGUMENTS

score$

A string expression: a line of the score language, MML (section 15.6).

channel

A numeric expression: the sound channel to play it on (section 15.1). Channel 0 if it is left out.

file$

A string expression ending in `.zsm` or `.mml`: the name of a song file, or of a file of score lines.

DESCRIPTION

HyBASIC does not make the sound itself. It runs the `play` program (section 15.7) with the string, and waits for it to end: `play -m channel score` for a line of score, and `play file` for a file. Whatever `play` can play, `PLAY` can. The program stops while the music lasts.

A string is taken as a file's name if it ends in `.zsm` or `.mml`, and as a score otherwise.

ERRORS

ERR	MESSAGE	CAUSE
511	`play`'s own message	`play` could not play it: a mistake in the score, a file that isn't there. `play` prints its message itself first, with its name before it
5	`illegal function call`	A channel number that is not one
13	`type mismatch`	The score is a number

NOTES

This is not QuickBASIC's `PLAY` language, though simple tunes look alike in both: the letters `c` to `b` are notes, `o` sets the octave and `l` the length. Section 15.6 has the whole language.

For a single tone, `SOUND` is simpler.

SEE ALSO

SOUND, BEEP, SHELL.

EXAMPLE

```
> PLAY "t240 l16 c e g" : PRINT "played"
played
> PLAY "zzz!"
play: zzz!: channel 0: what is z
channel 0: what is z
> PRINT ERR
 511
```

POINT

A numeric function: the colour of a pixel.

SYNTAX

```
POINT(x, y)
```

ARGUMENTS

x, y

Numeric expressions: the pixel, in the coordinates the drawing statements use, with the WINDOW and VIEW that are in force.

RESULT

The pixel's colour number. −1 if the point is off the bitmap, or if no graphics mode is set.

ERRORS

ERR	MESSAGE	CAUSE
13	type mismatch	An argument is a string

NOTES

The result is what is in the bitmap. Sprites and the console's text, which are shown over it, are not seen.

SEE ALSO

PSET, SCREEN, WINDOW, VIEW.

POS

A numeric function: the column the next character will be printed in.

SYNTAX

```
POS(x)
POS
```

ARGUMENTS

x

Any numeric expression. It is ignored; `POS(0)` is the usual form, from QuickBASIC, and `POS` alone means the same.

RESULT

The column, counted from 1 at the left edge. HyBASIC keeps count of what it has written since the last line end, so this is the column of its own output: after `PRINT "ab";` it is 3.

NOTES

The count is HyBASIC's, not the terminal's. It follows `LOCATE`, but it knows nothing of what other programs have written, or of escape sequences a program prints for itself. The row is `CSRLIN`, which does ask the terminal.

`TAB` and the print zones of a comma work from the same count.

SEE ALSO

`CSRLIN`, `LOCATE`, `TAB`, `SPC`, `WIDTH`.

EXAMPLE

```
> PRINT POS(0); : PRINT "ab"; : PRINT POS(0)
 1 ab 6
```

PRESERVE

Part of `REDIM`: `REDIM PRESERVE` changes an array's size and keeps what is in it. See `REDIM`.

PRESET, PSET

Statements: set one pixel.

SYNTAX

```
PSET [STEP] (x, y) [, c]
PRESET [STEP] (x, y) [, c]
```

ARGUMENTS

x, y

Numeric expressions: the pixel. With `STEP`, an offset from the last point referenced.

c

A numeric expression, 0 to 255: the colour. A mode with fewer colours uses the number's low bits.

DESCRIPTION

Needs a graphics mode. The two differ only when no colour is given: `PSET` then uses the pen's colour, and `PRESET` uses colour 0, the background, which rubs the pixel out.

The point becomes the last point referenced. A point off the bitmap sets nothing, and is not an error.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set; or *c* is not 0 to 255
13	`type mismatch`	An argument is a string

SEE ALSO

`POINT`, `LINE`, `COLOR`, `SCREEN`.

EXAMPLE

```
> SCREEN 13 : PSET (10, 10), 5 : PSET (20, 20) : PRINT POINT(10, 10); POINT(20, 20);
POINT(11, 10)
 5  15  0
> PRESET (20, 20) : PSET STEP (5, 0), 9 : PRINT POINT(20, 20); POINT(25, 20)
 0  9
> SCREEN 0
```

PRINT

A statement: writes values as text, to the screen or to a file.

SYNTAX

```
PRINT [#f,] [item [{; | ,} item] ...] [; | ,]
PRINT [#f,] USING format$; item [{; | ,} item] ... [; | ,]
? ...
```

ARGUMENTS

item

An expression of any type, or `TAB(n)` or `SPC(n)`.

f

The number of a file opened `FOR OUTPUT`, `FOR APPEND` or `FOR BINARY`. The output goes to the file in place of the screen, byte for byte the same.

format$

A string expression: a picture of how the items are to be written (below).

`?` is another spelling of `PRINT`.

DESCRIPTION

Values. A string is written as its bytes. A number is written as its text in the base that `BASE` has set, with a space before it if it isn't negative (a negative number has its minus sign there instead) and a space after it. The text is the number's exact value: an integer has all its digits however many there are, a rational is written with a `/`, and a decimal with its point and every digit it holds. There is no exponent form: a very large or very small number is written out in full.

Between items. A semicolon writes nothing: the next item follows at once. A comma moves to the next print zone: the zones are 14 columns wide, starting at columns 1, 15, 29 and so on, and if the next zone would start past the width of the line the item starts a new line instead. Two commas skip a zone.

At the end. `PRINT` ends the line, by writing a line feed, unless the statement ends with a semicolon or a comma: then the next `PRINT` carries on from there. `PRINT` with no items writes an empty line.

`TAB(n)` moves to column *n*, counting from 1, by writing spaces. If the output is already past column *n*, it starts a new line and moves to column *n* there. **`SPC(n)`** writes *n* spaces. Both are used only inside `PRINT`.

The line's width is 80 unless `WIDTH` has set another. It decides where the zones run out. Text longer than the width is not broken by HyBASIC; the terminal wraps it.

PRINT USING

The format is text with **fields** in it. Each item is written by the next field; the text between fields is written as it stands. If there are more items than fields, the format is used again from its start.

A number's field is a picture of the number:

IN THE FIELD	MEANS
`#`	A digit's place. The number is put at the right of its places, with spaces before it
`.`	The decimal point. The `#`s after it are decimal places, always filled, with zeros if need be
`,`	Before the point: put commas between each three digits
`+`	At the start: write the sign, `+` or `-`, always
`$$`	At the start: write a `$` just before the number
`**`	At the start: fill the spaces before the number with `*`

The number is **rounded** to the field's decimal places, a half going away from zero: 2.5 in `###` is 3, and −3.145 in `###.##` is −3.15. (This is not `CINT`'s rule, which sends a half to the even integer.) A minus sign takes a place. If the number needs more places than the field has, it is written in full with a `%` before it to show that the column is broken.

A string's field is one of three:

IN THE FIELD	MEANS
`!`	The first character of the string
`&`	The whole string
`\`, spaces, `\`	As many characters as the field is wide, counting both backslashes; a shorter string is padded with spaces

Two more things can be in a format:

IN THE FORMAT	MEANS
`{}`	The item as `PRINT` would write it, a number or a string, with no spaces added
`{`*base*`}`	A number in that base: `{x}` in hex, `{b}` in binary (a base string, section 18.4)
`_`	The next character is text, not part of a field: `_#` writes a `#`

A semicolon or a comma between the items of `PRINT USING` makes no difference: the format decides the spacing. At the end of the statement they hold the line open, as in `PRINT`.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	`USING`: a string for a number's field, or a number for a string's
5	`illegal function call`	`USING`: the format has no field in it
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR INPUT`

NOTES

`PRINT #` writes to a file exactly what `PRINT` would have put on the screen, zones and spaces included. For data that a program will read back with `INPUT #`, `WRITE #` is the better choice: it puts commas between the items and quotes round the strings.

To print a number without its spaces, use `PRINT USING "{}"`, or `LTRIM$(STR$(n))`.

SEE ALSO

`WRITE`, `WIDTH`, `LOCATE`, `POS`, `STR$`, `BASE`, `FIXED`, `OPEN`.

EXAMPLE

```
> PRINT 1; -2; 3 : PRINT "a"; "b"; 5; "c" : PRINT 1, 2, "x", -4
 1 -2  3
ab 5 c
 1             2             x             -4
> PRINT 1/3; 2.50; 2 ^ 64; 1+2i : PRINT "one"; : PRINT " line"
 1/3  2.5  18446744073709551616  1+2i
one line
> PRINT "A"; TAB(10); "B"; TAB(5); "C"; SPC(3); "D"
A         B
    C   D
> PRINT USING "###.##"; 3.14159 : PRINT USING "###.##|"; -3.145; 1234.5
  3.14
 -3.15|%1234.50|
> PRINT USING "#,###.# +##.# $$###.## **###.##"; 1234.56; 5; 12.5; 12.5
1,234.6  +5.0   $12.50 ***12.50
> PRINT USING "[!][&][\  \] {} {x} _#"; "hello"; "world"; "abcdefg"; 255; 255
[h][world][abcd] 255 FF #
```

PUT

A statement: writes a value to a `BINARY` file, at a given position.

SYNTAX

```
PUT [#]f, [position], value
```

ARGUMENTS

f

A numeric expression: the number of a file opened `FOR BINARY`.

position

A numeric expression: where to write, counted in bytes from 1 for the file's first byte. If it is left out (the two commas are still written) or is less than 1, the write starts where the last `GET` or `PUT` on the file ended.

value

An expression of either type. A variable is the usual thing, but any expression will do.

DESCRIPTION

A string is written as its bytes, as many as it has, with nothing before or after them. A number is written in the stored format, the form `MKN$` makes, which is as long as the number needs.

The bytes replace whatever was at that position. Writing past the end makes the file longer; writing at a position beyond the end leaves the gap between filled with zero bytes.

After the write, the file's position is just past what was written.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* was not opened `FOR BINARY`
61	`disk full`	No room left on the disk

NOTES

To write text to a file opened `FOR BINARY`, `PRINT #` works too, and adds its line end.

Since a number's stored form varies in length, a file of numbers written with `PUT` is read back in the same order with `GET`; a number can't be changed in place for a longer one without overwriting what follows it.

SEE ALSO

`GET`, `OPEN`, `SEEK`, `MKN$`, `CVN`, `PRINT`.

EXAMPLE

```
> OPEN "/ram/p.dat" FOR BINARY AS #1 : PUT #1, 1, "HELLO" : PUT #1, 3, "xy" : PUT #1, 9,
"!" : PRINT LOF(1)
 9
> A$ = SPACE$(9) : GET #1, 1, A$ : PRINT A$; ASC(MID$(A$, 6)) : CLOSE
HExyO! 0
```

RANDOMIZE

A statement: seeds the random number generator.

SYNTAX

```
RANDOMIZE [n]
```

ARGUMENTS

n

A numeric expression: the seed. Only its low 16 bits are used, so there are 65,536 different sequences.

DESCRIPTION

`RND` produces a fixed sequence of numbers from wherever the generator stands. `RANDOMIZE n` puts the generator at the start of sequence *n*: the same *n* always gives the same numbers afterwards, which is what a program being tested wants.

`RANDOMIZE` with no number, or with 0, takes the seed from the system's clock, so that each run is different. That is what a game wants, once, at its start.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*n* is a string

NOTES

QuickBASIC asks `Random-number seed?` when `RANDOMIZE` has no number. HyBASIC asks nothing.

SEE ALSO

`RND`, `TIMER`.

EXAMPLE

```
> RANDOMIZE 1 : A = RND : B = RND : RANDOMIZE 1 : PRINT A = RND; B = RND
-1 -1
```

RATIONAL

A numeric function: a real number as an exact fraction.

SYNTAX

```
RATIONAL(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The rational number equal to *x*, in lowest terms. A decimal becomes the fraction it exactly is: 0.75 becomes 3/4, and 0.1 becomes 1/10, with no error at all, since a decimal here is held in decimal. An integer or a rational is returned unchanged.

DESCRIPTION

Adding, subtracting and multiplying decimals gives a decimal. Division is always exact, and gives a fraction when the quotient isn't whole, a decimal's too: `0.75 / 3` is 1/4. `RATIONAL` turns a decimal into a fraction without dividing it by anything; `FIXED` goes the other way.

The result is exact for the decimal it is given. If that decimal was itself rounded, as the result of `SQR` or `SIN` is, the fraction is the rounded value's.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

FIXED, NUMERATOR, DENOMINATOR.

EXAMPLE

```
> PRINT RATIONAL(0.75); RATIONAL(2.5); RATIONAL(0.1); RATIONAL(7); RATIONAL(SQR(2))
 3/4  5/2  1/10  7  141421356237/100000000000
```

READ

A statement: takes the next items of the program's DATA into variables.

SYNTAX

```
READ variable [, variable] ...
```

ARGUMENTS

variable

A numeric or string variable, an array's element, or a record's field.

DESCRIPTION

Each variable is given the next unread item of the program's DATA, in the order the DATA statements stand in the program. The place carries on from one READ to the next, until RESTORE moves it.

An item read into a string variable is its text. An item read into a numeric variable is read as a number, in any form a program's numbers may take; an item that isn't a number, and an empty item, give 0 with no error. DATA describes the items.

ERRORS

ERR	MESSAGE	CAUSE
4	`out of DATA`	No item is left

SEE ALSO

DATA, RESTORE, INPUT.

EXAMPLE

```
> 10 DATA 1, 2
> 20 DATA 3, text
> 30 READ A, B, C, D$ : PRINT A; B; C; D$
> 40 RESTORE 20 : READ E : PRINT E
> RUN
 1  2  3 text
 3
```

REAL

A numeric function: the real part of a number.

SYNTAX

```
REAL(z)
```

RESULT

The real part of *z*, of the kind it has in *z*. A real number is returned unchanged. See `IMAG`, which has the errors and an example.

REDIM

A statement: makes an array again, with new bounds.

SYNTAX

```
REDIM [PRESERVE] name(bounds [, bounds] ...) [AS type] [, name ...] ...
```

ARGUMENTS

As `DIM`'s.

DESCRIPTION

Where `DIM` refuses to make an array that exists, `REDIM` replaces it. An array that doesn't exist yet is simply made, so `REDIM` can be used for the first time as well as the next.

Without `PRESERVE`, the new array is empty: every element 0 or the empty string.

With `PRESERVE`, the old array's elements are copied into the new one **in order of position**, not by index: the old array's first element goes to the new array's first element, the second to the second, and so on, for as many as both arrays have. If the two have the same lowest index, as they usually do, an element keeps its index; if the lowest index changes, every element moves. The number of dimensions may change too, the elements still being taken in their stored order.

ERRORS

ERR	MESSAGE	CAUSE
9	`subscript out of range`	Bounds that `DIM` would refuse
7	`out of memory`	No room for the new array

SEE ALSO

`DIM`, `ERASE`, `LBOUND`, `UBOUND`.

EXAMPLE

In the second line the lowest index moves from 0 to 1, so the old `A(0)` and `A(1)` become the new `A(1)` and `A(2)`:

```
> DIM A(3) : A(0) = 5 : A(1) = 6 : REDIM PRESERVE A(10) : PRINT A(0); A(1); UBOUND(A)
 5  6  10
> REDIM PRESERVE A(1 TO 2) : PRINT A(1); A(2) : REDIM A(4) : PRINT A(1)
 5  6
 0
```

REM

A statement: a comment.

SYNTAX

```
REM text
' text
```

DESCRIPTION

Everything from `REM` to the end of the line is ignored, colons included: a statement can't follow a comment on the same line. An apostrophe does the same, and can come after a statement without a colon before it.

NOTES

One comment is not ignored: `REM $INCLUDE: 'file'`, and the same with an apostrophe, is QuickBASIC's way of writing `INCLUDE`. See `INCLUDE`.

EXAMPLE

```
> REM this is ignored : PRINT "not printed"
> PRINT "printed" ' and this is ignored too : PRINT "nor this"
printed
```

RESTORE

A statement: moves the place `READ` takes `DATA` from.

SYNTAX

```
RESTORE [line | label]
```

DESCRIPTION

With nothing after it, the next `READ` starts again at the program's first `DATA` item. With a line number or a label, it starts at the first item of the first `DATA` statement at or after that line.

ERRORS

ERR	MESSAGE	CAUSE
8	`label not defined`	No line has that number or label

SEE ALSO

READ, DATA.

RESUME

A statement: ends an error handler, and says where the program carries on.

SYNTAX

```
RESUME [0]
RESUME NEXT
RESUME {line | label}
```

DESCRIPTION

FORM	CARRIES ON
`RESUME`, `RESUME 0`	At the statement that failed, which is run again
`RESUME NEXT`	At the statement after the one that failed
`RESUME line`	At that line

The statement that failed is the one in which the error happened, not the line: with several statements on a line, `RESUME NEXT` goes on with the next statement of that line.

`RESUME` is what marks the handler's end. After it, a new error goes to the handler again; until it, a second error stops the program.

`RESUME` alone suits an error that the handler has put right, a missing file it has now made. If the cause is still there the statement fails again, and the program goes round for ever.

ERRORS

ERR	MESSAGE	CAUSE
20	`RESUME without error`	A `RESUME` that is not in a handler that an error started
8	`label not defined`	No line has that number or label

SEE ALSO

ON, ERR, ERROR.

EXAMPLE

See `ON`.

RETURN

A statement: comes back from a subroutine that `GOSUB` called.

SYNTAX

```
RETURN
```

DESCRIPTION

Control goes back to the statement after the most recent `GOSUB` that hasn't been returned from.

ERRORS

ERR	MESSAGE	CAUSE
3	`RETURN without GOSUB`	There is no `GOSUB` to go back to: usually, control has run into a subroutine's lines from above

NOTES

To leave a `SUB` or a `FUNCTION`, use `EXIT SUB` or `EXIT FUNCTION`, not `RETURN`.

SEE ALSO

`GOSUB`, `ON`, `EXIT`.

RND

A numeric function: a random number.

SYNTAX

```
RND
RND(x)
```

ARGUMENTS

x

A numeric expression. Its sign chooses what happens:

X	RESULT
Left out, or more than 0	The next number of the sequence
0	The last number again
Less than 0	The generator is seeded with *x* first, then the next number. The same *x* gives the same number every time

RESULT

A decimal from 0 up to, but never reaching, 1, with as many digits as `DIGITS` has set (12 unless it has been changed).

DESCRIPTION

The numbers are a fixed sequence worked out from a seed, and they look random. HyBASIC seeds the generator afresh each time it starts, so two sessions give different numbers; within a session the sequence simply carries on, from one `RUN` to the next. `RANDOMIZE n` gives a sequence that can be repeated.

A whole number from 1 to *n* is `INT(RND * n) + 1`.

NOTES

The generator has 16 bits of seed. It is good enough for games and simulations and not for anything that has to be unguessable.

SEE ALSO

`RANDOMIZE`, `DIGITS`, `INT`.

EXAMPLE

```
> RANDOMIZE 1 : A = RND : PRINT A; RND(0) = A; A >= 0 AND A < 1
 0.778176127105 -1 -1
> X = RND(-5) : Y = RND(-5) : PRINT X = Y : DIGITS 5 : PRINT RND : DIGITS 12
-1
 0.25799
```

RREG

A statement: copies the processor's registers, as machine code left them, into variables.

SYNTAX

```
RREG a [, x [, y [, p]]]
```

ARGUMENTS

a, x, y, p

Numeric variables. Each is given one register's value, 0 to 255: A, X, Y, and the status flags, in that order. The list can stop after any of them.

DESCRIPTION

`SYS`, `CALL ABSOLUTE` and `CALL ASM` each run machine code and then save the registers it returned with. `RREG` reads them: the values are those of the most recent such call, and stay until the next.

In the flags, bit 0 is the carry (C), bit 1 zero (Z), bit 6 overflow (V) and bit 7 negative (N). A system call reports failure by returning with the carry set and an error code in A, so `BIT(p, 0)` tests whether a call worked.

ERRORS

ERR	MESSAGE	CAUSE
40	`variable required`	An argument that is not a variable

SEE ALSO

SYS, CALL, ASM, BIT.

EXAMPLE

GETPID returns the task's number in A. Here HyBASIC is task 4:

```
> SYS "GETPID" : RREG A, X, Y, P : PRINT A; BIT(P, 0)
 4  0
```

RUN

A command of the prompt: runs the program.

SYNTAX

```
RUN ["file"]
```

ARGUMENTS

file

A file's name, in quotes: a program to load first, as LOAD would, and then run.

DESCRIPTION

RUN first clears away what an earlier run left: every variable and array is dropped, every file is closed, DATA is back at its first item and any stopped program is forgotten. Then the whole program is compiled, and if that finds an error it is reported with its line and nothing runs. Otherwise the program runs from its first line.

The program's numbers are always read in decimal as it is compiled, whatever BASE was left at.

RUN must be alone on its line.

ERRORS

ERR	MESSAGE	CAUSE
89	only at the prompt	RUN in a program's line, or after other statements on a line typed at the prompt
288	not found	There is no such *file*

Any error the compiler finds in the program is reported by RUN, with its line.

NOTES

A line number after RUN is accepted and ignored: RUN 100 runs the program from its first line. To start part-way, put a GOTO at the top.

SEE ALSO

CONT, LOAD, NEW, END, STOP.

EXAMPLE

```
> 10 PRINT "running"; X : X = X + 1
> X = 5
> RUN
running 0
> RUN
running 0
```

SAVE

A command of the prompt: writes the program to a file.

SYNTAX

```
SAVE ["file"]
```

ARGUMENTS

file

The file's name, in quotes. Nothing is added to it.

DESCRIPTION

The program's lines are written to the file as plain text, a line to a line, as `LIST` shows them. A file of that name is replaced.

With no name, the program is saved to the file it was last loaded from or saved to.

ERRORS

ERR	MESSAGE	CAUSE
70	`permission denied`	The file, or its disk, can't be written
53	`file not found`	A directory in the name isn't there
61	`disk full`	No room left
89	`only at the prompt`	`SAVE` in a program's line

NOTES

`/ram` is emptied when the Hydra is switched off or reset. Save what matters to an SD card (`/sd/0`) or to the PC (`/pc`).

SEE ALSO

`LOAD`, `LIST`, `NEW`.

SCREEN

A statement: chooses a graphics mode, or returns to text alone.

SYNTAX

```
SCREEN n
```

ARGUMENTS

n

A numeric expression: the mode.

N	BITMAP	COLOURS	THE PEN STARTS AS
0	None: text alone		
1	320 × 240	4	3
2	640 × 480	2	1
7	320 × 240	16	15
12	640 × 480	4	3
13	320 × 240	256	15

DESCRIPTION

Graphics need a Vera X card (Chapter 16). In a graphics mode the screen has a **bitmap**, a grid of pixels each holding a colour number, with the console's text shown over it. The drawing statements change the bitmap, and `PRINT` goes on writing text as before.

`SCREEN n` sets the mode, clears the bitmap to colour 0, sets the **pen** (the colour used when a drawing statement is given none) to the mode's brightest, up to 15, and puts the **last point referenced** at the middle of the bitmap. Any `WINDOW` and `VIEW` are cancelled. `SCREEN 0` takes the bitmap away.

Coordinates. A point is written `(x, y)`. Unless `WINDOW` says otherwise, *x* counts pixels from the left edge, starting at 0, and *y* counts them down from the top: in a 320 × 240 mode the corners are (0, 0) and (319, 239). A coordinate that isn't an integer is rounded to the nearest, a half going to the even one.

The last point referenced is where the most recent drawing statement finished. `STEP (dx, dy)` means the point that far from it, and `LINE -(x, y)` starts from it.

Colours. A colour is a number from 0 to 255. A mode with fewer colours uses as many low bits as it has, so in a 4-colour mode 5 is colour 1. What colour a number looks like is the palette's business: see `PALETTE`.

The mode numbers are QuickBASIC's for the nearest thing each mode is.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is not one of the six modes
53	`file not found`	There is no Vera X card: the screen driver's files are not there

NOTES

The drawing is done by the screen's driver, to which HyBASIC writes commands (`/dev/vid/draw`, section 16.4). A program in another language, or the shell, can draw on the same bitmap by the same route.

SEE ALSO

`PSET`, `LINE`, `CIRCLE`, `PAINT`, `DRAW`, `GPRINT`, `POINT`, `COLOR`, `PALETTE`, `WINDOW`, `VIEW`, `SPRITE`, `CLS`.

SEEK

A statement and a function: sets, or reports, an open file's position.

SYNTAX

```
SEEK [#]f, position
SEEK(f)
```

ARGUMENTS

f

A numeric expression: the number of an open file, in any mode.

position

A numeric expression, 1 or more: the byte the next read or write will start at, counting the file's first byte as 1.

RESULT

`SEEK(f)` is the file's present position, counted from 1: one more than `LOC(f)`.

DESCRIPTION

The statement moves the position. What is read or written next starts there, whatever statement does it.

A position past the end of the file is allowed. A read there finds the end of the file; a write there makes the file longer, the gap filled with zero bytes.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*position* is less than 1
52	`bad file number`	File *f* isn't open

SEE ALSO

`LOC`, `LOF`, `GET`, `PUT`, `OPEN`.

EXAMPLE

```
> OPEN "/ram/s.txt" FOR OUTPUT AS #1 : PRINT #1, "abcdef" : SEEK #1, 3 : PRINT #1, "X";
: PRINT SEEK(1) : CLOSE
 4
> OPEN "/ram/s.txt" FOR INPUT AS #1 : LINE INPUT #1, L$ : PRINT L$ : SEEK #1, 100 :
PRINT EOF(1) : CLOSE
abXdef
-1
```

SELECT CASE

A statement block: runs one of several groups of statements, chosen by a value.

SYNTAX

```
SELECT CASE value
CASE test [, test] ...
    [statements]
[CASE ...] ...
[CASE ELSE
    [statements]]
END SELECT
```

where *test* is one of

```
expression
low TO high
IS operator expression
```

ARGUMENTS

value

An expression, numeric or string. It is evaluated once.

test

An expression of the same type as *value*. *operator* is =, <>, <, >, <= or >=.

DESCRIPTION

The `CASE` lines are taken in order from the top, and the tests of each from left to right. A plain expression matches if *value* equals it; `low TO high` if *value* is from *low* to *high*, both included; and `IS` if the comparison of *value* with the expression holds. The statements under the first `CASE` that has a matching test are run, and then control goes on after `END SELECT`: at most one group runs, even if a later `CASE` would match as well.

`CASE ELSE` matches anything, so it is put last. If nothing matches and there is no `CASE ELSE`, no group runs, and that is not an error.

Strings are compared byte by byte, as < and > compare them, so ranges of strings work: `CASE "a" TO "m"`.

ERRORS

ERR	MESSAGE	CAUSE
80	`block without its start or end`	`SELECT CASE` with no `END SELECT`, or `CASE` or `END SELECT` with none open
13	`type mismatch`	A test of the other type from *value*

NOTES

Statements written between `SELECT CASE` and the first `CASE` are run every time, before the choice is made. QuickBASIC doesn't allow them.

The whole block can be written on one line with colons, which is the only form the prompt can take.

SEE ALSO

`IF`, `ON`.

EXAMPLE

```
> 10 X = 2
> 20 SELECT CASE X
> 30 CASE 1 : PRINT "one"
> 40 CASE 2, 3 : PRINT "two or three"
> 50 CASE IS > 1 : PRINT "more than one: not run, an earlier CASE matched"
> 60 CASE ELSE : PRINT "other"
> 70 END SELECT
> 80 SELECT CASE "pear" : CASE "apple" TO "fig" : PRINT "a to f" : CASE IS > "m" : PRINT
"after m" : END SELECT
> RUN
two or three
after m
```

SGN

A numeric function: the sign of a number.

SYNTAX

```
SGN(x)
```

ARGUMENTS

x

A numeric expression.

RESULT

−1 if *x* is negative, 0 if it is zero, and 1 if it is positive. For a complex number, the sign of its real part.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*x* is a string

SEE ALSO

`ABS`.

EXAMPLE

```
> PRINT SGN(-5); SGN(0); SGN(7/2); SGN(-0.1); SGN(-1+2i)
-1  0  1 -1 -1
```

SHARED

A statement, in a procedure: makes variables of the main program visible there.

SYNTAX

```
SHARED name[()] [AS type] [, name ...] ...
DIM SHARED ...
```

ARGUMENTS

name

A variable of the main program. An array is written with empty parentheses.

DESCRIPTION

A `SUB` or a `FUNCTION` has variables of its own: a name used inside it is a new variable, which starts at 0 or empty on each call, whatever the main program calls its own variables. There are two ways to reach the main program's variables instead.

`SHARED`, inside a procedure, names the main program's variables that this procedure uses. From that line on, within the procedure, those names mean the main program's variables.

`DIM SHARED`, in the main program, declares variables and arrays that every procedure can use without saying so. It is `DIM` with one more word: see `DIM`.

Constants need neither. A `CONST` is known everywhere.

ERRORS

ERR	MESSAGE	CAUSE
85	`not in a procedure`	`SHARED` on its own in the main program. (`DIM SHARED` is the form for there)

SEE ALSO

`DIM`, `STATIC`, `SUB`, `FUNCTION`, `CONST`.

EXAMPLE

```
> 10 DIM SHARED G : G = 1 : L = 2
> 20 SUB Show : PRINT "G ="; G; "L ="; L : END SUB
> 30 SUB Uses : SHARED L : L = L + 10 : END SUB
> 40 Show : Uses : PRINT L
> RUN
G = 1 L = 0
 12
```

SHELL

A statement: runs a command line in the shell, and waits for it.

SYNTAX

```
SHELL [line$]
```

ARGUMENTS

line$

A string expression: a line as it would be typed at rc's prompt (Chapter 8). It can be anything rc accepts: a program and its arguments, a pipeline, several commands.

DESCRIPTION

HyBASIC starts rc with the line (`rc -c line`), with the program's standard input and output, its current directory and a copy of its environment, and waits until rc has finished. What the commands print appears on the screen in the ordinary way. `STATUS` then gives how the last command ended.

With no line, rc is started by itself: an interactive shell, which lasts until `exit` or CTRL-D, when the program carries on.

Nothing the shell does to its own variables or its own current directory comes back to HyBASIC: it is another task. Files it makes and removes are, of course, made and removed.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*line$* is a number

A command that fails is not an error in the program. Its message is printed by rc or by the command, and `STATUS` is not 0.

SEE ALSO

`SHELL$`, `STATUS`, `ENVIRON`, `SYSTEM`.

EXAMPLE

```
> SHELL "echo from the shell" : PRINT STATUS
from the shell
 0
> SHELL "ls /nothere" : PRINT STATUS
ls: /nothere: not found
 1
```

SHELL$

A string function: what a command line prints.

SYNTAX

```
SHELL$(line$)
```

ARGUMENTS

line$

A string expression: a line for rc, as `SHELL`'s.

RESULT

Everything the line's commands wrote to standard output, as one string, 8K at most. The line end after the last line is not included; the ones between lines are, as `CHR$(10)`.

What the commands write to standard error is not captured. It goes to the screen, as their messages should.

`STATUS` is set as `SHELL` sets it.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*line$* is a number

SEE ALSO

`SHELL`, `STATUS`.

EXAMPLE

```
> A$ = SHELL$("echo one; echo two") : PRINT LEN(A$); INSTR(A$, CHR$(10)); "["; LEFT$(A$,
3); "]"
 7  4 [one]
> A$ = SHELL$("ls /nothere") : PRINT "["; A$; "]"; STATUS
ls: /nothere: not found
[] 1
```

SHL, SHR

Numeric functions: an integer shifted left, or right, by a number of bits.

SYNTAX

```
SHL(x, n)
SHR(x, n)
```

ARGUMENTS

x

A numeric expression whose value is real, rounded to the nearest integer: the number to shift.

n

A numeric expression: how many bits.

RESULT

`SHL(x, n)` is *x* multiplied by 2 to the power *n*. There is no word size, so nothing is ever shifted out at the top: `SHL(1, 100)` is 2 to the power 100.

`SHR(x, n)` is *x* divided by 2 to the power *n* and rounded down. For a negative *x* this is the arithmetic shift, which keeps the sign: `SHR(-8, 1)` is −4.

A negative *n* shifts the other way: `SHR(1, -2)` is `SHL(1, 2)`.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	An argument is a string
2	`syntax error`	One argument only

SEE ALSO

AND, OR, XOR, BIT.

EXAMPLE

```
> PRINT SHL(1, 4); SHR(256, 4); SHL(1, 100); SHR(-8, 1); SHR(1, 5); SHR(1, -2)
 16  16  1267650600228229401496703205376 -4  0  4
```

SIN

A numeric function: the sine of a real number.

SYNTAX

```
SIN(x)
```

ARGUMENTS

x

A numeric expression whose value is real: an angle in radians.

RESULT

The sine of *x*, from −1 to 1. `SIN(0)` is the integer 0. Any other result is a decimal, correct to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

`PI` is itself rounded to the digits set, so `SIN(PI)` is not exactly 0: it is a very small number, the sine of the angle that `PI` really is.

SEE ALSO

`COS`, `TAN`, `ATN`, `PI`, `DIGITS`.

EXAMPLE

```
> PRINT SIN(0); SIN(1); SIN(PI / 2); SIN(-1); SIN(PI)
 0  0.841470984808  1 -0.841470984808 -0.00000000000002067615373 57
```

SLEEP

A statement: waits for a time, or for a key.

SYNTAX

```
SLEEP [seconds]
```

ARGUMENTS

seconds

A numeric expression: how long to wait. It need not be whole: `SLEEP 0.5` waits half a second.

DESCRIPTION

The program stops for that long and then goes on. While it waits it uses no processor time, so other tasks run freely; this is the right way to slow a loop down. The time is counted in the system's ticks, 200 to the second, so the shortest wait is five thousandths of a second, and a wait can be a tick longer than asked.

With no argument, or with a negative one, `SLEEP` waits until a key is pressed. The key is used up by the wait: the program doesn't see it.

`SLEEP 0` returns at once.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*seconds* is a string

SEE ALSO

`TIMER`, `INKEY$`, `INPUT$`.

EXAMPLE

```
> T = TIMER : SLEEP 0.5 : PRINT TIMER - T >= 0.5; TIMER - T < 0.6
-1 -1
```

SOUND

A statement: starts or stops a note; or sends the sound driver a command.

SYNTAX

```
SOUND channel [, note [, patch [, volume]]]
SOUND command$
```

ARGUMENTS

channel

A numeric expression: one of the sound driver's channels (section 15.1).

note

A numeric expression: the note's number. 60 is middle C and each step is a semitone, so 69 is the A above it (section 15.2). If it is left out, the channel's note is turned off.

patch

A numeric expression: the instrument to give the channel first (section 15.4).

volume

A numeric expression: the channel's volume, set before the note starts. It can be given only after a patch.

command$

A string expression: a line for the sound driver, as it would be written to `/dev/sndctl` (section 15.3): `"volume 150"`, `"pan 0 left"`.

DESCRIPTION

`SOUND channel, note` starts a note and returns at once. The note goes on sounding until `SOUND channel` stops it, or another note is started on the same channel. Timing is the program's business: `SOUND 0, 60 : SLEEP 0.5 : SOUND 0` plays middle C for half a second.

The patch and the volume, when they are given, are set before the note starts, and all of it is sent to the driver in one write, so that another program's commands can't come between.

The string form passes its text to the driver unchanged, and whatever the driver's commands can do, it does.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	A channel, note or patch number the driver doesn't have
258	`invalid argument`	The driver did not understand *command$*
2	`syntax error`	No arguments; or an argument skipped

NOTES

This is not QuickBASIC's `SOUND frequency, duration`. `SOUND 440, 18` is an error here: 440 is not a channel.

For a tune, `PLAY` is far less work.

SEE ALSO

`PLAY`, `BEEP`, `SLEEP`.

EXAMPLE

```
> SOUND 0, 60 : SLEEP 0.1 : SOUND 0 : PRINT "one note"
one note
> SOUND "volume 150" : SOUND "nonsense here"
invalid argument
```

SPACE$

A string function: a string of spaces.

SYNTAX

```
SPACE$(n)
```

ARGUMENTS

n

A numeric expression, 0 or more, rounded to the nearest integer.

RESULT

A string of *n* spaces; the empty string if *n* is 0.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is negative
13	`type mismatch`	*n* is a string

NOTES

`SPACE$` is the usual way to give a string variable a length before `GET` fills it.

SEE ALSO

`STRING$`, `SPC`, `GET`.

EXAMPLE

```
> PRINT "["; SPACE$(3); "]"; LEN(SPACE$(0)); LEN(SPACE$(1000))
[   ] 0  1000
```

SPC

Used in `PRINT`: `SPC(n)` writes *n* spaces. See `PRINT`.

SPRITE

A statement: shows, moves or hides a sprite.

SYNTAX

```
SPRITE n, x, y [, image$]
SPRITE n OFF
```

ARGUMENTS

n

A numeric expression, 1 to 127: which sprite.

x, y

Numeric expressions: where on the screen the sprite's top left corner goes, in pixels.

image$

A string expression of exactly 64 or 256 bytes: the sprite's picture, one byte to a pixel, each byte a colour number, row by row from the top left. 64 bytes make a sprite 8 pixels square; 256 make one 16 square. A pixel of colour 0 is transparent.

DESCRIPTION

A sprite is a small picture that the screen shows over everything else, wherever it is told to, without anything being drawn into the bitmap. Moving one is instant and leaves the bitmap as it was underneath.

`SPRITE n, x, y, image$` gives sprite *n* its picture and shows it at the point. `SPRITE n, x, y,` without a picture, moves it there with the picture it has. `SPRITE n OFF` hides it; its picture is kept.

Sprites need a Vera X card but no graphics mode: they show over text too.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is not 1 to 127; or *image$* is not 64 or 256 bytes long
13	`type mismatch`	A number where the picture should be, or a string where a number should be

NOTES

A picture is most easily built from `CHR$` and `STRING$`, or read from `DATA`. `STRING$(64, CHR$(7))` is a solid square of colour 7.

Sprites are the screen's, not HyBASIC's. One left showing stays on the screen after the program ends: hide it first.

SEE ALSO

`SCREEN`, `PALETTE`, `CHR$`, `STRING$`.

SQR

A numeric function: the square root.

SYNTAX

```
SQR(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The square root of *x*.

- If the root is exact, it is returned exactly, as an integer, a rational or a decimal: `SQR(16)` is 4, `SQR(1/4)` is 1/2, and `SQR(2 ^ 100)` is 2 to the power 50.
- Otherwise it is a decimal, rounded to the number of significant digits that `DIGITS` has set (12 unless it has been changed).
- If *x* is negative, the result is imaginary: the root of its size, times i. `SQR(-16)` is `4i`. It is not an error.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

A program brought from another BASIC that relies on `SQR` of a negative number being an error will go on with an imaginary number instead, and fail later, at the first function that won't take one. Test the sign first.

SEE ALSO

`EXP`, `LOG`, `DIGITS`, `COMPLEX`.

EXAMPLE

```
> PRINT SQR(16); SQR(2); SQR(1/4); SQR(0.25); SQR(-16); SQR(-2); SQR(2 ^ 100)
 4  1.41421356237  1/2  0.5  4i  1.41421356237i  1125899906842624
```

STATIC

A statement, and a word on a procedure's first line: makes a procedure's variables keep their values between calls.

SYNTAX

```
STATIC name[()] [AS type] [, name ...] ...
SUB name [(parameters)] STATIC
FUNCTION name [(parameters)] STATIC
```

DESCRIPTION

A procedure's own variables are normally made afresh, 0 or empty, each time it is called. A variable named in a `STATIC` statement inside the procedure is made once and keeps its value from one call to the next. It is still the procedure's own: no other part of the program can see it.

`STATIC` at the end of a `SUB` or `FUNCTION` line does the same for every variable of the procedure.

ERRORS

ERR	MESSAGE	CAUSE
85	`not in a procedure`	A `STATIC` statement in the main program

SEE ALSO

`SHARED`, `SUB`, `FUNCTION`, `DIM`.

EXAMPLE

`K` is static and counts the calls; `M` is not, and starts at 0 each time:

```
> 10 SUB Keeps : STATIC K : K = K + 1 : M = M + 1 : PRINT K; M : END SUB
> 20 Keeps : Keeps : Keeps
> RUN
 1  1
 2  1
 3  1
```

STATUS

A numeric function: how the last `SHELL` command ended.

SYNTAX

```
STATUS
```

RESULT

The exit code of the command line that the most recent `SHELL` or `SHELL$` ran: 0 if it succeeded, and not 0 if it failed. For a line of several commands it is the last one's.

SEE ALSO

`SHELL`, `SHELL$`, `SYSTEM`.

STEP

Part of `FOR`, where it gives the amount the counter changes by each time. In the graphics statements, `STEP` before a point means that its coordinates are an offset from the last point. See `FOR` and `PSET`.

STOP

A statement: stops the program in a way that can be continued.

SYNTAX

```
STOP
```

DESCRIPTION

The program stops, `Break in line` and the line's number are printed, and the prompt returns. Everything is as it was at that moment: the variables can be looked at and changed, and `CONT` carries on with the statement after the `STOP`. CTRL-C stops a running program in the same way, wherever it is.

SEE ALSO

`CONT`, `END`.

EXAMPLE

See `CONT`.

STR$

A string function: a number's text.

SYNTAX

```
STR$(x [, base$])
```

ARGUMENTS

x

A numeric expression of any kind.

base$

A string expression: a base string (section 18.4), `"x"` for hex, `"b"` for binary.

RESULT

With one argument, the text `PRINT` would write for *x* in the base that `BASE` has set, without the space after it: a number that isn't negative has a space in front, where a negative one has its minus sign.

With a base, the text of *x* in that base, with no space in front.

Either way the text is the number's exact value: all of an integer's digits, a rational with its `/`, a decimal with its point.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*base$* isn't a base string
13	`type mismatch`	*x* is a string, or *base$* a number

NOTES

To drop the leading space, `LTRIM$(STR$(x))`; or give the base, `STR$(x, "d")`.

SEE ALSO

`VAL`, `HEX$`, `OCT$`, `BASE`, `PRINT`.

EXAMPLE

```
> PRINT "["; STR$(5); "]["; STR$(-5); "]["; STR$(1/3); "]["; STR$(255, "x"); "][";
STR$(5, "b"); "]"
[ 5][-5][ 1/3][FF][101]
```

STRING$

A string function: one character, repeated. `STRING` is also the name of the string type, in `AS STRING`.

SYNTAX

```
STRING$(n, s$)
STRING$(n, code)
```

ARGUMENTS

n

A numeric expression, 0 or more, rounded to the nearest integer: how many.

s$

A string expression of one byte or more. Its first byte is the one repeated; the rest are not used.

code

A numeric expression from 0 to 255: the byte to repeat, given as its code.

RESULT

A string of *n* bytes, all the same.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is negative; *s$* is empty; *code* is not 0 to 255

SEE ALSO

`SPACE$`, `CHR$`.

EXAMPLE

```
> PRINT STRING$(3, "ab"); "|"; STRING$(3, 65); "|"; STRING$(0, "x"); "|"
aaa|AAA||
```

SUB ... END SUB

A block: defines a subprogram, a named piece of the program with parameters and variables of its own.

SYNTAX

```
SUB name [(parameter [, parameter] ...)] [STATIC]
    [statements]
END SUB
```

ARGUMENTS

name

The subprogram's name.

parameter

A variable's name, optionally followed by `AS type`; or an array's, written with empty parentheses.

STATIC

The subprogram's variables keep their values from one call to the next. See `STATIC`.

DESCRIPTION

A `SUB` is run by calling it: its name, then an argument for each parameter, with or without the word `CALL` and with or without parentheses. See `CALL`, which describes how arguments are passed: a variable by reference, anything else by value.

Inside the SUB, the names of variables are its own. Each is made anew at every call, 0 or empty, and is gone when the SUB returns, and it has nothing to do with a variable of the same name anywhere else. The exceptions are the parameters, the names declared SHARED or STATIC, and constants.

The SUB returns when control reaches END SUB or an EXIT SUB. A SUB may call itself.

Subprograms are found when the program is compiled, wherever they are in its text, so one can be called from a line before it. When control reaches a SUB line in the ordinary run of the program, it skips to after END SUB.

ERRORS

ERR	MESSAGE	CAUSE
10	`duplicate definition`	Two procedures of one name
80	`block without its start or end`	SUB with no END SUB
2	`syntax error`	SUB typed at the prompt: a subprogram can be defined only in a program

The errors of calling one are under CALL.

SEE ALSO

CALL, FUNCTION, EXIT, SHARED, STATIC, GOSUB.

EXAMPLE

```
> 10 SUB Greet (N$, Times)
> 20   FOR I = 1 TO Times : PRINT "Hello, "; N$ : NEXT
> 30 END SUB
> 40 I = 99 : Greet "Sam", 2 : PRINT I
> RUN
Hello, Sam
Hello, Sam
 99
```

SWAP

A statement: exchanges the values of two variables.

SYNTAX

```
SWAP a, b
```

ARGUMENTS

a, b

Two variables, array elements or record fields, of the same type.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	One is a number's and the other a string's
40	`variable required`	An argument that is not a variable

EXAMPLE

```
> A = 1 : B = 2 : SWAP A, B : PRINT A; B : DIM V(1) : V(0) = 7 : V(1) = 8 : SWAP V(0),
V(1) : PRINT V(0); V(1)
 2  1
 8  7
```

SYS

A statement: calls machine code at an address, or a system call by its name.

SYNTAX

```
SYS address [, a [, x [, y]]]
SYS "NAME" [, a [, x [, y]]]
```

ARGUMENTS

address

A numeric expression, 0 to 65,535: the address of a machine code routine.

NAME

A string expression: the name of a system call (Appendix K), in capitals or small letters.

a, x, y

Numeric expressions: the values of the A, X and Y registers as the code is entered. Each is 0 if it is left out.

DESCRIPTION

The code is called as a subroutine (`jsr`) and must return with `rts`. When it does, its A, X and Y and the flags are kept for `RREG` to read, and the program goes on.

Anything the program has printed is written out before the call, so output appears in order even if the code prints too.

A system call that takes its arguments in the call registers r0 to r15 needs them put there first. They are two bytes each in the zero page from address 2 (r0 is at 2 and 3, r1 at 4 and 5), and `POKE` sets them; HyBASIC leaves r0 and r1 as the program's `POKE`s made them until the call.

ERRORS

ERR	MESSAGE	CAUSE
259	`no such call`	*NAME* is not a system call's name
13	`type mismatch`	A register's value is a string

A system call that fails is not an error in the program. It returns with the carry flag set and its error code in A: read them with `RREG`.

NOTES

The code runs inside HyBASIC's task with nothing to protect HyBASIC from it. Code that doesn't return, or writes over HyBASIC's memory, ends the session.

For code of the program's own, an `ASM` block is easier than `POKE`s: it is assembled with the program, and `CALL ASM` finds it by name.

SEE ALSO

`RREG`, `CALL`, `ASM`, `PEEK`, `POKE`.

EXAMPLE

```
> SYS "getpid" : RREG A : PRINT A
 4
> SYS "NOSUCH"
no such call
```

SYSTEM

A statement: leaves HyBASIC.

SYNTAX

```
SYSTEM [n]
BYE [n]
```

ARGUMENTS

n

A numeric expression from 0 to 255, rounded to the nearest integer: the exit code. 0, for success, if it is left out.

DESCRIPTION

HyBASIC ends, from a program or from the prompt, and control returns to whatever started it, usually a shell. Open files are closed. The program in memory is lost unless it has been saved; there is no question first.

The exit code is what the shell sees as the command's status (section 8.9): 0 is success, and anything else a failure of the program's own choosing.

`BYE` is the same statement.

SEE ALSO

`END`, `STOP`, `SHELL`.

EXAMPLE

```
% echo 'PRINT "leaving" : SYSTEM 3 : PRINT "not printed"' >bye.bas
% basic bye.bas; echo status $status
leaving
status 3
```

TAB

Used in `PRINT`: `TAB(n)` moves to column *n*. See `PRINT`.

TAN

A numeric function: the tangent of a real number.

SYNTAX

```
TAN(x)
```

ARGUMENTS

x

A numeric expression whose value is real: an angle in radians.

RESULT

The tangent of *x*. `TAN(0)` is the integer 0. Any other result is a decimal, correct to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

The tangent has no value at π/2, but no number a program can write is exactly π/2. `TAN(PI / 2)` is therefore not an error: it is a very large number, the tangent of the angle that `PI / 2` really is.

SEE ALSO

`SIN`, `COS`, `ATN`, `PI`.

EXAMPLE

```
> PRINT TAN(0); TAN(1); TAN(PI / 4)
 0  1.55740772465  1
```

THEN

Part of `IF`. See `IF`.

TIME$, TIMER

Functions: the time of day, as text and as a number of seconds.

SYNTAX

```
TIME$
TIMER
```

RESULT

`TIME$` is a string of eight characters, `hh:mm:ss`, on the 24-hour clock.

`TIMER` is the number of seconds since midnight, as a decimal with a fraction: the system's clock ticks 200 times a second, so it moves in steps of 0.005.

Both come from the system's clock. On a Hydra with no clock chip that clock starts at midnight when the system starts, so both then tell the time since it was switched on.

NOTES

To time something, take `TIMER` before and after and subtract. The answer is wrong, by a day's worth of seconds, if midnight comes between.

SEE ALSO

`DATE$`, `SLEEP`, `RANDOMIZE`.

EXAMPLE

```
> PRINT LEN(TIME$); MID$(TIME$, 3, 1) : T = TIMER : SLEEP 1 : PRINT TIMER - T >= 1
 8 :
-1
```

TO

Part of `FOR` (`FOR i = a TO b`), of `DIM` (`DIM a(1 TO 10)`) and of `CASE` (`CASE 1 TO 5`). See those.

TYPE ... END TYPE

A block: defines a record type, a group of named fields that travel together.

SYNTAX

```
TYPE name
    field AS type
    [field AS type] ...
END TYPE
```

ARGUMENTS

name

The type's name.

field

A field's name.

type

`INTEGER`, `LONG`, `SINGLE`, `DOUBLE` or `NUMBER`, for a number; `STRING`, for a string of any length.

DESCRIPTION

`TYPE` makes no variable. It describes a kind of variable, which `DIM` then makes: `DIM P AS Point`, or an array of them, `DIM Pts(10) AS Point`. A field is written with a full stop: `P.X`, `Pts(3).Label`. It is used like any variable of its type.

Assigning one record to another of the same type copies every field. A record can be written to and read from a `BINARY` file with `PUT` and `GET`.

The type is known in the whole program, procedures included, from where it is defined.

ERRORS

ERR	MESSAGE	CAUSE
88	`no such field`	A field name that the record's type doesn't have. Found when the program is compiled
80	`block without its start or end`	`TYPE` with no `END TYPE`

NOTES

QuickBASIC's fixed-length strings, `STRING * 10`, are not here. A `STRING` field holds a string of whatever length it is given.

A full stop is also a legal character in an ordinary name. `P.X` is the field X of P only if `P` has been declared as a record; otherwise it is a variable called `P.X`.

SEE ALSO

`DIM`, `PUT`, `GET`, `SUB`.

EXAMPLE

```
> 10 TYPE Point
> 20   X AS INTEGER
> 30   Y AS INTEGER
> 40   Label AS STRING
> 50 END TYPE
> 60 DIM P AS Point, Q AS Point : P.X = 3 : P.Y = 4 : P.Label = "here" : Q = P : Q.X = 9
> 70 PRINT P.X; P.Y; P.Label; Q.X; Q.Label
> 80 DIM Pts(2) AS Point : Pts(1).X = 5 : PRINT Pts(1).X; Pts(0).X
> RUN
 3  4 here 9 here
 5  0
```

UNTIL

Part of `DO` loops: `DO UNTIL condition` and `LOOP UNTIL condition`. See `DO`.

USING

Part of `PRINT USING`, which writes values in a format. See `PRINT`.

VAL

A numeric function: the number that a string's text says.

SYNTAX

```
VAL(s$)
```

ARGUMENTS

s$

A string expression.

RESULT

The number written at the start of *s$*, after any spaces. It can be in any form a program's own numbers take: a whole number, a decimal, a rational (`1/3`), a complex number (`1+2i`), one with an exponent (`-3.5e2`), or one with a prefix that says its base (`&HFF`). Without a prefix it is read in the base that `BASE` has set.

Reading stops at the first character that can't be part of the number, and what was read up to there is the result: `VAL("12abc")` is 12, and `VAL("1,000")` is 1. If there is no number at the start, the result is 0. It is never an error for the text to be wrong.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*s$* is a number

NOTES

`VAL` reads a number. It doesn't work anything out: `VAL("2^3")` is 2.

Since a wrong text gives 0, `VAL` can't tell "0" from nonsense. A program that must know checks the text itself.

SEE ALSO

`STR$`, `INPUT`, `BASE`, `CVN`.

EXAMPLE

```
> PRINT VAL("12"); VAL(" -3.5e2"); VAL("1/3"); VAL("&HFF"); VAL("12abc"); VAL("abc");
VAL("")
 12 -350  1/3  255  12  0  0
```

VIEW

A statement: moves the origin of the drawing coordinates to a rectangle of the bitmap.

SYNTAX

```
VIEW [[SCREEN] (x1, y1)-(x2, y2) [, fill [, border]]]
```

ARGUMENTS

x1, y1, x2, y2

Numeric expressions: two opposite corners of a rectangle, the **viewport**, in pixels of the bitmap.

fill, border

Accepted, evaluated and ignored.

DESCRIPTION

After `VIEW (x1, y1)-(x2, y2)`, the drawing statements' coordinates are measured from the viewport's top left corner: the point (0, 0) is the pixel (x1, y1). A drawing can be moved about the screen by changing one statement.

With `SCREEN`, the coordinates stay the bitmap's own, and the viewport matters only to a `WINDOW`.

A `WINDOW` set afterwards spreads its coordinates over the viewport instead of over the whole bitmap, which is the usual reason for a viewport: a graph in one corner of the screen.

`VIEW` alone makes the viewport the whole bitmap again.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set
13	`type mismatch`	An argument is a string

NOTES

Unlike QuickBASIC's, the viewport does not clip. A point outside it is still drawn, as long as it is on the bitmap, and the viewport is neither cleared nor outlined.

SEE ALSO

`WINDOW`, `SCREEN`, `POINT`.

EXAMPLE

```
> SCREEN 13 : VIEW (100, 100)-(200, 200) : PSET (0, 0), 13 : PRINT POINT(0, 0)
 13
> VIEW : PRINT POINT(100, 100); POINT(0, 0)
 13  0
> SCREEN 0
```

WHILE ... WEND

A statement block: a loop that repeats while a condition holds.

SYNTAX

```
WHILE condition
    [statements]
WEND
```

ARGUMENTS

condition

A numeric expression. It holds if its value is not 0.

DESCRIPTION

The condition is tested before each pass, the first included. While it holds, the statements run and control goes back to the test; when it doesn't, the program goes on after `WEND`.

It is the same loop as `DO WHILE condition` ... `LOOP`, kept for older programs, with one difference: there is no `EXIT` for it. `EXIT DO` inside a `WHILE` loop is an error unless a `DO` loop encloses it, and then it leaves that one.

ERRORS

ERR	MESSAGE	CAUSE
29	`WHILE without WEND`	A `WHILE` whose `WEND` isn't there. Found when the program is compiled
30	`WEND without WHILE`	A `WEND` with no loop open
13	`type mismatch`	The condition is a string

SEE ALSO

`DO`, `FOR`, `EXIT`.

EXAMPLE

```
> I = 0 : WHILE I < 3 : I = I + 1 : WEND : PRINT I : WHILE 0 : PRINT "never" : WEND
 3
```

WIDTH

A statement: sets the width of the line that `PRINT` works to.

SYNTAX

```
WIDTH n
```

ARGUMENTS

n

A numeric expression from 1 to 255: the number of columns. 80 when HyBASIC starts.

DESCRIPTION

The width decides how many print zones fit on a line: `PRINT a, b, c` moves to a new line when the next zone would start beyond it. It is HyBASIC's idea of the line, used for its own layout.

It does not change the terminal or the screen, and it does not break long text: a string longer than the width is written whole, and the terminal wraps it wherever its own edge is.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 1 or more than 255

SEE ALSO

`PRINT`, `POS`, `TAB`.

EXAMPLE

```
> WIDTH 40 : PRINT 1, 2, 3, 4 : WIDTH 80
 1             2             3
 4
```

WINDOW

A statement: gives the drawing statements coordinates of the program's own choosing.

SYNTAX

```
WINDOW [[SCREEN] (x1, y1)-(x2, y2)]
```

ARGUMENTS

x1*, *y1*, *x2*, *y2

Numeric expressions: the coordinates that the left edge, one horizontal edge, the right edge and the other horizontal edge of the drawing area are to have.

DESCRIPTION

After `WINDOW (x1, y1)-(x2, y2)`, a point's *x* runs from *x1* at the left edge of the bitmap to *x2* at the right, and its *y* from *y1* at the **bottom** to *y2* at the top, as a graph is drawn on paper. Each point a statement is given is worked out exactly in those coordinates and then rounded to a pixel. `WINDOW (-1, -1)-(1, 1)` puts (0, 0) at the middle of the screen.

With `SCREEN`, *y* runs the screen's way instead: *y1* is at the top and *y2* at the bottom.

The window covers the viewport, if `VIEW` has set one, and the whole bitmap otherwise. Every statement that takes a point follows it, `POINT` included; a `CIRCLE`'s radius is in the window's x units.

`WINDOW` alone returns to pixels.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set
13	`type mismatch`	An argument is a string

SEE ALSO

`VIEW`, `SCREEN`, `POINT`, `CIRCLE`.

EXAMPLE

The point (50, 50) of a window 100 units each way is the pixel in the middle of the bitmap:

```
> SCREEN 13 : WINDOW (0, 0)-(100, 100) : PSET (50, 50), 12 : PRINT POINT(50, 50)
 12
> WINDOW : PRINT POINT(160, 120)
 12
> SCREEN 0
```

WRITE

A statement: writes values in a form that `INPUT` can read back.

SYNTAX

```
WRITE [#f,] [item [, item] ...]
```

ARGUMENTS

item

An expression of either type.

f

The number of a file opened `FOR OUTPUT`, `FOR APPEND` or `FOR BINARY`.

DESCRIPTION

The items are written on one line with a comma between each and nothing else: no spaces, no zones. A string is put in double quotes. A number is written as its text with no space before or after it. The line is then ended. `WRITE` with no items writes an empty line.

This is exactly the form `INPUT` and `INPUT #` take apart: the commas separate the fields, and the quotes keep a string's own commas and spaces inside one field. A line written by `WRITE #` and read by `INPUT #` with the same list of variables gives back the values that were written.

The items are separated by commas only. A semicolon is a syntax error here.

ERRORS

ERR	MESSAGE	CAUSE
2	`syntax error`	A semicolon between items
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR INPUT`

NOTES

A string that itself contains a double quote can't be read back whole: the quote ends the field. There is no way to write one that `INPUT` will accept.

SEE ALSO

`PRINT`, `INPUT`, `OPEN`.

EXAMPLE

```
> WRITE 1, "two", 1/3, -4.5 : WRITE "a,b", 5
1,"two",1/3,-4.5
"a,b",5
> OPEN "/ram/w.txt" FOR OUTPUT AS #1 : WRITE #1, "Sam", 12, "x, y" : CLOSE
> OPEN "/ram/w.txt" FOR INPUT AS #1 : INPUT #1, N$, G, T$ : PRINT N$; G; "["; T$; "]" :
CLOSE
Sam 12 [x, y]
```

XOR

An operator: the bitwise exclusive OR of two integers.

SYNTAX

```
a XOR b
```

RESULT

An integer. Each bit of it is 1 where the same bit of *a* and of *b* differ, and 0 where they are alike.

For two truth values, −1 and 0, the result is true if exactly one of them is.

Both operands are always evaluated, and they are rounded and treated as `AND`'s are: see `AND`. `XOR` binds less tightly than `OR` and more tightly than `EQV`.

EXAMPLE

```
> PRINT 5 XOR 3; -1 XOR -1; (1 = 1) XOR (1 = 2); -1 XOR 1
 6  0 -1 -2
```

CHAPTER 2

Commands

A command is a program in `/bin`, or one of the shell's own, which are at the end of the chapter. This chapter describes each one's command line: what it takes, what it does with it, what it prints when it can't, and the status it ends with. The languages and the larger tools have chapters of their own in the manual, to which their entries here point.

The exit status. Every program ends with a status, which rc keeps in `$status` (section 8.9). It is empty for success. For failure it is a short text: most often `1`, and `usage` when the arguments made no sense, in which case the program has printed its usage line. A few programs have statuses of their own, and their entries say so.

Messages. What a program has to say about a failure goes to standard error, not into a pipe's data, as `name: what: why`. The *why* is usually the system's own text for an error (Appendix G): `not found`, `read-only`, `is a directory`.

Options are single letters after a `-`, and letters can be joined: `-ld` is `-l -d`. They come before the other arguments. `-` alone is not an option, and unless an entry says otherwise it is not standard input either: it is a file called `-`.

Paths. A path in an argument can be at most 63 characters long once it has been made absolute.

Standard input. The programs that read files read standard input when they are given none, which is how they are used in a pipe.

as

A program: the 65C02 assembler.

SYNTAX

```
as [-bl] {file.s | -} [out | -]
```

ARGUMENTS

file.s

The source. `-` reads the source from standard input.

out

The file to write. If it is left out, the source's name without its `.s`; for a source read from standard input, standard output. `-` writes to standard output.

`-l`

Write a labels file too, *out*`.lbl`: every label and its address, which `db` and `dis -l` read. It needs a named output.

`-b`

Write the bytes alone, with no program header, assembled from the address `.org` gives, or `$0800` if there is none.

DESCRIPTION

The source is in ca65's assembly language (Chapter 27). Without `-b` the output is a program in the form the system loads and runs: `as hi.s` makes `hi`, which `./hi` runs. The assembler is the same code that assembles the `ASM` blocks of a HyBASIC program.

EXIT STATUS

Empty on success. `1` if the source can't be read, or has a mistake, which is reported as `as: file:line: message`. `usage` for bad arguments.

SEE ALSO

`dis`, `db`; Chapter 27.

EXAMPLE

```
% echo ' lda #' >bad.s
% as bad.s; echo status $status
as: bad.s:1: a bad expression
status 1
```

basic

A program: HyBASIC.

SYNTAX

```
basic [-l] [file [argument ...]]
```

ARGUMENTS

file

A program to load and run. When it ends, HyBASIC ends. The arguments after it are the program's own, which it reads with `ARG$` and `COMMAND$`.

`-l`

Start as a login shell would: make the default namespace and run the profile first (section 8.16).

DESCRIPTION

With no file, HyBASIC starts at its prompt, where lines are run as they are typed and numbered lines build a program. `SYSTEM` or `BYE` leaves it.

EXIT STATUS

At the prompt: the status given to `SYSTEM` or `BYE`, empty if none. Running a file: empty if the program ran to its end; the status given to `SYSTEM`; `1` if the file can't be read or the program stopped with an error, which is printed as `file:line: message`.

SEE ALSO

Chapter 1 of this book; Chapters 23 and 24.

calc

A program: works out an expression in the Hydra's numbers.

SYNTAX

```
calc [-b base] [-d digits] [expression ...]
```

ARGUMENTS

expression
: The arguments, joined with spaces, are one expression. With none, `calc` reads standard input and works out each line, until the end of the input.

-b *base*
: Print answers in this base: a base string (section 18.4).

-d *digits*
: The number of significant digits for results that can't be exact, 1 to 100. 12 if it is not given.

DESCRIPTION

The numbers are those of Chapter 18: integers of any length, exact fractions, decimals and complex numbers. The operators are `+`, `-`, `*`, `/`, `%` (the remainder of dividing integers) and `^` (a power), with parentheses. The functions, each applied to what follows it, are `sqrt`, `exp`, `log`, `sin`, `cos`, `tan`, `atan`, `abs`, `floor`, `round`, `truncate`, `fib`, `numerator`, `denominator`, `re`, `im`, `rational` and `random`; and with two arguments in parentheses, `gcd`, `pow`, `complex` and `fixed`. The constants are `pi`, `e` and `i`.

Each answer is printed on a line of its own. A mistake in one expression is reported and, when lines are being read, the next line is gone on with.

The shell reads the command line first. `^`, `(`, `)`, `#`, `*` and `'` all mean something to rc, so an expression that uses them goes in quotes: `calc '2^10'`.

EXIT STATUS

Empty on success. The message itself for an expression that failed: `division by zero`, `not a base`. `usage` for a bad option.

SEE ALSO

Chapter 18.

EXAMPLE

```
/> calc 1/3 + 1/6
1/2
/> calc -b x 255
FF
/> calc -d 5 sqrt 2
1.4142
/> calc 1 / 0
calc: division by zero
```

cat

A program: copies files to standard output, one after another.

SYNTAX

```
cat [file ...]
```

DESCRIPTION

Each file is read and written to standard output, 512 bytes at a time, in the order given. Nothing is added between them and nothing is changed: `cat` copies bytes, and is as good for a program or a picture as for text. With no files it copies standard input.

A file that can't be read is reported, and `cat` goes on with the next.

Anything that can be read can be given: a device (`cat /dev/time`), or a directory, which reads as its files' 64-byte records.

EXIT STATUS

Empty if every file was copied. `1` if any could not be read. `write error` if standard output could not be written: a full disk, or a pipe whose reader has gone.

NOTES

`-` is not standard input here: `cat -` looks for a file named `-`.

SEE ALSO

`more`, `head`, `tail`, `xd`, `cp`, `tee`.

EXAMPLE

```
% echo one >a; echo two >b
% cat a nothere b; echo status $status
one
cat: nothere: not found
two
status 1
```

cmp

A program: compares two files, byte by byte.

SYNTAX

```
cmp file1 file2
```

DESCRIPTION

The two files are read together. If every byte is the same and they end together, `cmp` prints nothing. At the first byte that differs, it prints the files' names and the byte's position, counted from 1, and stops. If one file ends while they still agree, it says which.

EXIT STATUS

Empty if the files are the same. `1` if they differ, or if either can't be read. `usage` unless there are exactly two names.

SEE ALSO

`cp`, `xd`.

EXAMPLE

```
% echo one >a; echo two >b; echo onetwo >ab
% cmp a a; echo status $status
status
% cmp a b; echo status $status
a b differ: byte 1
status 1
% cmp a ab
a ab differ: byte 4
```

cp

A program: copies files, and with `-r` directories.

SYNTAX

```
cp [-r] from to
cp [-r] from ... dir
```

ARGUMENTS

-r

Copy directories too: each one's whole tree.

from

A file to copy, or with `-r` a directory. There can be several if the last name is a directory.

to

The copy's name. If it is an existing directory, the copy goes into it under *from*'s own name.

dir

An existing directory. Each *from* is copied into it under its own name.

DESCRIPTION

Each *from* is handled in turn, and one that fails doesn't stop the rest.

A file is copied by opening it for reading, creating the destination, and reading and writing 512 bytes at a time until a read returns nothing. Creating the destination empties a file that is already there. The copy is a new file: it has its own date, the time of copying, and is not read-only even if the original is.

Before copying, `cp` compares the two files' device, disk and file identifier. If they are the same file, by whatever two names, it refuses to copy it onto itself, which would otherwise empty it.

Anything that can be opened and read can be a *from*, a device included: `cp /dev/time now` copies what one read of the clock gives.

Without `-r`, a *from* that is a directory is reported and skipped. With `-r`, the directory is made at the destination (one already there is used as it is), and then everything in it is copied in the same way, down to 16 directories deep.

EXIT STATUS

Empty if every file was copied. `1` if any was not. `usage`, after a line saying how `cp` is used, if there are fewer than two names or an option other than `-r`.

ERRORS

Each is printed as `cp: name: reason`, where *name* is the file that could not be read or written.

REASON	CAUSE
`not found`	*from* isn't there, or a directory in *to*'s path isn't
`is a directory`	*from* is a directory, and there's no `-r`
`not a directory`	Several *from*s, and the last name isn't a directory
`the same file`	*from* and *to* are one file
`read-only`	The destination's disk can't be written (`/rom`)
`name too long`	The destination's path would pass 63 characters

Any other reason is the system's own error message for what failed (Appendix G): a full disk, for one.

NOTES

If a read or a write fails part-way, the destination is left holding what was written up to then.

`cp` doesn't ask before it overwrites.

SEE ALSO

`mv`, `rm`, `mkdir`, `cat`, `cmp`.

EXAMPLE

```
% echo hello >a.txt
% cp a.txt b.txt; cat b.txt
hello
% mkdir d; cp a.txt b.txt d; ls d
a.txt
b.txt
% cp d e; echo $status
cp: d: is a directory
1
% cp -r d e; ls e
a.txt
b.txt
% cp a.txt a.txt
cp: a.txt: the same file
% cp a.txt /rom/x
cp: /rom/x: read-only
% cp a.txt; echo $status
usage: cp [-r] from to, or cp [-r] from ... dir
usage
```

date

A program: prints the date and the time.

SYNTAX

```
date [-n]
```

ARGUMENTS

-n

Print the time as one number: the seconds since the start of 1 January 2000.

DESCRIPTION

Without `-n`, the date and time are printed as `yyyy-mm-dd hh:mm:ss`, which is what `/dev/time` reads as. The clock is the system's; on a Hydra with no clock chip it starts at 2000-01-01 00:00:00 when the system does.

`date` doesn't set the clock. Write the time to it: `echo 2026-10-03 15:04:05 >/dev/time`.

EXIT STATUS

Empty on success. `usage` for any argument but `-n`.

SEE ALSO

`sleep`; section 17.5.

db

A program: the debugger.

SYNTAX

```
db program [argument ...]
db -p task
```

ARGUMENTS

program
A program to start under the debugger, stopped before its first instruction, with its arguments.

`-p` *`task`*
Attach to a task that is already running, by its number.

DESCRIPTION

`db` shows a program's registers and memory, runs it an instruction or a call at a time, and stops it at breakpoints. Its own commands are typed at its `db>` prompt (Chapter 29).

EXIT STATUS

`usage` with no arguments.

SEE ALSO

`as`, `dis`, `ps`; Chapter 29.

df

A program: lists the disks, with their sizes and the space free.

SYNTAX

```
df
```

DESCRIPTION

One line is printed for each disk the storage driver has started, under a heading: its name (the name it has under `/sd`, or the letter of a built-in disk), its kind, the size of its file system, how much of that is free, and its label. A disk that has no file system is listed with its own size.

Arguments are ignored.

EXIT STATUS

Empty.

SEE ALSO

`du`, `fsck`, `label`, `mkfs`; Chapter 14.

EXAMPLE

```
/> df
disk  kind   size        free        label
0     sdhc   16380 KB    15956 KB    HYDRA-16
x     rom    623 KB      10 KB       ROM
r     ram    255 KB      252 KB      RAM
s     sram   511 KB      510 KB      SRAM
```

dis

A program: the disassembler.

SYNTAX

```
dis [-cnw] [-l labels] [-o addr] file
```

ARGUMENTS

file
: A program, as `as` makes one, or with `-o` a file of raw bytes.

-l *labels*
: Use the names in a labels file (`as -l`'s) for the addresses they belong to.

-n
: Don't name the system calls and the call registers: show their addresses.

-w
: Wide: set the instructions out in the columns the system's own sources use.

-c
: Treat every byte as code. Without it, `dis` follows the program from its entry and takes what is never reached as data.

-o *addr*
: The file has no program header: it is bytes to be loaded at *addr*, given in hex.

DESCRIPTION

The program is written to standard output as assembly language that `as` accepts: `dis prog | as - copy` makes a copy that is byte for byte the same (Chapter 28).

EXIT STATUS

Empty on success. `1` if the file can't be read, and for bad arguments, after the usage line.

SEE ALSO

`as`, `db`, `xd`; Chapter 28.

du

A program: reports how much space files take.

SYNTAX

```
du [-a] [name ...]
```

ARGUMENTS

name

A file or a directory. The current directory if none is given.

-a

Print a line for every file as well as for every directory.

DESCRIPTION

For each name, `du` adds up the lengths of everything in its tree, each file's length rounded up to a whole kilobyte, and prints a line for each directory with the total of what is in and under it: the number of kilobytes, a tab, and the path. The deepest directories come first and the name itself last. A name that is a file gets its own line.

The figure is the files' lengths, not the space the file system gives them, which is in whole clusters and is a little more.

EXIT STATUS

Empty on success. `1` if a name isn't there. `usage` for an unknown option.

SEE ALSO

`df`, `ls`.

EXAMPLE

```
% echo 12345 >s1; mkdir t; echo x >t/s2
% du -a s1 t
1       s1
1       t/s2
1       t
```

echo

A program: prints its arguments.

SYNTAX

```
echo [-n] [word ...]
```

ARGUMENTS

-n

Don't end the line.

DESCRIPTION

The words are written to standard output with one space between each, and then a line end. The spaces are `echo`'s: however many there were between the words on the command line, the shell has already made them separate arguments. To keep spaces, quote them.

`-n` is the only option, and only as the first argument. Anything else that starts with – is a word and is printed.

The whole line is written in one piece (up to 256 bytes at a time), so two programs writing lines to the same place don't mix their lines' halves.

EXIT STATUS

Empty on success. `write error` if the output can't be written.

SEE ALSO

`cat`; section 8.3.

EXAMPLE

```
/> echo a  b   c; echo -n x; echo y; echo -x z
a b c
xy
-x z
```

ed

A program: the line editor.

SYNTAX

```
ed [file]
```

ARGUMENTS

file

The file to edit. One that doesn't exist is not an error: `ed` says `new file` and starts with nothing, and the file is made when the text is first written with `w`.

DESCRIPTION

`ed` holds the file's lines in memory and changes them by commands typed at its `*` prompt, each naming the lines it is about by their numbers. It needs nothing of the terminal but lines in and lines out, so it works over any connection, in a script, and with its input from a pipe. Its commands are in Chapter 11.

Nothing is written to the file until a `w` command.

SEE ALSO

`edit`; Chapter 11, under `ed`.

edit

A program: the screen editor.

SYNTAX

```
edit [+n] [file ...]
```

ARGUMENTS

file

A file to edit. Several can be given, each in a buffer of its own; one that doesn't exist starts empty. With none, `edit` starts with an empty, unnamed buffer.

+n

Start with the cursor on line *n*.

DESCRIPTION

`edit` shows the text on the whole screen and the cursor moves through it; what is typed goes in where the cursor is. Its commands are control keys, listed along the bottom of its screen, in the manner of `nano` (Chapter 11). It needs a terminal that understands the cursor's escape sequences (section 13.7), which HydraOS's own screen and any terminal emulator do.

The text is kept in the task's RAM banks, so a file's size is limited by those, not by the 32K of ordinary memory.

HyBASIC's `EDIT` command runs this program on the program in memory.

SEE ALSO

`ed`, `more`; Chapter 11, under `edit`.

forth

A program: HyForth.

SYNTAX

```
forth [-l] [file [argument ...]]
```

ARGUMENTS

file

A file of Forth to run. When it has been read to its end, HyForth ends. The arguments after it are the script's own (`argc` and `arg`, section 20.9).

-l

Start as a login shell: build the namespace and run the profile, `/lib/forth/profile.fs`, first (section 8.16).

DESCRIPTION

With no file, HyForth starts at its prompt, which shows the current directory. It is the shell a Hydra starts with: besides Forth's words it runs programs by name (section 8.15). `bye` leaves it.

EXIT STATUS

`1` if the file can't be read.

SEE ALSO

Chapter 4 of this book; Chapters 19 and 20.

free

A program: reports how much memory there is.

SYNTAX

```
free
```

DESCRIPTION

Two lines are printed. The first is the RAM every task has for itself in banks: 128K for each RAM module fitted. The second is the shared RAM: its size, how much of it is in shared segments and how many of those there are, and how much is free.

The figures are for the machine, not for one program. To see what a task is using, look in `/proc` (section 17.8).

Arguments are ignored.

EXIT STATUS

Empty.

SEE ALSO

`ps`, `df`; Appendix F.

EXAMPLE

```
/> free
ram     384 KB a task (3 modules)
shared  1024 KB, 512 KB in segments (1), 512 KB free
```

fsck

A program: checks a disk's file system, and can repair it.

SYNTAX

```
fsck [-f] disk
```

ARGUMENTS

disk

The disk's name: the name it has under `/sd`, or the letter of a built-in disk (`r` is the RAM disk).

`-f`

Fix what is found wrong.

DESCRIPTION

The check is done by the storage driver, which `fsck` asks through the disk's control file. It reads every directory and every file's list of clusters, and compares what the files use with the free map. It then prints the disk's label, its free space, and what it found:

- **lost**: clusters marked as used that no file has. They are wasted space.
- **unmarked**: clusters a file has that are marked free. Something else could be given them.
- **twice**: clusters that two files both claim.

All three are 0 on a sound disk. `-f` puts the free map right, which cures the first two.

EXIT STATUS

`1` if there is no such disk. `usage` with no disk.

SEE ALSO

`df`, `mkfs`, `label`; section 14.6.

EXAMPLE

```
/> fsck r
hydrafs label=RAM
free 252 KB of 255 KB
check: lost 0, unmarked 0, twice 0
banks $00-$1f
```

grep

A program: prints the lines that match a pattern.

SYNTAX

```
grep [-chilnsv] [-e] pattern [file ...]
```

ARGUMENTS

pattern

A regular expression.

file

The files to search. Standard input if there are none.

OPTION	EFFECT
`-c`	Print a count of the matching lines, not the lines
`-h`	Don't put the file's name before each line
`-i`	Treat capitals and small letters as the same
`-l`	Print only the names of the files that have a matching line
`-n`	Put each line's number before it
`-s`	Say nothing about files that can't be read
`-v`	Print the lines that don't match
`-e`	The next argument is the pattern, even if it starts with -

DESCRIPTION

Each line of each file is tested, and printed if some part of it matches the pattern. When more than one file is named, each line printed has its file's name and a colon before it.

The pattern's language:

PATTERN	MATCHES
c	The character *c*, if it is not one of those below
`\`*c*	The character *c*, whatever it is
`.`	Any one character
`[abc]`, `[a-z]`	One character of those listed, or in the range
`[^abc]`	One character not listed
`^`, `$`	The start of the line, and its end
e`*`	*e* any number of times, none included
e`+`	*e* once or more
e`?`	*e* once or not at all
e1`|`*e2*	Either
`(`*e*`)`	*e*, as a group

A pattern matches anywhere in a line unless `^` and `$` tie it down. The shell reads the command line first, so a pattern with any of rc's own characters in it goes in quotes.

EXIT STATUS

Empty if a line matched. `no matches` if none did. `1` if a file could not be read. `bad expression`, after a message saying what is wrong, for a pattern that isn't one. `usage` for bad arguments.

A script can therefore test for a match directly: `if(grep -s pattern file >/dev/null) ...`.

SEE ALSO

`sort`, `uniq`, `wc`.

EXAMPLE

```
% echo one >a; echo two >b
% grep o a b; grep -c o a; grep -l t a b
a:one
b:two
1
b
% grep z a; echo status $status
status no matches
% grep '[' a
grep: bad expression: no ]
```

head

A program: prints the first lines of files.

SYNTAX

```
head [-N] [file ...]
```

ARGUMENTS

-N

How many lines: a number. 10 if it is not given.

DESCRIPTION

The first *N* lines of each file are written to standard output, or all of a file that has fewer. With no files, standard input's. Several files are printed one after another with nothing between them.

EXIT STATUS

Empty on success. `1` if a file can't be read. `usage` for an option that isn't a number.

SEE ALSO

`tail`, `more`, `cat`.

hello

A program: a test program, which greets its arguments.

SYNTAX

```
hello [word ...]
```

DESCRIPTION

`hello` is the program `init` runs first when the system starts, to show that programs load and run. It prints `hello,` and its first argument, and ends on purpose with the exit code 7 and the status `bye`.

EXIT STATUS

`bye`, always.

hwtest

A program: restarts the Hydra into its hardware test.

SYNTAX

```
hwtest
```

DESCRIPTION

The machine is reset and the hardware test, which is in the ROM, runs in place of HydraOS (section 5.7). Everything that was running stops, as at any reset.

SEE ALSO

Section 5.7.

hylang

A program: hylang.

SYNTAX

```
hylang [-l] [file]
```

ARGUMENTS

file
: A file of hylang to run. When it has been read to its end, hylang ends.

-l
: Start as a login shell: build the namespace and run the profile, `/lib/hylang/profile.hl`, first (section 8.16).

DESCRIPTION

With no file, hylang starts at its `hylang>` prompt, where each expression typed is evaluated and its value printed after `=>`. `exit` leaves it.

EXIT STATUS

`1` if the file can't be read.

SEE ALSO

Chapter 5 of this book; Chapters 21 and 22.

kill

A program: ends tasks, by their numbers.

SYNTAX

```
kill [-i] task ...
```

ARGUMENTS

task
: A task's number, as `ps` shows it.

`-i`
: Interrupt the task instead of ending it.

DESCRIPTION

Each task is sent a note. Without `-i` it is the kill note, which no program can catch or ignore: the task ends at once, its files are closed and its memory is freed. With `-i` it is the interrupt note, the one CTRL-C sends: a program that handles it can tidy up, or carry on.

EXIT STATUS

Empty if every task was sent its note. `1` if any is not a task: `no such task`, or `invalid argument` for something that is not a number. `usage` with no task.

SEE ALSO

`slay`, `ps`.

EXAMPLE

```
% sleep 30 &
% kill $apid; echo status $status
status
% kill 99
kill: 99: no such task
```

label

A program: shows or sets a disk's label.

SYNTAX

```
label disk [word ...]
```

ARGUMENTS

disk
: The disk's name: the name it has under `/sd`, or the letter of a built-in disk.

word
: The new label: the words, joined with one space between each.

DESCRIPTION

With no words, the disk's label is printed. With words, it is set. The label is a name kept in the file system itself, which `df` shows; it travels with a card.

EXIT STATUS

Empty on success. `1` if the disk isn't there or can't be written. `usage` with no disk.

SEE ALSO

`df`, `mkfs`.

ls

A program: lists files.

SYNTAX

```
ls [-ld] [name ...]
```

ARGUMENTS

name
: A file or a directory. The current directory if none is given.

`-l`
: The long form: a line of details for each.

`-d`
: List a directory itself, not what is in it.

DESCRIPTION

For a directory, each entry in it is listed, one to a line, in the directory's own order, which is the order the entries were made in and not alphabetical. A directory's name has a `/` after it. For a file, its own name is listed.

The long form's line has five parts: the mode, the device, the length in bytes, the date and time the file was last changed, and the name.

```
d-rw-rw-rw- fr        0 2000-01-01 00:00 lib
```

The mode's first character is `d` for a directory and `-` otherwise; the second is `a` for an append-only file. Then come three groups of `r`, `w` and `x`, for reading, writing and running, as Plan 9 lists them; HydraOS has no users, so the three groups are always alike. A file that can't be written, such as anything on the ROM disk, has no `w`. The device is the letter of the driver the file belongs to, and the letter or digit of its disk if it has one: `fr` is the file system on the RAM disk, `fx` on the ROM disk, and `c-` the console.

EXIT STATUS

Empty on success. `1` if a name isn't there; the others are still listed. `usage` for an unknown option.

SEE ALSO

`du`, `df`, `pwd`.

EXAMPLE

```
% echo one >a; mkdir d
% ls
bin/
lib/
a
d/
% ls -ld d nothere
d-rw-rw-rw- fr        0 2000-01-01 00:00 d
ls: nothere: not found
```

mkdir

A program: makes directories.

SYNTAX

```
mkdir [-p] dir ...
```

ARGUMENTS

-p

Make the directories on the way too, as needed, and don't complain about one that exists.

DESCRIPTION

Each directory is made, empty. Without `-p`, the directory it goes in must exist, and the directory itself must not.

EXIT STATUS

Empty on success. `1` if any could not be made: `already exists`, `not found` (the directory it would go in), `read-only`.

SEE ALSO

`rmdir`, `rm`, `ls`.

EXAMPLE

```
% mkdir d; mkdir d; echo status $status
mkdir: d: already exists
status 1
% mkdir p/q
mkdir: p/q: not found
% mkdir -p d p/q; ls p
q/
```

mkfs

A program: makes a new, empty file system on a disk.

SYNTAX

```
mkfs [-fp] disk [word ...]
```

ARGUMENTS

disk
: The disk's name: the name it has under `/sd`, or the letter of a built-in disk.

word
: The new file system's label.

`-f`
: A full format: write the whole of the free map. Without it only what is needed is written, which is much quicker on a large card.

`-p`
: Make the file system in a new partition of its own, after the partitions a card already has, leaving them alone.

DESCRIPTION

Everything on the disk is lost. `mkfs` does not ask first. The disk gets a HydraFS file system with nothing in it, which the storage driver makes.

EXIT STATUS

Empty on success. `1` if there is no such disk, or it can't be done. `usage` with no disk.

SEE ALSO

`fsck`, `label`, `df`; Chapter 14.

mods

A program: lists the modules in the paged ROM.

SYNTAX

```
mods
```

DESCRIPTION

One line is printed for each module, under a heading: the ROM bank or banks it is in, its type, and its name. A module that shares a bank with others is shown as the bank, a `+`, and the page in the bank where it starts.

The types are `program` (run by name), `driver`, `library` (code that programs call), and `boot`: a driver that the kernel starts itself as the system comes up.

Programs that are not modules, the ones in `/rom/bin`, are files on the ROM disk and are not listed here: `ls /rom/bin` shows those.

EXIT STATUS

Empty.

SEE ALSO

`ps`; section 17.10.

more

A program: shows text a screen at a time.

SYNTAX

```
more [file ...]
```

DESCRIPTION

Each file, or standard input if there are none, is shown 22 lines at a time. After each 22 lines `more` prints `--more--` and waits for a line from the console: Return alone shows the next 22, and `q` then Return stops.

The answer is read from the console itself, not from standard input, so `more` works at the end of a pipe: `ls /rom/bin | more`.

Text that ends before 22 lines is printed and `more` ends without asking.

EXIT STATUS

Empty on success. `1` if a file can't be read.

SEE ALSO

`head`, `tail`, `cat`, `edit`.

mv

A program: moves or renames files.

SYNTAX

```
mv from to
mv from ... dir
```

ARGUMENTS

from

A file or directory to move. There can be several if the last name is a directory.

to

The new name. If it is an existing directory, *from* goes into it under its own name.

DESCRIPTION

Within one directory, a move is a rename: the entry's name is changed, and nothing is copied. A file already at the new name is removed first. Files and directories can both be renamed.

To another directory, a file is copied there and the original is then removed. A directory can't be moved to another directory: `mv` refuses, as Plan 9's does. Copy it with `cp -r` and remove the original with `rm -r`.

Moving a file to itself does nothing.

EXIT STATUS

Empty on success. `1` if anything could not be moved. `usage` with fewer than two names.

ERRORS

Each is printed as `mv: name: reason`.

REASON	CAUSE
`not found`	*from* isn't there, or a directory in *to*'s path isn't
`is a directory`	*from* is a directory, and *to* is in another directory
`read-only`	A disk that can't be written

SEE ALSO

`cp`, `rm`.

EXAMPLE

```
% echo one >a; echo two >b; mkdir d e
% mv a c; mv c b; cat b
one
% mv b d; ls d
b
% mv d e; echo status $status
mv: d: is a directory
status 1
```

new-window

A program: opens a window and runs a command in it.

SYNTAX

```
new-window [-g] [command [argument ...]]
```

ARGUMENTS

command

What to run in the window. With none, a shell: the one the system starts in each window.

-g

Put the new window in a group of its own. Without it, the window joins the group of the window the command was typed in (Chapter 10).

DESCRIPTION

A new window is made and shown, and the command is started in it with the window as its standard input, output and error. The command is run by a login shell (`rc -l -c`), so it has the default namespace and the profile's settings, and `$window` in its environment is the new window's number.

`new-window` does not wait: it ends as soon as the command has been started, and the prompt returns in the old window. The new window closes by itself when the command ends.

The command has a note group of its own, so CTRL-C typed in one window doesn't reach the programs of another.

SEE ALSO

Chapter 10; section 13.5.

ns

A program: shows a task's namespace.

SYNTAX

```
ns [task]
```

ARGUMENTS

task

A task's number. This shell's own task if it is left out.

DESCRIPTION

The namespace is printed as the `bind` and `mount` commands that would build it, in the order they were made (section 9.8). It is read from the task's file in `/proc`.

EXIT STATUS

Empty on success. `usage` for more than one argument, or a number that is no task's.

SEE ALSO

`bind`, `mount`, `newns`; Chapter 9.

EXAMPLE

```
/> ns | head -3
bind '#/' /
bind '#/dev' /dev
bind -a '#n' /dev
```

play

A program: plays a song, or a line of music.

SYNTAX

```
play [-l] song [n]
play -o score.mml song.zsm
play [-lx] -m channel mml ...
play [-lx] -c channel note ...
```

ARGUMENTS

song
A ZSM file, a score in MML (a name ending `.mml`), or on a Vera X a WAV file.

n
Play the song's repeating part this many more times.

-l
Play again and again, until CTRL-C.

-o
Don't play: turn the MML score into a ZSM file.

-m *channel*
Play the words that follow as one line of MML on that channel.

-c *channel*
Play the notes that follow as a chord: one on each channel from *channel* on.

-x
Read the MML in the Commander X16's dialect.

DESCRIPTION

`play` reads the music and sends the sound driver its notes at the times they fall due, and ends when the song does. It is the program HyBASIC's `PLAY` statement runs. Chapter 15 describes the sound channels and MML.

EXIT STATUS

Empty when the song has been played. `1` if the file can't be read or the music has a mistake in it, which is reported. `usage` for bad arguments.

SEE ALSO

Chapter 15.

ps

A program: lists the tasks.

SYNTAX

```
ps [-a]
```

ARGUMENTS

`-a`

Show each program's arguments after its name.

DESCRIPTION

One line is printed for each task in use, under a heading:

COLUMN	MEANING
`task`	The task's number, 0 to 15
`state`	`ready` to run; `wait`ing for a child to end; `sleep`ing; waiting for an `event`; or `idle`, a driver with nothing asked of it
`parent`	The task that started it; – for those the kernel started
`cpu`	The processor time it has had, in seconds to a tenth
`group`	Its note group: the tasks a CTRL-C reaches together
`name`	The program it is running

Task 0 is the kernel. The drivers are the tasks at the top of the range, and the programs a user starts are given the lowest free numbers.

Arguments other than `-a` are ignored.

EXIT STATUS

Empty.

SEE ALSO

`top`, `kill`, `slay`, `ns`; section 17.8.

pwd

A program: prints the current directory.

SYNTAX

```
pwd
```

DESCRIPTION

The current directory's full path is written to standard output, with a line end. Arguments are ignored.

A program's current directory is its own, copied from the program that started it; `pwd` shows the one it was started with, which is the shell's.

EXIT STATUS

Empty.

SEE ALSO

`cd`, `ls`.

---

rc

A program: the shell.

SYNTAX

```
rc [-l] [-i] [file [argument ...]]
rc [-l] -c commands
```

ARGUMENTS

file

A script: a file of commands to run. The arguments after it are the script's, `$1`, `$2` and so on, with the file's name as `$0`.

-c *commands*

Run the commands in this one argument, then end.

-l

A login shell: build the default namespace and run the profile, `/rom/lib/profile`, before anything else.

-i

Interactive: print the prompt and carry on after errors, even when the commands come from a file.

DESCRIPTION

With no file and no `-c`, rc reads commands from standard input, printing its prompt first if that is a console. It ends at the end of its input (CTRL-D at the keyboard) or at `exit`.

rc's language is described in Chapter 8, and its own commands at the end of this chapter.

EXIT STATUS

The status of the last command it ran, or the status given to `exit`. `1` if the file can't be read.

SEE ALSO

`forth`, `hylang`, `basic`; Chapter 8.

EXAMPLE

```
/> rc -c 'echo one; exit 3'
one
```

rm

A program: removes files and directories.

SYNTAX

```
rm [-rf] name ...
```

ARGUMENTS

-r

Remove a directory and everything in it, to any depth.

-f

Say nothing about what can't be removed, and don't count it a failure.

DESCRIPTION

Each name is removed. A directory is removed only if it is empty, unless `-r` is given, in which case what is in it is removed first, from the bottom up.

There is no question first and nothing can be brought back.

A file that is open, in any task, is not removed: the system refuses.

EXIT STATUS

Empty on success. `1` if anything could not be removed and `-f` was not given. `usage` for an unknown option.

ERRORS

Each is printed as `rm: name: reason`.

REASON	CAUSE
`not found`	There is no such name
`directory not empty`	A directory with something in it, and no `-r`
`read-only`	The file's disk can't be written
`busy`	The file is open

SEE ALSO

`rmdir`, `mv`, `cp`.

EXAMPLE

```
% mkdir d; echo x >d/f
% rm d; echo status $status
rm: d: directory not empty
status 1
% rm -r d; rm nothere; rm -f nothere; echo status $status
rm: nothere: not found
status
```

rmdir

A program: removes empty directories.

SYNTAX

```
rmdir dir ...
```

DESCRIPTION

Each directory is removed, if it has nothing in it. Unlike `rm`, it will not remove a file, which makes it the safe way to say "this should be an empty directory".

EXIT STATUS

Empty on success. `1` if any could not be removed: `directory not empty`, `not a directory`, `not found`.

SEE ALSO

`rm`, `mkdir`.

slay

A program: ends tasks, by the name of the program they are running.

SYNTAX

```
slay [-i] name ...
```

ARGUMENTS

name

A program's name, as `ps` shows it.

`-i`

Interrupt the tasks instead of ending them.

DESCRIPTION

Every task running a program of that name is sent the kill note, or with `-i` the interrupt note, as `kill` sends them. `slay` never ends itself.

EXIT STATUS

Empty if each name matched a task. `1` if a name matched none: `no such task`. `usage` with no name.

SEE ALSO

`kill`, `ps`.

EXAMPLE

```
% sleep 30 &
% slay sleep; echo status $status
status
% slay sleep
slay: sleep: no such task
```

---

sleep

A program: waits.

SYNTAX

```
sleep seconds
```

ARGUMENTS

seconds

A whole number of seconds.

DESCRIPTION

`sleep` does nothing for that long, and ends. It uses no processor time while it waits. CTRL-C ends it sooner.

EXIT STATUS

Empty when the time is up. `usage` if the argument is missing or is not a whole number: `sleep 0.5` is not accepted.

SEE ALSO

`date`.

---

sort

A program: sorts lines.

SYNTAX

```
sort [-bfnru] [file ...]
```

ARGUMENTS

OPTION	EFFECT
`-b`	Ignore spaces and tabs at the start of each line when comparing
`-f`	Fold: treat capitals and small letters as the same
`-n`	Compare the numbers that the lines start with, not their text
`-r`	Reverse the order
`-u`	Unique: print one of each set of lines that compare equal

DESCRIPTION

All the lines of all the files (or of standard input) are read into memory, sorted, and written to standard output. The order is that of the bytes' values, so every capital comes before every small letter, and digits before both, unless `-f` is given.

With `-n`, a line's number may have a sign and a decimal point; lines are in the order of their numbers' values, so 9 comes before 10, where plain sorting puts `10` first.

EXIT STATUS

Empty on success. `1` if a file can't be read; the others are still sorted. `usage` for an unknown option.

NOTES

The text must fit in memory: `sort` is for files of a size a person would edit, not for megabytes.

SEE ALSO

`uniq`, `grep`.

EXAMPLE

```
% echo 10 >n; echo 9 >>n; echo -5 >>n; echo 2.5 >>n
% sort n | tail -2; sort -n n | tail -2
2.5
9
9
10
```

tail

A program: prints the last lines of a file.

SYNTAX

```
tail [-N] [file]
```

ARGUMENTS

-N

How many lines: a number. 10 if it is not given.

file

One file. Standard input if it is left out.

DESCRIPTION

The file's last *N* lines are written to standard output, or all of it if it has fewer.

The file is read through to its end, with the most recent 16K of it kept in memory, and the lines are found there. What `tail` prints can therefore never be more than the last 16K of the file, however many lines are asked for.

EXIT STATUS

Empty on success. `1` if the file can't be read. `usage` for more than one file, or an option that isn't a number.

SEE ALSO

`head`, `more`.

tee

A program: copies its input to files as well as to its output.

SYNTAX

```
tee [-a] [file ...]
```

ARGUMENTS

-a

Add to the end of each file. Without it, each file is made, or emptied if it exists.

DESCRIPTION

Standard input is copied to standard output and, at the same time, to each file. It is for the middle of a pipe, to keep a copy of what is passing: `ls -l | tee listing | wc -l`.

A file that can't be opened or written is reported, and `tee` goes on with the others and with its output.

EXIT STATUS

Empty on success. `1` if any file could not be written.

SEE ALSO

`cat`.

EXAMPLE

```
% echo x | tee t1 /rom/no; echo status $status; cat t1
tee: /rom/no: read-only
x
status 1
x
```

top

A program: shows which tasks are using the processor.

SYNTAX

```
top
```

DESCRIPTION

Once a second, `top` clears the screen and lists the tasks, each with its number, its state, its name, and the share of the processor it took in that second, as a percentage. It goes on until CTRL-C.

The kernel's share, task 0's, includes the time when nothing needed the processor.

SEE ALSO

`ps`.

touch

A program: makes empty files, or marks files as just changed.

SYNTAX

```
touch name ...
```

DESCRIPTION

A name that doesn't exist is made, as an empty file. A name that does exist has its time of last change set to now, and nothing else about it altered.

EXIT STATUS

Empty on success. `1` if any could not be made or changed.

SEE ALSO

`ls`, `date`.

uniq

A program: removes repeated lines that are next to each other.

SYNTAX

```
uniq [-c] [file]
```

ARGUMENTS

`-c`
: Put before each line the number of times it came in a row.

DESCRIPTION

The file, or standard input, is copied to standard output with each run of identical lines shown once. Only lines that follow one another are compared: the same line appearing again later, after a different one, is printed again. To remove every repeat, sort first: `sort file | uniq`.

A line is compared as its first 255 bytes; a longer line is treated as more than one.

EXIT STATUS

Empty on success. `1` if the file can't be read. `usage` for more than one file or an unknown option.

SEE ALSO

`sort`, `wc`.

EXAMPLE

```
% echo a >u; echo a >>u; echo b >>u; echo a >>u
% uniq -c u
      2 a
      1 b
      1 a
```

WC

A program: counts lines, words and bytes.

SYNTAX

```
wc [-lwc] [file ...]
```

ARGUMENTS

`-l, -w, -c`
: Print only the lines, the words, or the bytes. They can be combined, and the counts always come in that order, whatever order the letters are in.

DESCRIPTION

For each file, or for standard input, `wc` prints its number of lines, of words and of bytes, each in a column seven characters wide, and then the file's name. With more than one file, a line of totals follows.

A line is counted at each line end, so a last line with no line end is not counted. A word is a run of anything but spaces, tabs and line ends.

EXIT STATUS

Empty on success. `1` if a file can't be read. `usage` for an unknown option.

SEE ALSO

`grep`, `sort`, `uniq`.

EXAMPLE

```
% echo one two three >a; echo -n 'no newline' >c
% wc a c
      1       3      14 a
      0       2      10 c
      1       5      24 total
% wc -l <a
      1
```

xd

A program: shows a file's bytes in hexadecimal.

SYNTAX

```
xd [file ...]
```

DESCRIPTION

Each file, or standard input, is printed 16 bytes to a line: the offset of the line's first byte, in hex; the bytes, each as two hex digits; and the same bytes as text, with a `.` for any that isn't a printable character. Each file's offsets start again at 0.

EXIT STATUS

Empty on success. `1` if a file can't be read.

SEE ALSO

`cat`, `cmp`, `dis`.

EXAMPLE

```
/> echo hello | xd
0000000  68 65 6c 6c 6f 0a                                 hello.
```

xmodem

A program: sends or receives a file over the serial port, by the XMODEM protocol.

SYNTAX

```
xmodem -r file
xmodem -s [-k] file
```

ARGUMENTS

-r file

Receive: the terminal program at the other end sends, and what arrives is written to *file*.

`-s` *`file`*
: Send *file* to the terminal program.

`-k`
: Send in blocks of 1K, with a CRC. Without it, in blocks of 128 bytes.

DESCRIPTION

XMODEM is the simplest of the file transfer protocols that terminal programs offer, and any of them can be the other end. Start `xmodem` on the Hydra, then start the transfer in the terminal program within a minute.

While the transfer runs, the serial line carries nothing else: the console holds back the screen's output until it is over, and keys are not acted on.

Receiving, `xmodem` asks first for a CRC and, if the sender doesn't answer to that, for a plain checksum; it takes blocks of 128 bytes or of 1K as they are sent. Each block is acknowledged if it arrived whole and asked for again if it didn't. XMODEM pads the last block; the padding bytes are not written to the file.

When it has finished it prints the file's name and its length.

EXIT STATUS

Empty when the file has been transferred. `1` otherwise, after a message: `too many errors` (a block failed ten times), `cancelled` (the other end, or CTRL-C before the first block), `no answer` (nothing for a minute). `usage` for bad arguments.

NOTES

`/pc` (section 9.9) is a far easier way to move files to and from a PC that runs the Hydra's own PC tool. `xmodem` is for when there is only an ordinary terminal program.

SEE ALSO

Sections 9.9 and 14.7.

The system's own programs

Three programs are started by the system and are not meant to be typed.

`init`
: The first program: it sets the system up and starts the shells, and waits for the tasks that have no other parent (section 7.4).

`wstart`
: Waits for the user to ask for a new window (CTRL-] C), starts a shell in it, and ends; `init` starts it again for the next.

`input`
: The driver for the keyboard and mouse of a Vera X card's input controller. With no controller it says so and ends.

The shell's own commands

These are built into rc. Each changes something about the shell itself, its directory, its variables or its namespace, which a separate program could not do: a program is another task, and changes only its own.

A built-in command's failure sets `$status` to the reason, and prints it as `rc: command: reason`.

. (dot)

```
. file [argument ...]
```

Reads *file* and runs its commands in this shell, as if they had been typed. Unlike running the file as a script, which starts another rc, anything the commands set stays set here: variables, functions, the current directory. Inside the file, `$1`, `$2` and `$*` are the arguments given.

Status: that of the file's last command; `not found` if there is no such file.

bind

```
bind [-a | -b] [-c] new old
```

Makes the directory or file *new* appear at the name *old* in this shell's namespace (Chapter 9). With no option *new* replaces what was at *old*; `-a` adds it after what is there, making a union directory, and `-b` before. `-c` lets files be created in it through the union.

Status: `not found` if either name doesn't exist.

builtin

```
builtin command [argument ...]
```

Runs *command* as rc's built-in of that name, or as a program, passing over any function of the same name. It is how a function that wraps a command calls the real one: `fn cd { builtin cd $*; echo now in` {pwd} }`.

cd

```
cd [dir]
```

Makes *dir* the shell's current directory. With no *dir*, it goes to the directory named by `$home`, or to `/` if `$home` isn't set. Every program the shell starts afterwards begins there.

Status: `not found`; `not a directory`.

eval

```
eval [word ...]
```

Joins its words with spaces and runs the result as a command line. The line is read again from scratch, so variables and quoting in the words are interpreted a second time: `eval echo '$'$v` prints the value of the variable whose name is in `v`. With no words it does nothing.

exec

```
exec command [argument ...]
```

Runs *command* in place of the shell. The shell does not start another task and wait: its own task becomes the program, with the same task number, files and environment, and there is no shell left to return to. Whatever started the shell sees the program's status as the shell's.

exit

```
exit [status ...]
```

Ends the shell. The words given are its exit status; with none, the status is that of the last command.

mount

```
mount [-a | -b] [-c] '#x' old [spec]
```

Attaches a driver's files, named by its device letter, at *old* in this shell's namespace. *spec* chooses among what the driver has: which disk, for the file system's driver. The options are `bind`'s. The device name is quoted because `#` starts a comment in rc.

Status: `no such device` if there is no such driver; `not found` if *old* doesn't exist.

newns

```
newns
```

Throws away this shell's namespace and builds the default one again, from `/lib/namespace`, as a login shell does when it starts. It undoes every `bind`, `mount` and `unmount` the shell has made.

shift

```
shift [n]
```

Drops the first *n* words of `$*`, the arguments of the script or function it is used in, so that `$2` becomes `$1`. *n* is 1 if it is left out. Shifting by more than there are leaves none.

unmount

```
unmount [new] old
```

Takes away what was bound or mounted at *old*. With *new*, only that one member of a union directory is taken out, and the others stay.

Status: `not found` if nothing is bound there.

wait

```
wait [task]
```

Waits for a background command to end. With a task's number, such as `$apid`, it waits for that one; with none, for all of them. A number that is not one of the shell's own children is not an error: there is nothing to wait for.

whatis

```
whatis name ...
```

Says what each name means to the shell, in a form that could be typed back in: a variable as its assignment, a function as its definition, a built-in as `builtin name`, and a program as the full path it would be run from. A name that means nothing prints nothing.

CHAPTER 3

System Calls

A system call is a `jsr` to a fixed address in the jump table at `$F800`. Its arguments go in A, X, Y and the call registers r0 to r15 (two bytes each, in the zero page from `$02`); its results come back in the same places, a 16-bit result in A (low byte) and X (high byte). On return, C = 0 means success. C = 1 means failure, and A holds an error code (Appendix G). A call may change A, X, Y, the flags and r0 to r15, including registers it doesn't use for a result; it never changes the program's own zero page, `$22` to `$7F`.

A call that *can wait* may suspend the task until something happens elsewhere: a key, a disk, another task. A note delivered to the task ends such a wait, and the call returns `E_INTR`.

Each call's name is defined in `hydra.inc` (`/lib/as`), with the constants and error codes used here.

OPEN

System call `$F8C0`: opens a file that exists, and returns a file descriptor for it. It can wait.

SYNTAX

```
            LDR r0, path        ; r0 = the address of the path
            lda #mode
            jsr OPEN
            bcs failed          ; A = the error code
            sta fd              ; A = the file descriptor
```

ARGUMENTS

r0

The address of the path: its bytes, then a zero byte.

A

The mode: one of `O_READ` (0), `O_WRITE` (1) or `O_RDWR` (2), to which can be added `O_TRUNC` (`$10`), to empty the file as it is opened, and `O_NONBLOCK` (`$40`), to make reads and writes of this descriptor return `E_AGAIN` where they would otherwise wait.

RESULT

C = 0, and A = the file descriptor: a number from 0 to 15, the lowest one the task has free.

DESCRIPTION

1. The path is made absolute and clean. One that doesn't start with `/` (or with `#`, a device's own name) has the task's current directory put before it; then `.` elements and doubled slashes are removed, and each `..` removes the element before it. The result may be at most 63 bytes (`PATH_MAX`).
2. The path is looked up in the task's namespace (section 9.5). Where a directory is a union of several (section 9.7), each member is tried in turn, and the first that has the name is the one opened. A member whose device has gone is passed over as if it didn't have the name.
3. The device's driver is asked to open the file in the mode given. If it refuses, its error is the call's.
4. The descriptor is set to the file, at offset 0.

A descriptor has an offset of its own, which `READ`, `WRITE` and `SEEK` move. Descriptors made from it by `DUP`, and those a child task is given by `SPAWN`, share that offset; a second `OPEN` of the same file does not.

Opening a directory with `O_READ` succeeds: it reads as a sequence of 64-byte stat records, one for each file in it. When the directory is a union, the records of all its members follow one another.

`OPEN` never makes a file. `CREATE` does, and opens it.

ERRORS

CODE	NAME	CAUSE
`$20`	`E_NOENT`	No such file, in any member of a union
`$2B`	`E_NODEV`	The path names a device that isn't there
`$0A`	`E_NAMETOOLONG`	The path, made absolute, is longer than 63 bytes
`$26`	`E_MFILE`	The task has all 16 of its descriptors in use
`$27`	`E_NFILE`	All 48 of the system's channels are in use (every open file in every task takes one; `DUP`s share)
`$05`	`E_INTR`	A note arrived while the call was waiting for the driver

A driver adds its own: `E_ROFS` (`$29`) for a write mode on a read-only disk, and `E_ISDIR` (`$23`) for a write mode on a directory.

NOTES

Descriptors 0, 1 and 2 are standard input, output and error, already open when a program starts. A task that closes one of them will get that number from its next `OPEN`.

A descriptor that isn't closed is closed when the task ends.

FROM THE LANGUAGES

LANGUAGE	FORM
HyForth (`lib hydra`)	`sys-open` (*z-addr mode -- fd ior*), the path zero-terminated (`>z` makes one)
hylang	`(sys-open path [mode])`: the descriptor. A failure is an error, its text the path and the system's message
HyBASIC	`SYS "OPEN"`; or, for most purposes, the `OPEN` statement (Chapter 1)

SEE ALSO

`CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `DUP`, `FSTAT`.

EXAMPLE

From hylang. Descriptors 0 to 2 are in use, so the first file opened is 3:

```
hylang> (sys-open "/dev/time" 0)
=> 3
hylang> (sys-open "/dev/nothing" 0)
=> Error: /dev/nothing: not found
hylang> (sys-open "/rom/bin/ls" 1)
=> Error: /rom/bin/ls: read-only
hylang> (sys-open "/ram" 1)
=> Error: /ram: is a directory
hylang> (sys-open "#q/x" 0)
=> Error: #q/x: no such device
```

READ

System call `$F8C9`: reads bytes from an open file into memory. It can wait.

SYNTAX

```
            LDR r0, buf         ; r0 = where the bytes go
            LDR r1, count       ; r1 = how many, at most
            lda fd
            jsr READ
            bcs failed          ; A = the error code
                                ; A/X = how many were read; 0 is the end of the file
```

ARGUMENTS

A

A file descriptor, open with `O_READ` or `O_RDWR`.

r0

The address of a buffer in the task's memory.

r1

The most bytes to read: 0 to 65,535. The buffer must have room for them.

RESULT

C = 0, and A/X = the number of bytes read, which were put in the buffer from its start. The descriptor's offset has moved on by that many.

DESCRIPTION

The kernel asks the file's driver for the bytes at the descriptor's offset, at most 512 at a time (`IO_UNIT`), and repeats until one of three things happens:

- **The count is reached.** The result equals r1.
- **The file ends.** The result is what there was before the end; a read that starts at the end returns 0. A result of 0 is how the end of a file is recognised, and it is not an error.
- **The driver returns fewer bytes than it was asked for.** The kernel stops there and returns them. This is how a device says "that is all for now": the console returns one line, however large the buffer.

A result smaller than r1 therefore does not by itself mean the end of the file. A program that needs exactly *n* bytes must call `READ` again for the remainder until it has them or gets 0.

When the driver has nothing yet (no key typed, an empty pipe), the task waits. With `O_NONBLOCK` set on the descriptor, the call returns `E_AGAIN` instead.

A directory reads as 64-byte stat records (`SR_SIZE`), one for each file in it: ask for a multiple of 64.

ERRORS

CODE	NAME	CAUSE
`$25`	`E_BADF`	A isn't an open descriptor, or it was opened with `O_WRITE`
`$04`	`E_AGAIN`	Nothing to read yet, and the descriptor is non-blocking
`$05`	`E_INTR`	A note arrived while the call was waiting

A driver adds its own: a disk that fails answers `E_IO` (`$2A`).

FROM THE LANGUAGES

LANGUAGE	FORM
HyForth (`lib hydra`)	`sys-read` (*fd c-addr u -- u2 ior*): *u2* bytes were read
hylang	`(sys-read fd count)`: the bytes read, as a string
HyBASIC	`SYS "READ"`; or `INPUT #`, `LINE INPUT #`, `INPUT$` and `GET`

SEE ALSO

`OPEN`, `WRITE`, `SEEK`, `CLOSE`, `PIPE`.

EXAMPLE

The offset moving on, a count cut short by the end of the file, and 0 at the end:

```
hylang> (set {fd} (sys-open "/ram/t" 0))
=> NIL
hylang> (sys-read fd 5)
=> "hello"
hylang> (sys-read fd 100)
=> " world\n"
hylang> (sys-read fd 100)
=> ""
hylang> (sys-close fd)
=> NIL
```

CHAPTER 4

HyForth Words

A cell is 16 bits. A double is two cells, 32 bits, its high cell on top of the stack. A flag is 0 for false and –1 (all bits set) for true. *ior* is 0 for success and a non-zero error code otherwise. Stack pictures follow the Forth standard's notation: *n* signed, *u* unsigned, *d* and *ud* doubles, *c-addr u* the address and length of a string.

Each entry says which word set a word belongs to, and whether it is in the ROM core or needs a library loaded with `lib` first. Where a word is the standard's, the entry describes what HyForth does in the cases the standard leaves open.

>number

Core word set, in the ROM core. Converts the digits at the start of a string, adding them to a double.

SYNTAX

```
>number  ( ud1 c-addr1 u1 -- ud2 c-addr2 u2 )
```

ARGUMENTS

ud1

The number so far: 0 to start a new one.

c-addr1 u1

The text to convert.

RESULT

ud2

ud1 with each converted digit added: for every digit, *ud* × `base` + the digit's value.

c-addr2 u2

What was not converted: the address of the first character that isn't a digit and the number of characters from there to the end. *u2* is 0 if the whole string was digits, and *c-addr2* is then the address just past it.

DESCRIPTION

Characters are taken from the left for as long as each is a digit in the current `base`. The digits are `0` to `9`, then the letters `A` to `Z` for 10 to 35; small letters count as capitals. A character whose value is not less than `base` is not a digit, and conversion stops at it: in decimal `a` stops it, in hex `g` does.

Nothing else is understood. A sign, a prefix (`$`, `#` or `%`) and a decimal point all stop the conversion like any other non-digit; an empty string converts nothing and returns *ud1* unchanged. Signs and prefixes are the text interpreter's business: it deals with them itself, and calls this word's code for the digits.

ERRORS

None. A string with no digits is not an error: *ud2* = *ud1* and *u2* = *u1*.

NOTES

A value that passes 32 bits wraps: only its low 32 bits are kept, and no error is raised. (The standard calls this an ambiguous condition.) To read a number of any length, use the numbers library (`lib numbers`).

`base` can be 2 to 36.

SEE ALSO

`base`, `decimal`, `hex`, `s>d`, `evaluate`.

EXAMPLE

`nip` drops the address, leaving the double (its low cell first, as `.s` shows it) and the count of characters not converted:

```
/> 0 0 s" 123abc" >number nip .s
<3> 123 0 3
/> drop 2drop
/> hex 0 0 s" ff zz" >number nip .s decimal
<3> FF 0 3
/> drop 2drop
/> 0 0 s" -5" >number nip .s
<3> 0 0 2
/> drop 2drop
/> 0 0 s" 99999999999" >number nip .s
<3> 2839 -20518 0
```

CHAPTER 5

hylang Forms

An expression in parentheses is evaluated: its first element names a function or a special form, and the rest are its arguments. A function's arguments are evaluated before it is called, left to right. A **special form** receives its arguments as they were written, and decides for itself which to evaluate and when; each entry for one says exactly that. An expression in braces, a Q-expression, is data: it is not evaluated until something runs it.

A name is looked up in the current **scope**, then in the scope that encloses it, and so on out to the global scope. Each call of a function has a scope of its own, enclosed by the scope the function was made in.

An error is a value: `Error:` and a message. An expression that is given an error as an argument returns it at once, so an error passes outward through whatever contains it until something catches it or it reaches the prompt.

`let`

A special form: evaluates expressions in a new scope, with names bound in it first.

SYNTAX

```
(let {{name [value]} ...} body ...)
(let body)
```

ARGUMENTS

name

A symbol.

value

An expression: what the name is bound to. If it is left out, the name is bound to `NIL`.

body

An expression. It can also be written as a Q-expression, `{print x}`, which is then run as code.

RESULT

The value of the last *body* expression.

DESCRIPTION

1. A new scope is made, enclosed by the scope the `let` is in.

2. The bindings are taken in order. For each, *value* is evaluated **in the new scope**, and *name* is bound to the result there. A value can therefore use the names bound before it in the same `let`; a name bound twice keeps its last value.
3. The *body* expressions are evaluated in order, in the new scope. The values of all but the last are discarded.
4. The new scope is dropped, unless something made in it still refers to it: a function made by `fn` in the body keeps the scope, and can go on using its names after the `let` has returned (a closure).

A name bound by `let` hides one of the same name outside it for as long as the body runs, and the outer one is unchanged afterwards.

Inside the body, `set` binds or changes a name in the `let`'s own scope; `set!` changes a name in whichever scope it was made, which may be outside; and `def` always binds globally.

The last body expression is evaluated in tail position: a function whose body ends in a `let` that ends in a call to itself runs in constant stack, however many times it repeats.

With one argument only, that argument is the body: `(let body)` evaluates it in a new, empty scope. This means `(let {{a 1}})` does not bind anything; its one argument is run as code, and the result is the list `{a 1}`.

ERRORS

MESSAGE	CAUSE
`A 'let' binding is {name value}`	A binding that isn't a list, whose first element isn't a symbol, or that has more than two elements
`'let' supplied too few parameters`	No arguments at all

An error from evaluating a *value* or a *body* expression ends the `let` at once, and is its result.

SEE ALSO

`def`, `set`, `set!`, `fn`, `do`.

EXAMPLE

```
hylang> (let {{x 2} {y (+ x 1)}} (list x y))
=> {2 3}
hylang> (def {x} 10)
=> NIL
hylang> (let {{x 1}} (set {x} 5) x)
=> 5
hylang> x
=> 10
hylang> (def {add5} (let {{n 5}} (fn {a} {+ a n})))
=> NIL
hylang> (add5 1)
=> 6
hylang> n
=> Error: Unbound Symbol 'n'
hylang> (let {a 1} a)
=> Error: A 'let' binding is {name value}
```